Punto 1.1 — Arquitectura monolítica vs. microservicios

Construir una referencia ejecutable antes de comparar arquitecturas

Una acería necesita registrar órdenes de producción y recuperarlas después. Antes de separar el sistema en microservicios, construiremos una aplicación de referencia que reúne entrada HTTP, modelo de órdenes y persistencia en una sola unidad desplegable. Esto permitirá observar qué responsabilidades conviven dentro del monolito y qué decisiones habría que revisar para separar los procesos.

Utilizaremos ASP.NET Core 8, EF Core 8 y SQL Server Express con la solución de `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO/1.1-monolito-vs-microservicios/`. El archivo `AceriaApp.sln`, los proyectos y la documentación ya están preparados. No los recrearemos: la actividad consiste en comprender y completar las clases y endpoints, verificar las dependencias y ejecutar las peticiones y pruebas. En los archivos con bloques comentados se puede escribir el código siguiendo el ejemplo o descomentar el fragmento identificado, pero nunca duplicar ambas versiones.

Al finalizar, la tabla `Ordenes` estará disponible en SQL Server; GET leerá la colección y POST registrará una orden y devolverá su ubicación. El test unitario de `Orden` comprobará el modelo, sin pretender demostrar un comportamiento HTTP que no ejecuta. La decisión de comenzar con un monolito quedará documentada en un ADR y se analizará por sus consecuencias técnicas, no por una preferencia automática a favor o en contra de los microservicios.

Desarrollo práctico

Esta primera práctica completa la solución preparada para la etapa 1.1. Comprobaremos su estructura antes de implementar las clases y endpoints, y después ejecutaremos las comprobaciones previstas.

Ubicación de trabajo: utiliza la carpeta real de tu copia del repositorio en Windows; `C:\cursos\` es únicamente una ruta de ejemplo.

Paso 1 — Localizar el proyecto independiente del punto 1.1

En tu copia del repositorio, abre `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO/1.1-monolito-vs-microservicios/`. No crees otra carpeta con el mismo nombre: ese directorio es la base preparada del ejercicio. En el Explorador de archivos comprueba que contiene `AceriaApp.sln`, las carpetas `src`, `tests`, `docs` y el `README.md` de la etapa. Si falta alguno, revisa que hayas descargado el proyecto completo.

Desde esa misma carpeta, abre **Ver > Terminal** en Visual Studio o una consola de Windows y ejecuta `dotnet --list-sdks`; debe aparecer un SDK `8.0.x`. Si no existe, instala el SDK .NET 8 antes de continuar. Ejecuta `dir` y registra la ruta de trabajo efectiva. Todos los comandos relativos posteriores se ejecutan desde la carpeta que contiene `AceriaApp.sln`, salvo que indiquemos expresamente otra ubicación.

Cada punto constituye una solución autónoma. Se evita copiar la solución anterior sobre archivos nuevos ya preparados y se garantiza que los bloques de código comentados correspondan exactamente a este punto.

Errores comunes: abrir la solución del punto anterior; copiar archivos que sobrescriban los bloques preparados; confundir la carpeta del repositorio con la carpeta del proyecto.

Verificación: `AceriaApp.sln` existe y la terminal está situada dentro de la carpeta de este punto.

Paso 2 — Abrir la solución y comprobar los proyectos heredados

En Visual Studio selecciona **Archivo > Abrir > Proyecto o solución** y elige el archivo `AceriaApp.sln` de la etapa 1.1, no el de otra etapa. Abre la terminal integrada en el mismo directorio y ejecuta `dotnet sln AceriaApp.sln list`; identifica los proyectos `AceriaApp.Monolito.Api` y `AceriaApp.Monolito.Tests`.

En el Explorador de soluciones expande ambos proyectos. Antes de los pasos de activación, los archivos nuevos contienen bloques comentados: que los veas en pantalla no significa que la funcionalidad ya esté activa. Ejecuta `dotnet build AceriaApp.sln -c Release`; comprueba que el código base compila sin activar las incorporaciones. Si aparece un fallo, resuélvelo antes de editar los bloques del alumno.

se identifica el estado heredado ejecutable y se evita confundir una clase comentada con un proyecto todavía no creado.

Errores comunes: abrir la carpeta de otro punto; contar únicamente los proyectos con pruebas activas; descomentar todos los bloques a la vez.

Verificación: la solución abre y compila antes de introducir los cambios del punto.

Paso 3 — Reconocer la estructura preparada del módulo y del punto

En el **Explorador de soluciones** localiza `src/AceriaApp.Monolito.Api` y `tests/AceriaApp.Monolito.Tests`. Ambos ya se entregan creados, por lo que no debes duplicarlos. Desde la terminal situada junto a `AceriaApp.sln` ejecuta `dir src` y `dir tests`. Observa que la estructura separa implementación y pruebas aunque los proyectos compartan la misma solución.

Abre también la carpeta `docs` y localiza el ADR preparado del primer punto; los documentos de diseño no se compilan ni sustituyen al código de la API. Dibuja un árbol con la raíz de la etapa, solución, proyectos, documentación y configuración. Contrasta ese árbol con las rutas reales del repositorio; la ruta `C:\cursos\...` es solo un ejemplo si tú has elegido esa instalación.

Cada módulo tiene su propia carpeta en la raíz del repositorio y cada punto tiene su propia carpeta dentro del módulo. Esta separación permite que cada punto sea autocontenido y ejecutable sin depender de otros.

Errores comunes:

- Confundir el directorio del repositorio con la carpeta de una etapa. Usa comillas cuando una ruta de Windows incluya espacios y trabaja desde el directorio que contiene `AceriaApp.sln`.
- Crear la carpeta con un nombre distinto al slug indicado: el slug debe ser exactamente 1.1-monolito-vs-microservicios.
- Olvidar cambiar a la carpeta del punto con cd: los siguientes comandos se ejecutan en la carpeta del módulo.

Verificación: al ejecutar `dir` desde la carpeta del punto se muestran `AceriaApp.sln`, `src`, `tests` y `docs`; la ubicación absoluta depende de dónde descargaste el repositorio.

Paso 4 — Revisar la solución AceriaApp.sln dentro del punto

Localiza `AceriaApp.sln` en la raíz de la etapa y ábrelo en Visual Studio. Este archivo **ya existe**: no uses «Crear solución vacía» encima del material distribuido. Ejecuta `dotnet sln AceriaApp.sln list` desde esa carpeta e identifica las rutas relativas hacia `src` y `tests`.

Distingue la **solución** (`.sln`), que organiza proyectos, de un **proyecto** (`.csproj`), que declara la compilación y sus dependencias. Guarda el estado inicial sin cambiar el `.sln`. Si la solución no enumera los proyectos que aparecen físicamente, revisa que hayas abierto el fichero de la etapa correcta antes de continuar.

La solución es el contenedor que agrupa todos los proyectos del punto. El nombre AceriaApp se mantiene constante en todos los puntos para que el alumno reconozca la solución al abrir cualquier carpeta.

Errores comunes:

- Ejecutar dotnet new sln sin -n AceriaApp: genera una solución con el nombre de la carpeta (1.1-monolito-vs-microservicios.sln), lo que rompe la convención.
- Ejecutar el comando desde la raíz del repositorio en lugar de dentro de la carpeta del punto: la solución se crea en el lugar equivocado.
- Usar dotnet new solution en lugar de dotnet new sln: el alias sln es el correcto.

Verificación: el archivo AceriaApp.sln debe existir en la carpeta actual.

Paso 5 — Revisar el proyecto de API monolítico de referencia

Desde el Explorador de soluciones expande `src/AceriaApp.Monolito.Api` y abre el archivo ya preparado `AceriaApp.Monolito.Api.csproj`. No invoques una plantilla nueva. Comprueba que el SDK declarado es `Microsoft.NET.Sdk.Web` y el destino es `net8.0`, lo que permite compilar la aplicación ASP.NET Core 8.

Busca los `PackageReference` a EF Core SQL Server, EF Core Design y Swashbuckle; relaciona cada paquete con persistencia, herramientas de migración y documentación de API. Abre `Program.cs` y `appsettings.json` sin activar código aún. Ejecuta `dotnet build src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` y verifica que el esqueleto del proyecto es válido.

El proyecto monolítico de referencia sirve como punto de partida del curso y como término de comparación con los microservicios que se construirán en módulos posteriores. El nombre AceriaApp.Monolito.Api deja claro que se trata del monolito.

Errores comunes:

- Usar dotnet new webapi en lugar de dotnet new web: webapi genera controladores, mientras que el curso usa Minimal API.
- Crear el proyecto fuera de src/: rompe la estructura del repositorio establecida para este curso.
- Usar dotnet new console: genera una aplicación de consola, no una API web.

Verificación: debe existir la carpeta src\AceriaApp.Monolito.Api con los archivos Program.cs, AceriaApp.Monolito.Api.csproj y appsettings.json.

Paso 6 — Comprobar la inclusión del monolito en la solución

En la carpeta raíz de 1.1 abre `AceriaApp.sln` y confirma que `AceriaApp.Monolito.Api` ya aparece en el Explorador de soluciones. Ejecuta `dotnet sln AceriaApp.sln list` para ver la ruta de `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` dentro de la solución.

Inspecciona las propiedades del proyecto desde Visual Studio y comprueba que la ruta coincide con el proyecto físico entregado. No ejecutes `dotnet sln add` si ya está incluido. Si recibes un error de proyecto no cargado, restaura con `dotnet restore AceriaApp.sln` y vuelve a abrirlo; documenta el mensaje antes de intentar recrear archivos.

Comprobar que el proyecto figura en la solución permite verificar que participa en Visual Studio y `dotnet build AceriaApp.sln`, sin añadir una referencia duplicada.

Errores comunes:

- Escribir la ruta sin la extensión .csproj: dotnet sln add requiere la ruta completa al archivo de proyecto.
- Ejecutar el comando desde src/ en lugar de la raíz del punto: la solución no se encuentra.
- Olvidar el nombre de la solución (AceriaApp.sln): el comando falla si hay más de una solución en la carpeta.

Verificación: la salida de dotnet sln list debe mostrar el proyecto AceriaApp.Monolito.Api con su ruta relativa.

Paso 7 — Revisar el proyecto de tests

Localiza la carpeta `tests/AceriaApp.Monolito.Tests` y abre `AceriaApp.Monolito.Tests.csproj`. El proyecto xUnit **se entrega preparado**, no tienes que generarlo con `dotnet new xunit`. Revisa `TargetFramework net8.0`, las referencias a `Microsoft.NET.Test.Sdk`, xUnit, `xunit.runner.visualstudio` y FluentAssertions ya declaradas.

En la misma carpeta encuentra `OrdenTests.cs`, que contiene código didáctico comentado que se activará en el paso 17. No busques `UnitTest1.cs`: no forma parte de este proyecto de referencia. Abre **Prueba > Explorador de pruebas** y confirma que el proyecto es visible. Si no aparecen casos aún, es normal: el test estará disponible después de activar su bloque.

El proyecto de tests preparado permite incorporar una prueba unitaria del modelo en este punto. No constituye todavía una suite completa de integración HTTP y SQL Server.

Errores comunes:

- Usar dotnet new mstest o dotnet new nunit: el curso usa xUnit como framework de pruebas.
- Crear el proyecto en src/ en lugar de tests/: rompe la separación entre código de producción y código de pruebas.
- Usar dotnet new xunit sin -n: genera un proyecto con el nombre de la carpeta (tests), lo que rompe la convención.

Verificación: debe existir la carpeta tests\AceriaApp.Monolito.Tests con el archivo AceriaApp.Monolito.Tests.csproj y el archivo OrdenTests.cs.

Paso 8 — Comprobar la referencia entre el proyecto de pruebas y la API

En Visual Studio, expande las **Dependencias** del proyecto de tests y observa la referencia al proyecto de API monolítica. Abre `tests/AceriaApp.Monolito.Tests/AceriaApp.Monolito.Tests.csproj` y localiza el elemento `ProjectReference` que apunta a `../../src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj`.

Ejecuta `dotnet sln AceriaApp.sln list` y comprueba que ambos proyectos se incluyen en la misma solución. La referencia del test **hacia** la API ya está creada. No agregues otra referencia idéntica. Si el editor señala un problema, comprueba primero la ruta relativa y realiza `dotnet restore AceriaApp.sln`.

La referencia permite que el proyecto de tests acceda a las clases públicas del proyecto monolítico. Sin esta referencia, los tests no pueden instanciar los tipos que deben verificar.

Errores comunes:

- Añadir la referencia en dirección inversa (del monolito a los tests): crea una dependencia circular.
- Olvidar añadir el proyecto de tests a la solución: dotnet test AceriaApp.sln no encontrará el proyecto.
- Escribir la ruta de referencia sin la extensión .csproj: el comando falla.

Verificación: dotnet sln list debe mostrar AceriaApp.Monolito.Api y AceriaApp.Monolito.Tests.

Paso 9 — Comprobar las dependencias NuGet ya declaradas

Desde el proyecto de API abre **Administrar paquetes NuGet** y comprueba en la pestaña **Instalado** `Microsoft.EntityFrameworkCore.SqlServer` versión 8.0.0, `Microsoft.EntityFrameworkCore.Design` 8.0.0 y `Swashbuckle.AspNetCore` 6.6.2. Abre `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` y localiza sus tres referencias existentes. No instales copias duplicadas ni cambies versiones sin una razón documentada.

Desde la raíz de la etapa ejecuta `dotnet list src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj package` y contrasta los nombres y versiones con el `.csproj`. Ejecuta `dotnet restore AceriaApp.sln`; si falla por paquetes no disponibles, revisa el origen NuGet y la conexión antes de modificar los archivos de proyecto.

EF Core es el ORM del curso y SQL Server Express es la base de datos. Design es necesario para las herramientas de migraciones. Swashbuckle genera la documentación OpenAPI.

Errores comunes:

- Instalar una versión preliminar de EF Core (8.0.0-rc.1): el curso usa la versión estable 8.0.0.
- Olvidar el paquete Design: dotnet ef migrations add falla con el error Unable to create an object of type 'AceriaDbContext'.
- Instalar una versión no fijada de Swashbuckle sin comprobar su compatibilidad. El archivo `.csproj` ya declara la versión utilizada en este punto.

Verificación: la salida de dotnet list package debe mostrar Microsoft.EntityFrameworkCore.SqlServer 8.0.0, Microsoft.EntityFrameworkCore.Design 8.0.0 y Swashbuckle.AspNetCore con su versión.

Paso 10 — Configurar la cadena de conexión en appsettings.json

Abre `src/AceriaApp.Monolito.Api/appsettings.json` y localiza la sección `ConnectionStrings`. El fichero se entrega preparado y no debe sustituirse por otro de ejemplo. Comprueba que la clave del JSON se llama exactamente `AceriaDb`, porque el `GetConnectionString("AceriaDb")` que activaremos en `Program.cs` depende de esa coincidencia.

Lee el servidor y el nombre de base de datos configurados. En el ejercicio de M01 se utiliza **SQL Server Express**, no se debe cambiar silenciosamente el proveedor por InMemory ni LocalDB. Valida las comillas, comas y barras invertidas de JSON; si usas un servidor SQL Express distinto, configura la conexión de tu copia local sin subir credenciales reales al repositorio.

{ "ConnectionStrings": { "AceriaDb": "Server=.
\SQLEXPRESS;Database=AceriaDb;Trusted_Connection=True;TrustServerCertificate=True" }, "Logging": { "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" } }, "AllowedHosts": "*" }

La cadena de conexión define el servidor, la base de datos y el modo de autenticación. Trusted_Connection=True usa la identidad de Windows, y TrustServerCertificate=True evita errores de certificado en entornos locales.

Errores comunes:

- Escribir Server=.\SQLEXPRESS con una sola barra invertida: en JSON, \ representa una barra literal; una sola barra produce un error de escape.
- Usar Database=Aceria en lugar de Database=AceriaDb: el curso usa AceriaDb como nombre canónico.
- Olvidar TrustServerCertificate=True: SQL Server Express puede rechazar la conexión por certificado no confiable.

Verificación: al abrir appsettings.json, debe verse la sección ConnectionStrings con la clave AceriaDb y la cadena completa.

Paso 11 — Implementar y comprender Orden.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Dominio/Orden.cs`. El ejemplo muestra el código final de este paso. Escríbelo en el esqueleto o descomenta únicamente el bloque marcado; no dupliques declaraciones ni actives todavía el reto.

En `src/AceriaApp.Monolito.Api/Dominio/Orden.cs`, localiza el marcador de apertura `INICIO PASO 1.1.11` y el de cierre `FIN PASO 1.1.11`. Selecciona **solo las líneas C# comentadas entre esos marcadores**, sin seleccionar las dos líneas de marcador ni el código de un reto de otra etapa. Usa `Ctrl+K, Ctrl+U` para eliminar un nivel de comentario.

Contrasta el resultado con el bloque de referencia que aparece en esta práctica: comprueba el namespace, `Id`, `Cliente`, `Material`, `Cantidad` y `FechaCreacion`, así como sus tipos. Pulsa `Ctrl+S` y ejecuta `dotnet build src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj`. Si quedan líneas con `//` al principio de las declaraciones, revisa qué rango seleccionaste.

```
namespace AceriaApp.Monolito.Api.Dominio;

public class Orden
{
    public Guid Id { get; set; }
    public string Cliente { get; set; } = string.Empty;
    public string Material { get; set; } = string.Empty;
    public int Cantidad { get; set; }
    public DateTime FechaCreacion { get; set; }
}
```

La entidad `Orden` reúne los datos de la operación: `Id` identifica el registro, `Cliente` y `Material` describen la solicitud y `Cantidad` expresa las unidades. Las cadenas se inicializan para evitar referencias nulas. `FechaCreacion` recibirá una marca UTC cuando POST cree la orden. Este modelo inicial todavía no implementa invariantes de negocio: esa diferencia se abordará al introducir un dominio más rico.

La entidad Orden representa la orden de producción. Es la unidad de negocio sobre la que operará el monolito de referencia.

Errores comunes:

- Olvidar el namespace: el archivo no compila porque la clase no se resuelve desde otros archivos.
- Usar int para el Id: el curso usa Guid para evitar colisiones en entornos distribuidos.
- No inicializar Cliente y Material con string.Empty: el compilador emite advertencias de nulabilidad.

Verificación: la clase Orden debe aparecer sin subrayados rojos en el editor y el proyecto debe compilar.

Paso 12 — Implementar y comprender AceriaDbContext.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Infraestructura/AceriaDbContext.cs`. El ejemplo muestra el código final de este paso. Escríbelo en el esqueleto o descomenta únicamente el bloque marcado; no dupliques declaraciones ni actives todavía el reto.

Abre `src/AceriaApp.Monolito.Api/Infraestructura/AceriaDbContext.cs` y localiza el bloque delimitado por `INICIO PASO 1.1.12` y `FIN PASO 1.1.12`. Selecciona únicamente el código comentado interior y pulsa `Ctrl+K, Ctrl+U` una vez. No descomentes los delimitadores ni crees otra clase de contexto.

Comprueba que se importan `Dominio` y `Microsoft.EntityFrameworkCore`, que el constructor recibe `DbContextOptions<AceriaDbContext>` y que existe `DbSet<Orden> Ordenes`. Guarda y compila el proyecto de API. Verifica `HasKey(Id)` y `HasMaxLength(100)` de Cliente y Material; estas reglas configurarán después la migración.

```
using AceriaApp.Monolito.Api.Dominio;
using Microsoft.EntityFrameworkCore;

namespace AceriaApp.Monolito.Api.Infraestructura;

public class AceriaDbContext : DbContext
{
    public AceriaDbContext(DbContextOptions<AceriaDbContext> options)
        : base(options)
    {
    }

    public DbSet<Orden> Ordenes => Set<Orden>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Orden>().HasKey(o => o.Id);
        modelBuilder.Entity<Orden>().Property(o => o.Cliente).HasMaxLength(100).IsRequired();
        modelBuilder.Entity<Orden>().Property(o => o.Material).HasMaxLength(100).IsRequired();
    }
}
```

El `AceriaDbContext` recibe las opciones desde la configuración del host y expone `Ordenes` como conjunto consultable por EF Core. Su `OnModelCreating` establece la clave y exige `Cliente` y `Material`, limitados a 100 caracteres. La configuración describe el esquema, pero la tabla física no existe hasta aplicar la migración; tampoco sustituye la validación de las entradas HTTP.

El `DbContext` representa el modelo persistente y las restricciones utilizadas por EF Core. La creación física de las tablas se realiza al aplicar la migración, no al declarar el contexto.

Errores comunes:

- Olvidar el constructor que recibe DbContextOptions: EF Core no puede instanciar el contexto.
- No configurar las longitudes máximas: SQL Server genera columnas nvarchar(max) que son ineficientes.
- Usar DbSet Ordenes { get; set; } sin => Set(): el curso usa la forma con expresión para evitar setters.

Verificación: la clase AceriaDbContext debe compilar sin errores y el proyecto debe reconocer Microsoft.EntityFrameworkCore.

Interpretación del resultado: `AceriaDbContext` expone `Ordenes` como conjunto persistente. La clase del dominio no debe encargarse de abrir conexiones; esa responsabilidad corresponde a la infraestructura.

Paso 13 — Implementar y comprender Program.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Program.cs`. El ejemplo muestra el código final de este paso. Escríbelo en el esqueleto o descomenta únicamente el bloque marcado; no dupliques declaraciones ni actives todavía el reto.

Abre `Program.cs` del proyecto API y localiza **todos** los bloques que llevan `INICIO PASO 1.1.13`; aparecen en distintas zonas del archivo para respetar el orden de construcción de ASP.NET Core. Descomenta el código interior de cada bloque **por separado** con `Ctrl+K, Ctrl+U`, sin seleccionar los marcadores ni el bloque `RETO 1.1`. Conserva una sola llamada a `WebApplication.CreateBuilder`, `builder.Build` y `app.Run`.

Comprueba el registro de `AceriaDbContext` con `UseSqlServer`, los servicios `AddEndpointsApiExplorer`/`AddSwaggerGen` y los endpoints GET/POST de `/api/v1/ordenes`. Guarda, compila y corrige cualquier subrayado. Si Swagger no se muestra más adelante, revisa el entorno Development; si el compilador no encuentra `Orden`, comprueba primero el paso 11.

```
using AceriaApp.Monolito.Api.Dominio;
using AceriaApp.Monolito.Api.Infraestructura;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AceriaDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("AceriaDb")));
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.MapGet("/api/v1/ordenes", async (AceriaDbContext db) =>
    await db.Ordenes.AsNoTracking().ToListAsync());

app.MapPost("/api/v1/ordenes", async (AceriaDbContext db, Orden orden) =>
{
    orden.Id = Guid.NewGuid();
    orden.FechaCreacion = DateTime.UtcNow;
    db.Ordenes.Add(orden);
    await db.SaveChangesAsync();
    return Results.Created($"/api/v1/ordenes/{orden.Id}", orden);
});

app.Run();
```

En `Program.cs`, primero se registran EF Core y los servicios OpenAPI, después se construye la aplicación y se habilita Swagger únicamente en Development. GET usa `AsNoTracking` porque solo lee las órdenes. POST genera un GUID y una marca UTC, agrega la entidad, ejecuta `SaveChangesAsync` y devuelve `201 Created` con la ubicación del recurso. La consulta individual por ID queda como ampliación posterior; `app.Run()` inicia la escucha HTTP.

registrar el DbContext en el contenedor de dependencias permite que los endpoints lo reciban por inyección. AddEndpointsApiExplorer y AddSwaggerGen habilitan OpenAPI. Los endpoints GET y POST permiten consultar y crear órdenes.

Errores comunes:

- Escribir GetConnectionString("Aceria") en lugar de GetConnectionString("AceriaDb"): devuelve null y EF Core lanza InvalidOperationException.
- Olvidar el using Microsoft.EntityFrameworkCore: UseSqlServer no se resuelve.
- Olvidar app.Run(); al final: el proceso termina inmediatamente y no escucha peticiones.

Verificación: Program.cs debe compilar sin errores y mostrar las llamadas a AddDbContext, AddEndpointsApiExplorer, AddSwaggerGen, los dos MapGet/MapPost y la llamada a app.Run().

Interpretación del resultado: Al ejecutar el host se resuelven dependencias por inyección. GET consulta órdenes existentes y POST crea una nueva; las respuestas HTTP dependen del resultado de EF Core.

Paso 14 — Examinar la migración inicial incluida

En `src/AceriaApp.Monolito.Api/Infraestructura/Migraciones`, abre `202610140001_Inicial.cs.txt` y el snapshot `AceriaDbContextModelSnapshot.cs.txt`. Están preparados como texto para evitar que la migración se active anticipadamente. Cambia **ambas extensiones** de `.cs.txt` a `.cs` dentro de la carpeta de trabajo, sin crear una migración nueva ni alterar el contenido generado.

Abre la clase `Inicial`, revisa `Up` y `Down` y comprueba que `Up` crea `Ordenes` con Id, Cliente, Material, Cantidad y FechaCreacion. Compila y ejecuta, desde la carpeta donde está `AceriaApp.sln`, `dotnet ef migrations list --project src/AceriaApp.Monolito.Api --startup-project src/AceriaApp.Monolito.Api`. Si no está instalado `dotnet-ef`, instala la herramienta versión 8.

La migración se entrega versionada para que todos los alumnos apliquen el mismo esquema; generar otra con idéntico nombre duplicaría la operación.

Errores comunes: ejecutar `migrations add Inicial` otra vez; cambiar la migración heredada; buscar una marca temporal arbitraria.

Verificación: los dos archivos ahora tienen extensión `.cs`, la solución compila y `202610140001_Inicial` figura en la lista de migraciones.

Interpretación del resultado: La migración versionada describe la tabla que EF Core aplicará al servidor. Una clase `DbContext` por sí sola no crea físicamente la base de datos.

Paso 15 — Aplicar la migración a SQL Server Express

Comprueba que el servicio SQL Server Express `SQLEXPRESS` está instalado y en ejecución. Revisa la cadena de conexión de `appsettings.json` y no la confundas con la usada en M02. En la terminal de la raíz del punto ejecuta `dotnet ef database update --project src/AceriaApp.Monolito.Api --startup-project src/AceriaApp.Monolito.Api`. No lo lances desde una carpeta arbitraria sin indicar el proyecto.

Lee el resultado: debe aplicar `202610140001_Inicial` o informar de que ya está aplicada. Si aparece un error de autenticación o conexión, corrige la instancia antes de seguir. En SQL Server Management Studio o el Explorador de objetos de SQL Server, conecta a `.\SQLEXPRESS` y abre **Databases > AceriaDb > Tables > dbo.Ordenes**. Inspecciona las cinco columnas creadas.

dotnet ef database update aplica la migración y crea la base de datos AceriaDb con la tabla Ordenes.

Errores comunes:

- No tener SQL Server Express arrancado: falla con A network-related or instance-specific error.
- Usar una cadena de conexión con Database=Aceria en lugar de AceriaDb: crea una base de datos distinta.
- No tener permisos de CREATE DATABASE: falla con CREATE DATABASE permission denied.

Verificación: la tabla dbo.Ordenes debe existir en la base de datos AceriaDb con las columnas Id, Cliente, Material, Cantidad y FechaCreacion.

Paso 16 — Ejecutar la API y verificar los endpoints

Con la migración aplicada y SQL Express arrancado, abre la terminal en la raíz de 1.1 y ejecuta `dotnet run --project src/AceriaApp.Monolito.Api`. Copia la URL **real** que el servidor muestra como `Now listening on:`; puede ser HTTP o HTTPS según el perfil y la configuración. No presupongas que se muestran ambas.

Navega a `<URL_REAL>/swagger` en Development y abre GET `/api/v1/ordenes` mediante **Try it out > Execute**. En una base vacía, comprueba 200 y el array `[]`. Ejecuta también el POST con una orden válida y después repite GET para ver la persistencia. Detén el proceso con `Ctrl+C` o con el botón de Visual Studio antes de continuar.

ejecutar la API verifica que el proyecto compila, que la configuración es correcta y que los endpoints están expuestos.

Errores comunes:

- No confiar en el certificado de desarrollo: el navegador bloquea la página. Ejecutar dotnet dev-certs https --trust una vez.
- Usar el puerto equivocado: la salida de dotnet run indica los puertos exactos.
- Olvidar Ctrl+C antes de continuar: el proceso queda en segundo plano y bloquea el siguiente dotnet run.

Verificación: Swagger UI debe mostrar los dos endpoints y permitir ejecutar el GET (que devuelve una lista vacía []).

Interpretación del resultado: La respuesta `[]` de GET antes de crear órdenes confirma la ruta y consulta inicial. Tras POST, un nuevo GET debe devolver el registro almacenado; ese comportamiento es distinto de comprobar solo la compilación.

Paso 17 — Implementar y comprender OrdenTests.cs

El archivo preparado es `tests/AceriaApp.Monolito.Tests/OrdenTests.cs`. El ejemplo muestra el código final de este paso. Escríbelo en el esqueleto o descomenta únicamente el bloque marcado; no dupliques declaraciones ni actives todavía el reto.

En `tests/AceriaApp.Monolito.Tests/OrdenTests.cs` localiza `INICIO PASO 1.1.17` y `FIN PASO 1.1.17`. Selecciona las líneas interiores que contienen imports, clase de test y método `Orden_ConDatosValidos_SeCreaCorrectamente`; pulsa `Ctrl+K, Ctrl+U`, conservando los delimitadores comentados.

Guarda y revisa las aserciones: el test instancia `Orden` y verifica Cliente y Cantidad; **no** constituye todavía una prueba HTTP de la ruta GET. Comprueba en `AceriaApp.Monolito.Tests.csproj` que `FluentAssertions` 6.12.2 ya está instalado. No ejecutes otro `dotnet add package` para duplicarlo; compila y abre el Explorador de pruebas.

```
using AceriaApp.Monolito.Api.Dominio;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Monolito.Tests;

public class OrdenTests
{
    [Fact]
    public void Orden_ConDatosValidos_SeCreaCorrectamente()
    {
        var orden = new Orden
        {
            Id = Guid.NewGuid(),
```

```
            Cliente = "ACX-001",
            Material = "CHAPA-3MM",
            Cantidad = 10,
            FechaCreacion = DateTime.UtcNow
        };

        orden.Cliente.Should().Be("ACX-001");
        orden.Cantidad.Should().Be(10);
    }
}
```

`OrdenTests` crea una orden con datos conocidos y comprueba sus propiedades mediante xUnit y FluentAssertions. Es una prueba unitaria del modelo, no un ensayo del endpoint HTTP ni de la persistencia en SQL Server. Esta distinción evita concluir que GET devuelve `404` o que POST guarda los datos únicamente porque una clase puede construirse correctamente.

La prueba unitaria verifica que la entidad Orden se instancia correctamente con datos válidos. FluentAssertions proporciona una sintaxis de aserción más legible que Assert.

Errores comunes:

- No añadir el paquete FluentAssertions: Should() no se resuelve.
- Usar Assert.Equal en lugar de Should().Be(): el curso usa FluentAssertions como convención.
- Olvidar el using AceriaApp.Monolito.Api.Dominio;: la clase Orden no se resuelve.

Verificación: OrdenTests.cs debe compilar sin errores y mostrar el método Orden_ConDatosValidos_SeCreaCorrectamente.

Interpretación del resultado: La prueba unitaria de `Orden` comprueba la creación y los valores de la entidad. No hace una petición HTTP ni comprueba SQL Server.

Paso 18 — Ejecutar los tests

Desde la carpeta que contiene `AceriaApp.sln`, ejecuta `dotnet test AceriaApp.sln` con los bloques de 1.1.11–1.1.17 ya activados. Comprueba que la salida informa de **1 prueba superada, 0 fallidas** en la configuración básica, y localiza su nombre `Orden_ConDatosValidos_SeCreaCorrectamente`.

Si no se descubre el test, revisa que `[Fact]` y la clase estén descomentados, que el proyecto de pruebas esté incluido en la solución y que xUnit se restauró. Guarda la salida de test como evidencia y no ejecutes el reto opcional de GET individual hasta que hayas terminado el ejercicio base.

ejecutar los tests verifica que el proyecto de pruebas está correctamente configurado y que la entidad Orden funciona como se espera.

Errores comunes:

- Ejecutar dotnet test sin especificar la solución: puede ejecutar tests de otros proyectos.
- No compilar antes de ejecutar: dotnet test compila por defecto, pero si hay errores de compilación, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado, aunque dotnet test no arranca la API.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 1, Skipped: 0, Total: 1.

Paso 19 — Revisar el ADR de la decisión arquitectónica

Abre `docs/ADR-001-monolito-vs-microservicios.md` ya incluido en 1.1, usando el explorador de archivos o **Mostrar todos los archivos** en Visual Studio. Lee en orden Estado, Contexto, Decisión, Alternativas consideradas y Consecuencias. Relaciona la decisión de comenzar con un monolito con la transacción EF Core y los endpoints que has probado.

Anota en una frase qué evidencia técnica apoya la decisión (una API, una conexión y una unidad desplegable) y qué limitaciones tendría extraer microservicios antes de justificar sus fronteras. No sobrescribas el ADR original ni crees una copia nueva. Comprueba que el fichero en `docs` coincide con el texto de referencia del manual y permanece versionado.

```
# ADR-001: Elección entre monolito y microservicios

## Estado
Aceptado

## Contexto
El proyecto AceriaApp debe gestionar órdenes de producción, inventario de materias primas y registro de
clientes. El equipo inicial es de dos personas y la frecuencia de despliegue prevista es semanal.

## Decisión
Se adopta un monolito modular como punto de partida, organizado por capas (presentación, aplicación, dominio,
infraestructura), con la posibilidad de extraer microservicios en módulos posteriores.

## Alternativas consideradas
- Microservicios desde el inicio: descartado por el coste operativo y el tamaño reducido del equipo.
- Monolito sin separación de capas: descartado por la dificultad de evolución futura.

## Consecuencias
- Positivas: simplicidad operativa, transacciones ACID, despliegue único.
- Negativas: acoplamiento del ciclo de vida, escalado homogéneo.
- Mitigación: modularización interna estricta para facilitar la extracción futura de servicios.
```

el ADR documenta la decisión arquitectónica, las alternativas consideradas y las consecuencias. Es un artefacto obligatorio en cada punto del curso.

Errores comunes:

- Omitir la sección Alternativas consideradas: el ADR pierde valor porque no muestra el análisis.
- No incluir las consecuencias negativas: el ADR queda sesgado.
- Guardar el ADR en la raíz del punto en lugar de docs/: rompe la estructura del repositorio.

Verificación: el archivo docs\ADR-001-monolito-vs-microservicios.md debe existir con las cinco secciones.

Paso 20 — Revisar el README del punto

Abre el `README.md` de la **carpeta 1.1**, no el README de la raíz de todo el curso. Comprueba que explica el objetivo de 1.1, el estado de los proyectos, cómo ejecutarlos y qué documentación acompaña a la solución.

Sigue literalmente las rutas de ejecución allí descritas y comprueba que se refieren a la misma `AceriaApp.sln` y cadena de conexión que acabas de utilizar. Escribe en tus notas si hay alguna diferencia entre la ejecución que acabas de realizar y el README; no edites el documento distribuido salvo que el ejercicio te pida expresamente corregirlo.

```
# Punto 1.1 — Arquitectura monolítica vs. microservicios

## Objetivo
Comprender las diferencias entre monolito y microservicios y crear la estructura inicial del proyecto
AceriaApp.
```

```
## Estado del proyecto
- Solución `AceriaApp.sln` con dos proyectos: `AceriaApp.Monolito.Api` y `AceriaApp.Monolito.Tests`.
- API Minimal con endpoints `GET /api/v1/ordenes` y `POST /api/v1/ordenes`.
- Persistencia con EF Core 8.0.0 y SQL Server Express (`AceriaDb`).
- Documentación OpenAPI con Swagger.
- Un test unitario de la entidad `Orden`.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.

## Qué se ha añadido respecto al punto anterior
Este es el primer punto del curso. No hay punto anterior.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md)
```

el README explica el objetivo, el estado, la ejecución y los añadidos del punto. Es obligatorio en cada punto del curso.

Errores comunes:

- Omitir la sección Qué se ha añadido respecto al punto anterior: el alumno pierde la trazabilidad.
- No enlazar el ADR: el README queda incompleto.
- Escribir el README en la raíz del repositorio en lugar de la carpeta del punto.

Verificación: el archivo README.md debe existir en la carpeta del punto con las cinco secciones.

Paso 21 — Compilar la solución completa

Abre una terminal en la carpeta de 1.1 (donde está `AceriaApp.sln`) y ejecuta `dotnet build AceriaApp.sln -c Release`. Comprueba que el compilador utiliza los proyectos de API y tests del mismo punto y que finaliza con `Build succeeded` y **0 errores**.

Si hay errores, lee el primer mensaje de compilación y localiza archivo y número de línea; vuelve a comprobar los marcadores del paso correspondiente antes de editar algo más. Guarda una copia del resumen de compilación y verifica que no has introducido dependencias o proyectos que pertenezcan a los puntos 1.2–1.5.

compilar la solución completa verifica que todos los proyectos se resuelven correctamente y que no hay errores de dependencias.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 22 — Ejecutar todos los tests de la solución

Con la solución recién compilada, ejecuta `dotnet test AceriaApp.sln -c Release --no-build` desde la carpeta de 1.1. Comprueba que el test de `Orden` aparece y que el total de la etapa básica es **1 superado y 0 fallidos**.

Si recibes el error de no encontrar ensamblados, ejecuta primero el `dotnet build -c Release` del paso anterior y no mezcles configuraciones Debug/Release. Relaciona la prueba unitaria con el comportamiento observado en Swagger: una aserción sobre la clase Orden no sustituye la comprobación de HTTP y SQL Server.

ejecutar todos los tests verifica que la solución compila y que las pruebas pasan. --no-build reutiliza la compilación anterior para ahorrar tiempo.

Errores comunes:

- Ejecutar dotnet test sin --no-build después de dotnet build: recompila innecesariamente.
- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros puntos en la solución: la solución solo contiene los proyectos de este punto.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 1, Skipped: 0, Total: 1.

Paso 23 — Revisar el .gitignore de la solución

En la raíz de 1.1 abre el archivo ya preparado `.gitignore` mediante el Explorador de archivos o un editor que muestre archivos ocultos. Revisa que excluye carpetas temporales `bin/`, `obj/`, `.vs/`, los ajustes de usuario y resultados generados, en vez de ignorar el código o la documentación del proyecto.

Desde la carpeta en que tengas el repositorio Git de trabajo ejecuta `git status --short` después de compilar; comprueba que no ofrece binarios ni archivos intermedios para commit. No sobreescribas el `.gitignore` si ya contiene esas exclusiones. Explica por qué el código fuente y los tests sí deben versionarse, pero los resultados compilados no.

bin/ obj/ *.user *.suo .vs/ TestResults/

el .gitignore evita que los artefactos de compilación y los archivos de usuario de Visual Studio se incluyan en el control de versiones.

Errores comunes:

- Olvidar bin/ y obj/: el repositorio se llena de artefactos.
- Escribir bin sin la barra final: el patrón no coincide con las carpetas dentro de los proyectos.
- Escribir el .gitignore en la raíz del repositorio en lugar de la carpeta del punto: afecta a todos los puntos.

Verificación: git status no debe mostrar archivos dentro de bin/ ni obj/.

Paso 24 — Inicializar Git, hacer el commit inicial y verificar el estado final

Utiliza una **copia personal de trabajo**, no el repositorio del profesor ni la rama oficial. Desde el directorio raíz de esa copia ejecuta `git status` para comprobar si Git ya está inicializado. Solo si trabajas en una carpeta sin repositorio Git, inicializa con `git init`; si la carpeta ya proviene de `git clone`, **no** crees un repositorio anidado dentro de la etapa 1.1.

Ejecuta `git add` sobre los archivos que has trabajado y revisa `git diff --cached --stat` para evitar incluir compilados o secretos de conexión. Crea el commit con `git commit -m "Punto 1.1: estructura inicial del proyecto AceriaApp"`. Comprueba con `git log -1 --oneline` que existe ese commit y con `git status` que no quedan cambios imprevistos. Si el repositorio de trabajo contiene más módulos, confirma que no has incluido archivos de puntos futuros.

El commit inicial registra el estado del punto 1.1 en el historial de Git. Cada punto del curso tendrá su propio commit para facilitar la comparación.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla. Ejecutar git config --global user.name "Nombre" y git config --global user.email "correo@dominio.com".
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore del paso 23 lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: En tu copia personal, `git log -1 --oneline` contiene el commit realizado y `git status` no muestra cambios inesperados de compilación.

Nota de trazabilidad: en una descarga del repositorio puede no haber cambios nuevos que confirmar. Antes de ejecutar `git commit`, usar `git status` y confirmar únicamente los archivos efectivamente modificados durante el recorrido. No ejecutar `git init` dentro de cada punto: la raíz del repositorio ya contiene el historial.

Reto de ampliación: solución guiada

Problema a resolver: añadir un endpoint GET /api/v1/ordenes/{id} que devuelva una orden concreta por su identificador, y un test unitario que verifique la ausencia de la orden en la capa de persistencia cuando el identificador no existe.

Etapa 1 — En Program.cs, añadir después del MapGet existente:

```
app.MapGet("/api/v1/ordenes/{id:guid}", async (Guid id, AceriaDbContext db) =>
{
    var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id);
    return orden is not null ? Results.Ok(orden) : Results.NotFound();
});
```

Etapa 2 — Explicar línea por línea:

- **Línea 1:** `app.MapGet("/api/v1/ordenes/{id:guid}", ...)` — define un endpoint GET con un parámetro de ruta id restringido a Guid.
- **Línea 2:** `async (Guid id, AceriaDbContext db) =>` — el parámetro id se enlaza desde la ruta y db se inyecta desde el contenedor.
- **Línea 3:** `var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id);` — consulta la primera orden cuyo Id coincide, sin tracking.
- **Línea 4:** `return orden is not null ? Results.Ok(orden) : Results.NotFound();` — devuelve 200 con la orden o 404 si no existe.

Etapa 3 — Guardar el archivo con Ctrl+S.

Etapa 4 — En tests\AceriaApp.Monolito.Tests, añadir un archivo OrdenEndpointsTests.cs con:

```
using AceriaApp.Monolito.Api.Dominio;
using AceriaApp.Monolito.Api.Infraestructura;
using FluentAssertions;
using Microsoft.EntityFrameworkCore;
using Xunit;

namespace AceriaApp.Monolito.Tests;

public class OrdenEndpointsTests
{
    [Fact]
    public async Task GetOrdenPorId_Inexistente_DevuelveNull()
    {
        var options = new DbContextOptionsBuilder<AceriaDbContext>()
```

```
            .UseInMemoryDatabase("TestDb")
            .Options;
        await using var db = new AceriaDbContext(options);
        var id = Guid.NewGuid();

        var orden = await db.Ordenes.AsNoTracking().FirstOrDefaultAsync(o => o.Id == id);

        orden.Should().BeNull();
    }
}
```

Etapa 5 — Añadir el paquete Microsoft.EntityFrameworkCore.InMemory:

dotnet add tests\AceriaApp.Monolito.Tests\AceriaApp.Monolito.Tests.csproj package Microsoft.EntityFrameworkCore.InMemory --version 8.0.0

Etapa 6 — Ejecutar dotnet test AceriaApp.sln y verificar que pasan los dos tests.

Etapa 7 — Verificar en Swagger que aparece el nuevo endpoint GET /api/v1/ordenes/{id}.

Etapa 8 — Hacer commit con git commit -m "Reto 1.1: endpoint GET por id y test de 404".

Verificación: Swagger debe mostrar tres endpoints y dotnet test debe mostrar Passed: 2.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta raíz C:\cursos\AceriaApp\ con el README.md raíz del repositorio.
- La carpeta modulo-1-introduccion-microservicios/1.1-monolito-vs-microservicios/ con la solución AceriaApp.sln, los proyectos src/AceriaApp.Monolito.Api y tests/AceriaApp.Monolito.Tests, la carpeta docs/ con el ADR-001, el README.md del punto y el .gitignore.
- La API ejecutándose con Swagger mostrando GET /api/v1/ordenes, POST /api/v1/ordenes y (tras el reto) GET /api/v1/ordenes/{id}.
- La tabla dbo.Ordenes creada en SQL Server Express dentro de la base de datos AceriaDb.
- Los tests pasando con dotnet test AceriaApp.sln.
- Un commit inicial en Git con el mensaje Punto 1.1: estructura inicial del proyecto AceriaApp.
- Un archivo .gitignore que excluye bin/, obj/, *.user, *.suo, .vs/ y TestResults/.

Conclusión y enlace al siguiente punto

El punto 1.1 ha establecido los fundamentos conceptuales de la comparación entre monolito y microservicios, y ha creado la estructura raíz del repositorio AceriaApp junto con la estructura inicial del punto. Esta base permite avanzar hacia el punto 1.2, donde se analizarán con detalle las ventajas e inconvenientes de cada estilo arquitectónico y se medirá el impacto de cada decisión sobre el proyecto creciente.

Fin del punto 1.1.

Punto 1.2 — Ventajas e inconvenientes

Comprender el coste real de separar una aplicación

Antes de introducir otro proceso en AceriaApp conviene estudiar las ventajas y los costes de cada frontera de despliegue. En lugar de asignar una puntuación universal al monolito y a los microservicios, describiremos dependencias, tablas compartidas y complejidad mediante indicadores sencillos. Estos valores nos ayudan a formular preguntas sobre el diseño, pero no pueden determinar por sí solos si una extracción merece la pena.

Trabajaremos con la solución preparada del punto 1.2, que incorpora todo lo construido en 1.1. Conservaremos la API y su persistencia mientras implementamos `MetricasAcoplamiento`, `MetricasServicio` y `AnalizadorMonolito`, acompañados de pruebas aisladas. Cada cálculo tendrá ejemplos concretos y se explicará qué representa su resultado y qué aspectos importantes, como las transacciones o la autonomía operativa, quedan fuera de su alcance.

Al terminar debemos poder recalcular un indicador al cambiar sus entradas, explicar por qué cero proyectos requiere un tratamiento especial y distinguir una clasificación programada de una decisión arquitectónica contrastada. El análisis documentado no sustituye la evidencia obtenida de la aplicación en ejecución.

Desarrollo práctico

Ejercicio: Añadir al punto 1.1 un documento de análisis comparativo de ventajas e inconvenientes en docs/, una clase de cálculo de métricas de acoplamiento en el proyecto monolítico y un test unitario que verifique las métricas. El punto 1.2 parte del contenido del punto 1.1 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: utiliza la carpeta real de tu copia del repositorio en Windows; `C:\cursos\` solo representa una ubicación posible.AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Localizar el proyecto independiente del punto 1.2

Abre `PROYECTO/1.2-ventajas-inconvenientes` de tu copia del repositorio. No modifiques ni sustituyas la carpeta 1.1: cada punto mantiene una solución independiente con los avances heredados. Comprueba en esa carpeta la presencia de `AceriaApp.sln`, `src`, `tests`, `docs` y `README.md`. Los archivos específicos de 1.2 están preparados, aunque parte de sus declaraciones siguen comentadas.

Abre la terminal en la carpeta que contiene `AceriaApp.sln` y ejecuta `dotnet --list-sdks`; verifica .NET 8 y que el entorno de trabajo es el de 1.2. Anota la ruta real de esta etapa y confirma que en su carpeta `tests` hay proyectos heredados y uno nuevo para métricas. No crees proyectos con nombres duplicados.

Cada punto constituye una solución autónoma. Se evita copiar la solución anterior sobre archivos nuevos ya preparados y se garantiza que los bloques de código comentados correspondan exactamente a este punto.

Errores comunes: abrir la solución del punto anterior; copiar archivos que sobrescriban los bloques preparados; confundir la carpeta del repositorio con la carpeta del proyecto.

Verificación: `AceriaApp.sln` existe y la terminal está situada dentro de la carpeta de este punto.

Paso 2 — Abrir la solución y comprobar los proyectos heredados

Abre `AceriaApp.sln` de 1.2 en Visual Studio mediante **Archivo > Abrir > Proyecto o solución**, y no el de 1.1 que ya has terminado. Ejecuta `dotnet sln AceriaApp.sln list` desde la carpeta de 1.2 e identifica `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests` y `AceriaApp.Metricas.Tests`.

En `src`, comprueba que el código de `Dominio` e `Infraestructura` heredado está activo y que los nuevos tipos de `Metricas` aparecen todavía comentados con marcadores de paso. Ejecuta `dotnet build AceriaApp.sln` antes de activar los nuevos bloques para tener una línea base reproducible, y guarda el resultado de la compilación.

se identifica el estado heredado ejecutable y se evita confundir una clase comentada con un proyecto todavía no creado.

Errores comunes: abrir la carpeta de otro punto; contar únicamente los proyectos con pruebas activas; descomentar todos los bloques a la vez.

Verificación: la solución abre y compila antes de introducir los cambios del punto.

Paso 3 — Crear la carpeta docs del punto 1.2 si no existe

En el Explorador de archivos abre `1.2-ventajas-inconvenientes/docs` y comprueba que la carpeta ya existe; no es necesario crearla de nuevo. Verifica que `ADR-001-monolito-vs-microservicios.md` conserva el documento heredado y localiza los nuevos `analisis-ventajas-inconvenientes.md` y `ADR-002-metricas-acoplamiento.md`.

Abre los documentos en un visor Markdown de Visual Studio y distingue los contenidos heredados de los documentos exclusivos de 1.2. Comprueba que los archivos están en la carpeta `docs` de **esta etapa**; no edites el original de 1.1 para documentar las ventajas e inconvenientes de 1.2.

El punto 1.2 parte del contenido del punto 1.1, que ya incluye la carpeta docs con el ADR-001. No se modifica ningún archivo existente.

Errores comunes:

- Crear la carpeta docs si ya existe: mkdir falla con Cannot create a file when that file already exists.
- Modificar el ADR-001 existente: rompe la regla de no modificar archivos del punto anterior.
- Eliminar el ADR-001 para crear uno nuevo: pierde la trazabilidad del punto 1.1.

Verificación: la carpeta docs existe y contiene el archivo ADR-001-monolito-vs-microservicios.md sin modificaciones.

Paso 4 — Revisar el documento preparado: analisis-ventajas-inconvenientes.md

Abre `docs/analisis-ventajas-inconvenientes.md` que ya se entrega preparado y lee su primera sección para identificar el propósito del análisis comparativo. Recorre las ventajas y desventajas del monolito y de los microservicios, separando costes de despliegue, acoplamiento, autonomía de los equipos y consistencia de datos.

Relaciona cada criterio con el monolito AceriaApp existente: señala qué operaciones comparten la misma base de datos y por qué la extracción de servicios requeriría contratos. Contrasta las secciones con el documento reproducido en la práctica; anota una decisión justificada para el caso de acería sin alterar el archivo preparado del profesor.

```
# Análisis de ventajas e inconvenientes — Punto 1.2
```

```
## Marco de análisis

Cuatro dimensiones: despliegue, escalado, organización y consistencia.
Consideración transversal: coste operativo total.

## Ventajas del monolito

### Despliegue
- Pipeline único.
- Despliegue atómico.
- Reversión atómica.

### Consistencia
- Transacciones ACID sobre una única base de datos.
- Integridad referencial declarativa.
- Consultas con JOIN sin llamadas de red.

### Escalado
- Réplica del artefacto completo.
- Configuración idéntica para todas las réplicas.

### Organización
- Un único repositorio.
- Un único equipo responsable.
- Decisiones técnicas en una sola conversación.

## Inconvenientes del monolito

- Acoplamiento del ciclo de vida.
- Escalado homogéneo.
- Barrera tecnológica.
- Coordinación de equipos creciente.
- Adopción de nuevas tecnologías bloqueada por el conjunto.
- Cuello de botella organizativo a partir de cierto tamaño.

## Ventajas de los microservicios

### Despliegue
- Pipeline independiente por servicio.
- Frecuencia de despliegue ajustada a cada servicio.
- Riesgo y reversión limitados al servicio afectado.

### Escalado
- Selectividad: cada servicio se replica según su carga.
- Coste de infraestructura ajustado a la demanda real.

### Organización
- Autonomía de equipos.
- Decisiones técnicas distribuidas.
- Sin conflictos de fusión entre equipos.

## Inconvenientes de los microservicios

- Complejidad distribuida.
- Consistencia eventual.
- Observabilidad distribuida.
- Coste operativo por servicio.
- Duplicación de utilidades comunes.
- Gestión de versiones de contratos públicos.
- Latencia de red en cada llamada.

## Costes ocultos

### Monolito
- Coordinación de equipos.
- Oportunidad tecnológica.
- Escalado homogéneo.

### Microservicios
- Complejidad accidental.
- Observabilidad distribuida.
- Duplicación de utilidades.

## Criterios de decisión aplicados al proyecto del curso

| Criterio | Valor | Decisión |
```

```
|----------|-------|----------|
| Tamaño del dominio | Mediano | Monolito modular |
| Número de equipos | 2 | Monolito modular |
| Frecuencia de despliegue | Semanal | Monolito modular |
| Tolerancia al fallo | Baja | Monolito modular |
| Consistencia requerida | Fuerte | Monolito modular |
| Madurez operativa | Inicial | Monolito modular |

## Conclusión

El proyecto empieza como monolito modular. La evolución hacia microservicios se justificará cuando alguno de
los seis criterios cambie.
```

El documento de análisis comparativo es el artefacto de docs específico del punto 1.2. No modifica el ADR-001 existente ni ningún otro archivo del punto 1.1.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar el ADR-001 existente: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\analisis-ventajas-inconvenientes.md debe existir con las nueve secciones.

Paso 5 — Localizar la carpeta de código ya preparada

Desde Visual Studio expande `src/AceriaApp.Monolito.Api/Metricas` dentro de la solución **1.2** y localiza `MetricasAcoplamiento.cs`, `MetricasServicio.cs` y `AnalizadorMonolito.cs`. Abre cada archivo y verifica que contiene un bloque delimitado por `INICIO PASO 1.2.6`, `1.2.7` o `1.2.8`, respectivamente, y que las clases todavía no están activadas.

Comprueba que los archivos heredados de `Dominio` e `Infraestructura` no contienen marcadores nuevos de la etapa 1.2 y no necesitan ser modificados. Guarda la solución y prepara los archivos en ese orden: primero métricas globales, después métricas por servicio y finalmente el analizador.

La carpeta ya existe en el proyecto preparado; la progresión didáctica se logra activando el contenido, no creando archivos duplicados.

Errores comunes: crear un archivo con el mismo nombre; descomentar código de otros pasos; modificar archivos heredados.

Verificación: la carpeta `Metricas` y los archivos del punto están presentes, inicialmente comentados.

Paso 6 — Implementar y comprender MetricasAcoplamiento.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Metricas/MetricasAcoplamiento.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `Metricas/MetricasAcoplamiento.cs` y localiza el bloque delimitado por `INICIO PASO 1.2.6` y `FIN PASO 1.2.6`. Selecciona solo las declaraciones C# entre ambos marcadores, pulsa **Ctrl+K, Ctrl+U** una vez y conserva las líneas delimitadoras comentadas.

Comprueba `NumeroDeProyectos`, `NumeroDeReferenciasEntreProyectos` y `NumeroDeTablasCompartidas` como datos del indicador; estudia los cocientes de `IndiceAcoplamiento` e `IndiceComparticionDeDatos`. Guarda y compila el proyecto de API. Comprueba que el caso `NumeroDeProyectos == 0` devuelve cero en ambos métodos y no divide por cero.

```
namespace AceriaApp.Monolito.Api.Metricas;

public class MetricasAcoplamiento
{
    public int NumeroDeProyectos { get; init; }
    public int NumeroDeReferenciasEntreProyectos { get; init; }
    public int NumeroDeTablasCompartidas { get; init; }

    public double IndiceAcoplamiento()
    {
        if (NumeroDeProyectos == 0)
        {
            return 0;
        }

        return (double)NumeroDeReferenciasEntreProyectos / NumeroDeProyectos;
    }

    public double IndiceComparticionDeDatos()
    {
        if (NumeroDeProyectos == 0)
        {
            return 0;
        }

        return (double)NumeroDeTablasCompartidas / NumeroDeProyectos;
    }
}
```

`MetricasAcoplamiento` construye dos cocientes a partir de cantidades introducidas explícitamente: referencias entre proyectos y tablas compartidas, cada una comparada con el número de proyectos. Los métodos devuelven cero si no hay proyectos, evitando una división inválida. Son indicadores docentes que hacen visible un razonamiento; no descubren dependencias del repositorio ni permiten evaluar automáticamente una arquitectura real.

La clase `MetricasAcoplamiento` calcula dos indicadores didácticos de dependencias y compartición de datos. Estos cocientes comparan los valores declarados, pero no representan una medida universal de la calidad arquitectónica.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.
- Usar int para los índices: los índices son ratios y deben ser double.
- No comprobar la división por cero: el método lanza DivideByZeroException si el divisor es cero.

Verificación: la clase MetricasAcoplamiento compila sin errores y expone cuatro propiedades y dos métodos.

Interpretación del resultado: Un índice de acoplamiento compara referencias y número de proyectos. La ausencia de proyectos requiere tratamiento explícito para no dividir por cero.

Paso 7 — Implementar y comprender MetricasServicio.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Metricas/MetricasServicio.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `Metricas/MetricasServicio.cs` y comprueba que el paso marcado es `INICIO PASO 1.2.7`, sin mezclar código de otra etapa. Descomenta únicamente las líneas interiores del bloque con **Ctrl+K, Ctrl+U**, conservando su namespace y las propiedades públicas.

Localiza la fórmula `IndiceDeComplejidad` y la condición de `EsCandidatoAExtraccion`; compara los casos de pocas dependencias y muchas tablas. Guarda y compila la API. Comprueba que los métodos se calculan sobre las propiedades del servicio y que no has creado una segunda clase del mismo nombre.

```
namespace AceriaApp.Monolito.Api.Metricas;

public class MetricasServicio
{
    public string Nombre { get; init; } = string.Empty;
    public int NumeroDeEndpoints { get; init; }
    public int NumeroDeTablas { get; init; }
    public int NumeroDeDependenciasExternas { get; init; }

    public double IndiceDeComplejidad()
    {
        return NumeroDeEndpoints + NumeroDeTablas + NumeroDeDependenciasExternas;
    }

    public bool EsCandidatoAExtraccion()
    {
        return NumeroDeDependenciasExternas <= 1 && NumeroDeTablas <= 3;
    }
}
```

`MetricasServicio` describe una capacidad mediante el número declarado de endpoints, tablas y dependencias. `IndiceDeComplejidad` combina esos valores y `EsCandidatoAExtraccion` aplica un umbral del ejercicio. La clasificación es consecuencia de la fórmula programada; extraer realmente una capacidad requiere estudiar también su cohesión, datos y operaciones compartidas.

La clase `MetricasServicio` compara capacidades mediante datos introducidos en el ejercicio. `EsCandidatoAExtraccion` aplica umbrales docentes que por sí solos no justifican crear un microservicio.

Errores comunes:

- Usar string Nombre { get; set; } sin inicializar: el compilador emite advertencia de nulabilidad.
- Devolver int en IndiceDeComplejidad: el índice es una suma de enteros, pero se expone como double para facilitar comparaciones.
- No documentar los umbrales de EsCandidatoAExtraccion: el método devuelve un bool sin explicar los criterios.

Verificación: la clase MetricasServicio compila sin errores y expone cuatro propiedades y dos métodos.

Interpretación del resultado: El método candidato combina dos umbrales. Un `true` muestra que el escenario cumple esta heurística, no que ya se pueda desplegar como microservicio.

Paso 8 — Implementar y comprender AnalizadorMonolito.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Metricas/AnalizadorMonolito.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `Metricas/AnalizadorMonolito.cs`, identifica `INICIO PASO 1.2.8` y selecciona solo el código comentado de ese bloque. Descomenta un nivel con **Ctrl+K, Ctrl+U**; deja sin activar el método interno marcado como **RETO 1.2**, que corresponde a una ampliación opcional posterior.

Comprueba en el constructor la lista de `MetricasServicio` y recorre los métodos `CalcularAcoplamientoGlobal` y `ServiciosCandidatosAExtraccion`, relacionándolos con las dos clases previas. Guarda y compila la solución. Si `Sum` o `Where` no se resuelven, revisa el `using System.Linq` del bloque antes de añadir imports duplicados.

```
namespace AceriaApp.Monolito.Api.Metricas;

public class AnalizadorMonolito
{
    private readonly List<MetricasServicio> _servicios;

    public AnalizadorMonolito(List<MetricasServicio> servicios)
    {
        _servicios = servicios;
    }

    public MetricasAcoplamiento CalcularAcoplamientoGlobal()
    {
        var numeroDeProyectos = _servicios.Count;
        var numeroDeReferencias = _servicios.Sum(s => s.NumeroDeDependenciasExternas);
        var numeroDeTablasCompartidas = _servicios.Sum(s => s.NumeroDeTablas);

        return new MetricasAcoplamiento
        {
            NumeroDeProyectos = numeroDeProyectos,
            NumeroDeReferenciasEntreProyectos = numeroDeReferencias,
            NumeroDeTablasCompartidas = numeroDeTablasCompartidas
        };
    }

    public IEnumerable<MetricasServicio> ServiciosCandidatosAExtraccion()
    {
        return _servicios.Where(s => s.EsCandidatoAExtraccion());
    }
}
```

`AnalizadorMonolito` recibe una colección de métricas, agrega sus valores y obtiene candidatos mediante el criterio de `MetricasServicio`. No inspecciona todos los ensamblados ni ejecuta una migración a microservicios. El ejercicio sirve para distinguir el cálculo correcto de la decisión que un arquitecto debe justificar.

La clase `AnalizadorMonolito` agrega los datos declarados y filtra candidatos según el criterio programado. Extraer realmente una capacidad requiere evaluar su cohesión, contratos, persistencia y costes operativos.

Errores comunes:

- No inicializar _servicios en el constructor: la clase lanza NullReferenceException al usarla.
- Usar List sin using System.Collections.Generic: el tipo no se resuelve en proyectos con ImplicitUsings desactivado.
- Devolver IEnumerable sin using System.Linq: los métodos Sum y Where no se resuelven.

Verificación: la clase AnalizadorMonolito compila sin errores y expone un constructor y dos métodos.

Interpretación del resultado: El agregador calcula indicadores a partir de las capacidades declaradas; no descubre automáticamente dependencias reales en la solución .NET.

Paso 9 — Añadir el using necesario en AnalizadorMonolito

Abre el archivo `src/AceriaApp.Monolito.Api/Metricas/AnalizadorMonolito.cs` ya activado y busca al comienzo la directiva `using System.Linq;`. Si la directiva ya se descomentó en 1.2.8, **no vuelvas a añadirla**. Solo incorpórala si falta y los métodos `Sum`, `Where` u `OrderBy` no se reconocen.

Utiliza el editor de Visual Studio para comprobar las referencias de `CalcularAcoplamientoGlobal` y de `ServiciosCandidatosAExtraccion`. No modifiques sus reglas de negocio. Guarda, ejecuta `dotnet build src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` y documenta si el problema estaba en el `using` o en la activación incompleta del bloque.

los métodos Sum y Where pertenecen a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve los métodos de consulta.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: AnalizadorMonolito.cs compila sin errores y no muestra subrayados rojos.

Paso 10 — Comprobar el proyecto de tests del punto 1.2

En el Explorador de soluciones localiza `tests/AceriaApp.Metricas.Tests/AceriaApp.Metricas.Tests.csproj`: el proyecto xUnit **ya está entregado** en 1.2. Comprueba que `dotnet sln AceriaApp.sln list` lo muestra junto a la API y al proyecto de tests heredado de 1.1.

Abre su `.csproj` para revisar `net8.0`, xUnit, Microsoft.NET.Test.Sdk, FluentAssertions y la referencia de proyecto a la API. Ejecuta `dotnet restore AceriaApp.sln`. No uses `dotnet new xunit` encima de la carpeta preparada ni agregues un `UnitTest1.cs` adicional.

El punto 1.2 añade un proyecto de tests específico para las métricas. Se crea como proyecto nuevo sin modificar el proyecto de tests existente AceriaApp.Monolito.Tests del punto 1.1.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar el proyecto de tests del punto 1.1: viola la regla de no modificar archivos del punto anterior.
- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Metricas.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Metricas.Tests.

Paso 11 — Comprobar la referencia del proyecto de tests a la API monolítica

Abre `tests/AceriaApp.Metricas.Tests/AceriaApp.Metricas.Tests.csproj` en Visual Studio, sin editar la API monolítica. Localiza el elemento `ProjectReference` que enlaza desde el proyecto de tests hacia `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj`.

Comprueba que la referencia apunta **del test a la API**, no al revés, y que la ruta relativa resuelve realmente el archivo existente. Desde la raíz del punto ejecuta `dotnet restore AceriaApp.sln` y, si todo restaura, no agregues otra referencia duplicada.

La referencia permite que el proyecto de tests acceda a las clases de Metricas del proyecto monolítico. Sin esta referencia, los tests no pueden instanciar MetricasAcoplamiento, MetricasServicio ni AnalizadorMonolito.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Metricas.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 12 — Comprobar el paquete FluentAssertions al proyecto de tests

En el `.csproj` de `AceriaApp.Metricas.Tests` encuentra la dependencia `FluentAssertions` ya preparada con versión 6.12.2. Desde la raíz de la etapa ejecuta `dotnet list tests/AceriaApp.Metricas.Tests/AceriaApp.Metricas.Tests.csproj package` y localiza esa referencia.

Comprueba que `using FluentAssertions;` podrá utilizarse al descomentar los tests de 1.2.13 a 1.2.15. No ejecutes `dotnet add package FluentAssertions` si ya está presente; un `PackageReference` adicional no aporta aprendizaje ni mejora los tests.

FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar el proyecto de tests del punto 1.1.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 13 — Implementar y comprender MetricasAcoplamientoTests.cs

El archivo preparado es `tests/AceriaApp.Metricas.Tests/MetricasAcoplamientoTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.Metricas.Tests/MetricasAcoplamientoTests.cs` y localiza `INICIO PASO 1.2.13` y su `FIN PASO`. Selecciona el código C# interior y descomenta con **Ctrl+K, Ctrl+U**, sin activar bloques de reto opcionales que no pertenezcan al paso.

Revisa las aserciones sobre los índices globales de acoplamiento y compartición y relaciona los valores esperados con las fórmulas de `MetricasAcoplamiento`. Guarda y compila el proyecto de tests. Si falla una aserción, comprueba la inicialización numérica del caso antes de alterar las fórmulas.

```
using AceriaApp.Monolito.Api.Metricas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Metricas.Tests;

public class MetricasAcoplamientoTests
{
    [Fact]
    public void IndiceAcoplamiento_SinProyectos_DevuelveCero()
    {
        var metricas = new MetricasAcoplamiento { NumeroDeProyectos = 0 };

        metricas.IndiceAcoplamiento().Should().Be(0);
    }

    [Fact]
    public void IndiceAcoplamiento_ConProyectos_DevuelveRatio()
    {
        var metricas = new MetricasAcoplamiento
        {
            NumeroDeProyectos = 4,
            NumeroDeReferenciasEntreProyectos = 8
        };
```

```
        metricas.IndiceAcoplamiento().Should().Be(2.0);
    }

    [Fact]
    public void IndiceComparticionDeDatos_SinProyectos_DevuelveCero()
    {
        var metricas = new MetricasAcoplamiento { NumeroDeProyectos = 0, NumeroDeTablasCompartidas = 3 };

        metricas.IndiceComparticionDeDatos().Should().Be(0);
    }
}
```

`MetricasAcoplamientoTests` prueba los resultados numéricos de casos conocidos y el comportamiento con un denominador cero. Al aislar la fórmula, el test identifica un error de cálculo sin levantar la API ni consultar SQL Server. Su éxito acredita el contrato de estos métodos, no el valor de los indicadores para cualquier proyecto.

los tests verifican los dos métodos de MetricasAcoplamiento con casos normales y con casos límite (división por cero).

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Metricas;: la clase no se resuelve.
- Usar Assert.Equal en lugar de Should().Be(): el curso usa FluentAssertions.
- No cubrir el caso de división por cero: el test no detecta el error.

Verificación: el archivo MetricasAcoplamientoTests.cs compila y contiene tres métodos de test.

Paso 14 — Implementar y comprender MetricasServicioTests.cs

El archivo preparado es `tests/AceriaApp.Metricas.Tests/MetricasServicioTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.Metricas.Tests/MetricasServicioTests.cs` en el proyecto de la etapa 1.2 y localiza el marcador `INICIO PASO 1.2.14`. Descomenta solo el bloque C# correspondiente y compara los casos de prueba con las propiedades `NumeroDeEndpoints`, `NumeroDeTablas` y `NumeroDeDependenciasExternas`.

Identifica los tests que aceptan o descartan una extracción y justifica cada resultado con la condición de `EsCandidatoAExtraccion`. Guarda y compila; no cambies los valores esperados para esconder un error en el código que activaste en 1.2.7.

```
using AceriaApp.Monolito.Api.Metricas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Metricas.Tests;

public class MetricasServicioTests
{
    [Fact]
    public void EsCandidatoAExtraccion_ConPocasDependencias_DevuelveTrue()
    {
        var metricas = new MetricasServicio
        {
            Nombre = "Inventario",
            NumeroDeEndpoints = 3,
            NumeroDeTablas = 2,
            NumeroDeDependenciasExternas = 1
        };

        metricas.EsCandidatoAExtraccion().Should().BeTrue();
    }
```

```
    [Fact]
    public void EsCandidatoAExtraccion_ConMuchasDependencias_DevuelveFalse()
    {
        var metricas = new MetricasServicio
        {
            Nombre = "Ordenes",
            NumeroDeEndpoints = 5,
            NumeroDeTablas = 4,
            NumeroDeDependenciasExternas = 3
        };

        metricas.EsCandidatoAExtraccion().Should().BeFalse();
    }

    [Fact]
    public void IndiceDeComplejidad_SumaDePropiedades()
    {
        var metricas = new MetricasServicio
        {
            NumeroDeEndpoints = 3,
            NumeroDeTablas = 2,
            NumeroDeDependenciasExternas = 1
        };

        metricas.IndiceDeComplejidad().Should().Be(6);
    }
}
```

`MetricasServicioTests` compara entradas que cumplen o incumplen el umbral de candidatura y verifica el cálculo del índice de complejidad. Es importante observar el cambio en la salida al variar las dependencias: la prueba demuestra la regla programada y hace visible que no existe una recomendación de extracción universal.

los tests verifican la lógica de EsCandidatoAExtraccion con casos que cumplen y no cumplen los umbrales, y verifican que IndiceDeComplejidad suma correctamente las tres propiedades.

Errores comunes:

- No cubrir el caso que supera los umbrales: el test no verifica la lógica de rechazo.
- Olvidar el using AceriaApp.Monolito.Api.Metricas;: la clase no se resuelve.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.

Verificación: el archivo MetricasServicioTests.cs compila y contiene tres métodos de test.

Paso 15 — Implementar y comprender AnalizadorMonolitoTests.cs

El archivo preparado es `tests/AceriaApp.Metricas.Tests/AnalizadorMonolitoTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.Metricas.Tests/AnalizadorMonolitoTests.cs` y localiza `INICIO PASO 1.2.15`. Selecciona y descomenta las clases y métodos de prueba preparados con **Ctrl+K, Ctrl+U**, dejando comentados los retos que todavía no se han ejecutado.

Revisa cómo se construye una lista de servicios y cómo se verifican `CalcularAcoplamientoGlobal` y `ServiciosCandidatosAExtraccion` contra resultados esperados. Guarda y compila el proyecto de tests; verifica que no has añadido una segunda clase de pruebas ni una dependencia ajena al punto 1.2.

```
using AceriaApp.Monolito.Api.Metricas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Metricas.Tests;

public class AnalizadorMonolitoTests
```

```
{
    [Fact]
    public void CalcularAcoplamientoGlobal_ConServicios_AgregaMetricas()
    {
        var servicios = new List<MetricasServicio>
        {
            new() { Nombre = "Ordenes", NumeroDeDependenciasExternas = 2, NumeroDeTablas = 3 },
            new() { Nombre = "Inventario", NumeroDeDependenciasExternas = 1, NumeroDeTablas = 2 }
        };
        var analizador = new AnalizadorMonolito(servicios);

        var resultado = analizador.CalcularAcoplamientoGlobal();

        resultado.NumeroDeProyectos.Should().Be(2);
        resultado.NumeroDeReferenciasEntreProyectos.Should().Be(3);
        resultado.NumeroDeTablasCompartidas.Should().Be(5);
    }

    [Fact]
    public void ServiciosCandidatosAExtraccion_FiltraCorrectamente()
    {
        var servicios = new List<MetricasServicio>
        {
            new() { Nombre = "Ordenes", NumeroDeDependenciasExternas = 3, NumeroDeTablas = 4 },
            new() { Nombre = "Inventario", NumeroDeDependenciasExternas = 1, NumeroDeTablas = 2 }
        };
        var analizador = new AnalizadorMonolito(servicios);

        var candidatos = analizador.ServiciosCandidatosAExtraccion().ToList();

        candidatos.Should().HaveCount(1);
        candidatos[0].Nombre.Should().Be("Inventario");
    }
}
```

`AnalizadorMonolitoTests` prepara varios servicios con valores controlados y comprueba la agregación y el filtrado. El resultado depende de los datos del test, no de una medición en vivo del monolito. Así comprobamos la clasificación sin atribuirle una capacidad de análisis automático que todavía no tiene.

los tests verifican que AnalizadorMonolito agrega correctamente las métricas y filtra los servicios candidatos a extracción.

Errores comunes:

- Olvidar using System.Collections.Generic;: List no se resuelve.
- Olvidar using System.Linq;: ToList() no se resuelve.
- No comprobar el orden de los candidatos: el test podría pasar por casualidad.

Verificación: el archivo AnalizadorMonolitoTests.cs compila y contiene dos métodos de test.

Paso 16 — Compilar la solución completa

En la terminal situada junto a `AceriaApp.sln` ejecuta `dotnet build AceriaApp.sln -c Release` después de activar los archivos de código y tests de 1.2. Comprueba que compilan los tres proyectos de esta etapa y que el resumen termina con `Build succeeded` y **0 errores**.

Si hay un fallo, comienza por el primer error que indica archivo y línea y vuelve al paso que activó ese bloque; no sustituyas la solución por la de otro punto. Guarda el resumen de compilación como evidencia y comprueba que no ha sido necesario modificar archivos del proyecto 1.1.

compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.

- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.2

En Visual Studio abre **Prueba > Explorador de pruebas** o la terminal de la carpeta 1.2. Ejecuta `dotnet test tests/AceriaApp.Metricas.Tests/AceriaApp.Metricas.Tests.csproj -c Release` para probar exclusivamente las clases nuevas de métricas.

Comprueba el recuento **8 pruebas superadas y 0 fallidas**. Si se encuentra un reto sin activar, revisa la selección de comentarios de las pruebas. Identifica en el Explorador de pruebas cuáles corresponden a acoplamiento, complejidad y analizador; relaciona cada una con la clase implementada.

ejecutar los tests del punto 1.2 verifica que las clases de métricas funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests del punto 1.1, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 8, Skipped: 0, Total: 8.

Paso 18 — Ejecutar todos los tests de la solución

Ejecuta desde la carpeta 1.2 `dotnet test AceriaApp.sln -c Release --no-build`, después de la compilación Release del paso 16. Comprueba **9 tests en total**: el heredado de Orden en 1.1 y los 8 de métricas añadidos en 1.2.

Si el ejecutor no encuentra DLLs, ejecuta primero `dotnet build AceriaApp.sln -c Release` y repite sin alterar las pruebas. Registra la salida y confirma que una nueva etapa conserva las comprobaciones anteriores en lugar de sustituirlas.

ejecutar todos los tests de la solución verifica que los tests del punto 1.1 y del punto 1.2 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests del punto 1.1: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 9, Skipped: 0, Total: 9.

Paso 19 — Revisar el documento preparado: README.md

Abre el `README.md` de **1.2**, no el de 1.1; este documento ya incluye el objetivo y el estado del punto. Revisa que describe la API heredada, las clases de `Metricas` y el proyecto xUnit nuevo, y que los comandos se refieren a la solución 1.2.

Contrasta las rutas `src`, `tests` y `docs` citadas con el árbol real del proyecto; apunta cualquier diferencia detectada. No pegues encima el README del punto anterior: sus contenidos deben diferir porque describen etapas distintas.

```
# Punto 1.2 — Ventajas e inconvenientes

## Objetivo
Analizar las ventajas e inconvenientes de monolito y microservicios con un marco explícito de cuatro
dimensiones y añadir al proyecto las métricas de acoplamiento.

## Estado del proyecto
- Solución `AceriaApp.sln` con tres proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests` y
`AceriaApp.Metricas.Tests`.
- Documento `docs/analisis-ventajas-inconvenientes.md` con el análisis comparativo.
- Clases `MetricasAcoplamiento`, `MetricasServicio` y `AnalizadorMonolito` en `src/AceriaApp.Monolito.Api/
Metricas/`.
- Ocho tests nuevos en `tests/AceriaApp.Metricas.Tests`.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/analisis-ventajas-inconvenientes.md`.
- Carpeta `Metricas` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.Metricas.Tests` con ocho tests nuevos.
- No se ha modificado ningún archivo del punto 1.1.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md).
```

el README del punto 1.2 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.1.

Errores comunes:

- Modificar el README del punto 1.1: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 1.1: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.2 con las cinco secciones.

Paso 20 — Revisar el documento preparado: ADR-002-metricas-acoplamiento.md

Abre `docs/ADR-002-metricas-acoplamiento.md` de la etapa 1.2, ya incluido como material de referencia. Comprueba sus secciones Estado, Contexto, Decisión, Alternativas y Consecuencias y relaciona los indicadores con la decisión de extracción de servicios.

Anota las limitaciones de las métricas del ejemplo: sirven para discutir arquitectura, no sustituyen información de despliegue, negocio y autonomía de equipos. Verifica que el ADR-001 heredado permanece separado y sin modificaciones; no reemplaces un documento de decisión anterior por uno nuevo.

```
# ADR-002: Métricas de acoplamiento para decidir la extracción de servicios

## Estado
Aceptado

## Contexto
```

```
El proyecto AceriaApp ha adoptado un monolito modular. Es necesario decidir qué funcionalidades son
candidatas a extraerse como microservicios en módulos posteriores.

## Decisión
Se adoptan dos indicadores objetivos: el índice de acoplamiento entre proyectos y el índice de compartición
de datos. Una funcionalidad es candidata a extracción si tiene como máximo una dependencia externa y como
máximo tres tablas propias.

## Alternativas consideradas
- Decidir la extracción por criterio subjetivo: descartado por falta de trazabilidad.
- Extraer todas las funcionalidades a la vez: descartado por el coste operativo.

## Consecuencias
- Positivas: decisiones trazables, comparables y medibles.
- Negativas: los umbrales son iniciales y pueden requerir ajuste.
- Mitigación: revisar los umbrales al final de cada módulo.
```

el ADR-002 documenta la decisión de adoptar métricas de acoplamiento. Es un artefacto nuevo del punto 1.2 que no modifica el ADR-001 del punto 1.1.

Errores comunes:

- Modificar el ADR-001: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-002: rompe la convención.

Verificación: el archivo docs\ADR-002-metricas-acoplamiento.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.1

Desde la raíz del punto 1.2, localiza el `ADR-001` heredado y el fichero del mismo nombre dentro de `../1.1-monolito-vs-microservicios/docs`. Ejecuta `fc /L` sobre ambos ADR-001 si trabajas en Windows, o compara sus hashes; **deben ser idénticos** porque se heredan sin modificar.

Abre ambos `README.md` para distinguir su estado: el de 1.2 **debe ser distinto** del de 1.1 porque documenta los archivos nuevos y sus pruebas. No copies de nuevo los archivos para forzar igualdad. Registra como evidencia qué documento es heredado sin cambios y cuál es propio de la etapa.

La comparación con fc verifica que el ADR-001 no se ha modificado y que el README del punto 1.2 es distinto al del punto 1.1.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: el ADR-001 no muestra diferencias. El README del punto 1.2 muestra diferencias respecto al del punto 1.1.

Paso 22 — Verificar la estructura del punto 1.2

Desde la raíz común `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO` ejecuta `tree /F 1.2-ventajas-inconvenientes` en una terminal de Windows. Comprueba en `src` las clases heredadas y las tres clases de `Metricas`, en `tests` el proyecto xUnit de métricas y en `docs` el análisis junto a ADR-001 y ADR-002.

Confirma que `AceriaApp.sln` de esta etapa incluye los proyectos declarados y que los archivos nuevos están donde indican sus namespaces. Anota cualquier fichero o carpeta inesperados, distinguiendo fuentes versionadas de los temporales `bin` y `obj` creados al compilar.

La verificación estructural confirma que el punto 1.2 añade archivos nuevos sin modificar los existentes.

Errores comunes:

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 1.1: viola la regla de no modificar.
- Omitir la carpeta Metricas: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.1.

Paso 23 — Revisar el documento preparado: .gitignore

Abre `.gitignore` de la carpeta 1.2 y revisa que conserva las exclusiones `bin/`, `obj/`, `.vs/` y resultados de pruebas. Ejecuta `git status --short` en tu repositorio de trabajo para comprobar que los binarios y temporales no se incluyen entre cambios pendientes.

Compara el `.gitignore` con el del punto 1.1; debe mantener la misma política de exclusión, aunque otros documentos como README sí cambien. No borres código ni tests para dejar limpio el estado Git: si aparecen binarios, corrige las exclusiones en tu copia y revisa el resultado.

bin/ obj/ *.user *.suo .vs/ TestResults/

el .gitignore evita que los artefactos de compilación del punto 1.2 se incluyan en el control de versiones. Si ya existe copiado del punto 1.1, no se modifica.

Errores comunes:

- Modificar el .gitignore copiado del punto 1.1: viola la regla de no modificar archivos del punto anterior.
- Olvidar bin/ y obj/: el repositorio se llena de artefactos.
- Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: git status no muestra archivos dentro de bin/ ni obj/.

Paso 24 — Hacer el commit del punto 1.2 y verificar el estado final

Antes de crear un commit, ejecuta `git status` desde tu copia personal del repositorio y comprueba que los cambios afectan al punto 1.2 y no al 1.1. Revisa `git diff --stat` y selecciona con `git add` únicamente el contenido que corresponde a la etapa de métricas; no incluyas `bin/` u otros archivos generados.

Ejecuta el commit del punto 1.2 con el mensaje que aparece en la práctica, siempre dentro de tu rama de trabajo y sin modificar la rama oficial del curso. Comprueba `git log -1 --oneline` y `git status` para confirmar que el commit existe y no quedan cambios inesperados; conserva la salida de los nueve tests como evidencia de cierre.

El commit del punto 1.2 registra el estado acumulado del proyecto. Los dos commits consecutivos permiten comparar el punto 1.1 y el punto 1.2 con git diff.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra dos commits: Punto 1.1: estructura inicial del proyecto AceriaApp y Punto 1.2: análisis de ventajas e inconvenientes y métricas de acoplamiento. git status muestra nothing to commit, working tree clean.

Nota de trazabilidad: en una descarga del repositorio puede no haber cambios nuevos que confirmar. Antes de ejecutar `git commit`, usar `git status` y confirmar únicamente los archivos efectivamente modificados durante el recorrido. No ejecutar `git init` dentro de cada punto: la raíz del repositorio ya contiene el historial.

Reto de ampliación: solución guiada

Problema a resolver: añadir un método ServicioMenosAcoplado a AnalizadorMonolito que devuelva el servicio con menor índice de complejidad, y un test que verifique que devuelve el servicio correcto cuando hay empate.

Etapa 1 — En AnalizadorMonolito.cs, añadir el siguiente método después de ServiciosCandidatosAExtraccion:

```
public MetricasServicio? ServicioMenosAcoplado()
{
    if (_servicios.Count == 0)
    {
        return null;
    }

    return _servicios
        .OrderBy(s => s.IndiceDeComplejidad())
        .ThenBy(s => s.Nombre)
        .First();
}
```

Etapa 2 — Explicar línea por línea:

- **Línea 1:** `public MetricasServicio? ServicioMenosAcoplado()` — declara el método con retorno anulable.
- **Línea 2:** `{` — abre el cuerpo del método.
- **Línea 3:** `if (_servicios.Count == 0)` — comprueba si la lista está vacía.
- **Línea 4:** `{` — abre el bloque del caso vacío.
- **Línea 5:** `return null;` — devuelve null si no hay servicios.
- **Línea 6:** `}` — cierra el bloque del caso vacío.
- **Línea 7:** `return _servicios` — comienza la consulta LINQ.
- **Línea 8:** `.OrderBy(s => s.IndiceDeComplejidad())` — ordena por índice de complejidad ascendente.
- **Línea 9:** `.ThenBy(s => s.Nombre)` — desempata por nombre ascendente.
- **Línea 10:** `.First();` — devuelve el primer elemento.
- **Línea 11:** `}` — cierra el cuerpo del método.

Etapa 3 — Guardar el archivo con Ctrl+S.

Etapa 4 — En AnalizadorMonolitoTests.cs, añadir el siguiente test:

```
[Fact]
public void ServicioMenosAcoplado_ConEmpate_DesempataPorNombre()
{
    var servicios = new List<MetricasServicio>
    {
        new() { Nombre = "Ordenes", NumeroDeEndpoints = 1, NumeroDeTablas = 1, NumeroDeDependenciasExternas =
1 },
```

```
        new() { Nombre = "Inventario", NumeroDeEndpoints = 1, NumeroDeTablas = 1,
NumeroDeDependenciasExternas = 1 }
    };
    var analizador = new AnalizadorMonolito(servicios);

    var resultado = analizador.ServicioMenosAcoplado();

    resultado.Should().NotBeNull();
    resultado!.Nombre.Should().Be("Inventario");
}
```

Etapa 5 — Explicar línea por línea:

- **Línea 1:** `[Fact]` — marca el método como test de xUnit.
- **Línea 2:** `public void ServicioMenosAcoplado_ConEmpate_DesempataPorNombre()` — declara el test.
- **Línea 3:** `{` — abre el cuerpo del test.
- **Línea 4:** `var servicios = new List<MetricasServicio>` — crea la lista de servicios.
- **Línea 5:** `{` — abre el inicializador de la lista.
- **Línea 6:** `new() { Nombre = "Ordenes", ... }` — primer servicio con índice de complejidad 3.
- **Línea 7:** `new() { Nombre = "Inventario", ... }` — segundo servicio con el mismo índice.
- **Línea 8:** `};` — cierra el inicializador.
- **Línea 9:** `var analizador = new AnalizadorMonolito(servicios);` — instancia el analizador.
- **Línea 10:** `var resultado = analizador.ServicioMenosAcoplado();` — invoca el método.
- **Línea 11:** `resultado.Should().NotBeNull();` — verifica que no es nulo.
- **Línea 12:** `resultado!.Nombre.Should().Be("Inventario");` — verifica que el desempate es por nombre.
- **Línea 13:** `}` — cierra el cuerpo del test.

Etapa 6 — Ejecutar dotnet test tests\AceriaApp.Metricas.Tests\AceriaApp.Metricas.Tests.csproj y verificar que pasan los nueve tests.

Etapa 7 — Verificar que el método ServicioMenosAcoplado no modifica ningún archivo del punto 1.1.

Etapa 8 — Hacer commit con git commit -m "Reto 1.2: método ServicioMenosAcoplado con desempate por nombre".

Verificación: la salida de dotnet test muestra Passed: 9.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-1-introduccion-microservicios/1.2-ventajas-inconvenientes/ con la solución AceriaApp.sln que contiene cuatro proyectos.
- El documento docs/analisis-ventajas-inconvenientes.md con el análisis comparativo completo.
- El documento docs/ADR-002-metricas-acoplamiento.md con la decisión sobre las métricas.
- Las clases MetricasAcoplamiento, MetricasServicio y AnalizadorMonolito en src/AceriaApp.Monolito.Api/Metricas/.
- El proyecto tests/AceriaApp.Metricas.Tests con ocho tests (nueve tras el reto).
- Los tests del punto 1.1 y del punto 1.2 pasando conjuntamente con dotnet test AceriaApp.sln.
- El ADR-001 del punto 1.1 sin modificar.
- Dos commits en Git: el del punto 1.1 y el del punto 1.2.

Conclusión y enlace al siguiente punto

El punto 1.2 ha completado el análisis de ventajas e inconvenientes con un marco explícito de cuatro dimensiones y ha añadido al proyecto las métricas de acoplamiento que permitirán decidir qué funcionalidades son candidatas a extraerse como microservicios. Esta base permite avanzar hacia el punto 1.3, donde se estudiarán los principios de diseño que guían la descomposición de un sistema en servicios autónomos.

Fin del punto 1.2.

Punto 1.3 — Principios de diseño

Diseñar límites a partir de responsabilidades coherentes

Una frontera de servicio no es una carpeta ni una nueva aplicación iniciada con `dotnet new`. Comienza por una responsabilidad de negocio que puede expresarse con un vocabulario estable y por el reconocimiento de los datos y decisiones que le pertenecen. El punto 1.3 incorpora una representación de límites propuestos y unas reglas didácticas para razonar sobre cohesión y autonomía, sin presentar esos resultados como una validación automática de la arquitectura.

La solución de 1.3 contiene el estado acumulado de la API monolítica. Sobre ella implementaremos `LimiteServicio`, `ValidadorLimiteServicio` y `CatalogoLimites`. Después ejecutaremos casos aceptados y rechazados para entender exactamente qué comprueba el código. El alumno debe poder conectar cada regla con una decisión de diseño y reconocer dónde sería imprescindible consultar al negocio antes de dibujar un microservicio real.

El punto habrá cumplido su propósito cuando podamos explicar por qué un límite propuesto resulta coherente o problemático según el catálogo, sin confundir esa comprobación con un despliegue independiente, aislamiento de datos o una prueba de comunicación remota.

Desarrollo práctico

Ejercicio: Añadir al punto 1.2 un documento de principios de diseño en docs/, una clase de validación de límites de servicio en el proyecto monolítico y un test unitario que verifique la validación. El punto 1.3 parte del contenido del punto 1.2 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: utiliza la carpeta real de tu copia del repositorio en Windows; `C:\cursos\` solo representa una ubicación posible.AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Localizar el proyecto independiente del punto 1.3

Localiza `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO/1.3-principios-diseno` en tu copia del repositorio; no sobrescribas la carpeta del punto anterior `1.2-ventajas-inconvenientes`. Comprueba `AceriaApp.sln`, `src`, `tests`, `docs` y `README.md`; las clases nuevas están preparadas, pero parte del código permanece comentado.

Abre la terminal en esa misma carpeta y ejecuta `dotnet --list-sdks` para comprobar la disponibilidad del SDK .NET 8. Registra la ruta de trabajo y diferencia el proyecto distribuido de uno vacío que no debes crear otra vez.

Cada punto constituye una solución autónoma. Se evita copiar la solución anterior sobre archivos nuevos ya preparados y se garantiza que los bloques de código comentados correspondan exactamente a este punto.

Errores comunes: abrir la solución del punto anterior; copiar archivos que sobrescriban los bloques preparados; confundir la carpeta del repositorio con la carpeta del proyecto.

Verificación: `AceriaApp.sln` existe y la terminal está situada dentro de la carpeta de este punto.

Paso 2 — Abrir la solución y comprobar los proyectos heredados

Abre en Visual Studio el archivo `AceriaApp.sln` de `1.3-principios-diseno` desde **Archivo > Abrir > Proyecto o solución**. Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 5 proyectos acumulados, incluido `AceriaApp.Diseno.Tests`.

Expande la carpeta `src/AceriaApp.Monolito.Api/Diseno` y observa que los tipos de esta etapa tienen marcadores `INICIO PASO` sin activar. Ejecuta `dotnet build AceriaApp.sln` y guarda la salida de la compilación inicial antes de editar ninguna clase.

se identifica el estado heredado ejecutable y se evita confundir una clase comentada con un proyecto todavía no creado.

Errores comunes: abrir la carpeta de otro punto; contar únicamente los proyectos con pruebas activas; descomentar todos los bloques a la vez.

Verificación: la solución abre y compila antes de introducir los cambios del punto.

Paso 3 — Reconocer documentación heredada y nuevas fuentes de Diseno

Abre `docs` en la carpeta `1.3-principios-diseno` y comprueba que conserva los documentos del punto anterior `1.2-ventajas-inconvenientes`. Identifica los archivos nuevos `docs/principios-diseno.md` y `docs/ADR-003-principios-diseno.md` que vienen incluidos como material preparado.

Contrasta uno de los ADR heredados con el fichero homónimo de la etapa anterior, sin reescribirlo. Anota qué archivos son heredados y cuáles nuevos; no exijas una carpeta `docs` vacía antes de comenzar, porque los documentos ya están entregados.

El punto 1.3 parte del contenido del punto 1.2, que ya incluye la carpeta docs con tres documentos. No se modifica ningún documento existente.

Errores comunes:

- Modificar el ADR-001 o el ADR-002: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 1.2.
- Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los tres documentos del punto 1.2 sin modificaciones.

Paso 4 — Revisar el documento preparado: principios-diseno.md

Abre `docs/principios-diseno.md` en Visual Studio con la vista Markdown, sin crear otro archivo de igual nombre. Localiza los apartados sobre límites de servicio, cohesión y acoplamiento y vincula dos de sus decisiones con AceriaApp.

Elabora una tabla breve de ventajas, riesgos y criterios técnicos con un ejemplo aplicable al dominio de órdenes de producción. Contrasta la estructura con el texto incluido en esta práctica y guarda tus conclusiones en un archivo personal separado.

```
# Principios de diseño — Punto 1.3

## Categorías de principios

Tres categorías: responsabilidad, ocultación y acoplamiento.

## Principio de responsabilidad única

Un servicio debe tener una sola razón para cambiar, identificada con una capacidad de negocio.

## Principio de ocultación de información

Un servicio debe exponer solo lo necesario y ocultar el modelo de datos, la lógica interna y la topología.

## Principio de acoplamiento débil y cohesión fuerte

Los elementos de un servicio deben estar fuertemente relacionados entre sí y débilmente con otros.

## Principio de límite del contexto

Cada servicio debe corresponder a un contexto delimitado del dominio.

## Principio de autonomía del servicio

Un servicio debe poder operar de forma independiente en despliegue, datos y fallo.

## Principio de contrato explícito

La comunicación entre servicios se realiza a través de contratos públicos, versionados y documentados.

## Principio de tolerancia al fallo

Un servicio debe seguir operando cuando una dependencia falla, aunque sea con funcionalidad degradada.

## Principio de evolución independiente

Un servicio debe poder evolucionar sin obligar a los demás a cambiar al mismo ritmo.

## Aplicación al proyecto del curso

| Principio | Aplicación |
|-----------|------------|
| Responsabilidad única | Módulos por capacidad de negocio |
| Ocultación | Cada módulo con su propia carpeta |
| Acoplamiento débil | Interfaces entre módulos |
| Límite del contexto | Contextos delimitados |
| Autonomía | Evolución hacia microservicios |
| Contrato explícito | DTOs públicos |
| Tolerancia al fallo | Polly |
| Evolución independiente | Versionado de contratos |
```

El documento de principios de diseño es el artefacto de docs específico del punto 1.3. No modifica los tres documentos existentes del punto 1.2.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\principios-diseno.md existe con las diez secciones.

Paso 5 — Localizar la carpeta de código ya preparada

En el Explorador de soluciones abre `src/AceriaApp.Monolito.Api/Diseno` y localiza `LimiteServicio.cs`, `ValidadorLimiteServicio.cs` y `CatalogoLimites.cs`. En cada archivo identifica el marcador `INICIO PASO 1.3.6`, `1.3.7` o `1.3.9`; comprueba que las declaraciones nuevas están comentadas.

Confirma que las clases heredadas de Dominio e Infraestructura siguen activas y no se deben modificar. Planifica la secuencia de activación: primero `LimiteServicio`, después `ValidadorLimiteServicio` y finalmente `CatalogoLimites`.

La carpeta ya existe en el proyecto preparado; la progresión didáctica se logra activando el contenido, no creando archivos duplicados.

Errores comunes: crear un archivo con el mismo nombre; descomentar código de otros pasos; modificar archivos heredados.

Verificación: la carpeta `Diseno` y los archivos del punto están presentes, inicialmente comentados.

Paso 6 — Implementar y comprender LimiteServicio.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Diseno/LimiteServicio.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/Diseno/LimiteServicio.cs` y localiza los delimitadores `INICIO PASO 1.3.6` y `FIN PASO 1.3.6`. Selecciona solo las líneas C# interiores, pulsa **Ctrl+K, Ctrl+U** una vez y conserva los marcadores como comentarios.

Comprueba que aparecen las propiedades CapacidadDeNegocio, ContextoDelimitado, NumeroDeConceptos y NumeroDeDependenciasSalientes y explica qué describe cada una. Guarda y compila `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` sin crear una segunda declaración de `LimiteServicio`.

```
namespace AceriaApp.Monolito.Api.Diseno;

public class LimiteServicio
{
    public string Nombre { get; init; } = string.Empty;
    public string CapacidadDeNegocio { get; init; } = string.Empty;
    public string ContextoDelimitado { get; init; } = string.Empty;
    public int NumeroDeConceptos { get; init; }
    public int NumeroDeDependenciasSalientes { get; init; }
    public bool ExponeEntidadesDeDominio { get; init; }
    public bool ComparteBaseDeDatos { get; init; }
}
```

`LimiteServicio` contiene los atributos de una frontera propuesta: su capacidad de negocio, el contexto delimitado y otros indicadores docentes de cohesión o autonomía. No se trata de un servicio ejecutable y su creación no modifica el despliegue. El objetivo es disponer de entradas explícitas para discutir reglas de diseño.

La clase LimiteServicio modela los atributos que definen un límite de servicio según los principios de diseño. Cada propiedad corresponde a un principio: capacidad de negocio al de responsabilidad única, contexto delimitado al de límite del contexto, número de conceptos al de cohesión, dependencias salientes al de acoplamiento, exposición de entidades al de ocultación y compartición de base de datos al de autonomía.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.

- No inicializar las propiedades de tipo string con string.Empty: el compilador emite advertencias de nulabilidad.
- Usar set en lugar de init para las propiedades: permite modificar la instancia después de crearla.

Verificación: la clase LimiteServicio compila sin errores y expone siete propiedades.

Paso 7 — Implementar y comprender ValidadorLimiteServicio.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Diseno/ValidadorLimiteServicio.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/Diseno/ValidadorLimiteServicio.cs` y localiza el bloque delimitado para el paso 1.3.7. Descomenta únicamente su código con **Ctrl+K, Ctrl+U** y deja inactivos los miembros marcados como `RETO`.

Sigue la lógica de `Validar` para capacidad y contexto, conceptos y dependencias, base compartida y entidades expuestas; identifica qué condición genera cada error. Guarda y compila; comprueba que la clase devuelve su tipo de resultado de validación y no depende de un reto opcional.

```
namespace AceriaApp.Monolito.Api.Diseno;

public class ValidadorLimiteServicio
{
    public ResultadoValidacion Validar(LimiteServicio limite)
    {
        var errores = new List<string>();

        if (string.IsNullOrWhiteSpace(limite.CapacidadDeNegocio))
        {
            errores.Add("El servicio no declara una capacidad de negocio.");
        }

        if (string.IsNullOrWhiteSpace(limite.ContextoDelimitado))
        {
            errores.Add("El servicio no declara un contexto delimitado.");
        }

        if (limite.NumeroDeConceptos > 3)
        {
            errores.Add("El servicio maneja más de tres conceptos: cohesión baja.");
        }

        if (limite.NumeroDeDependenciasSalientes > 3)
        {
            errores.Add("El servicio depende de más de tres servicios: acoplamiento alto.");
        }

        if (limite.ExponeEntidadesDeDominio)
        {
            errores.Add("El servicio expone entidades de dominio: viola la ocultación.");
        }

        if (limite.ComparteBaseDeDatos)
        {
            errores.Add("El servicio comparte base de datos: viola la autonomía.");
        }

        return new ResultadoValidacion(errores.Count == 0, errores);
    }
}

public record ResultadoValidacion(bool EsValido, IReadOnlyList<string> Errores);
```

`ValidadorLimiteServicio` examina las propiedades del límite y reúne los motivos que impiden aceptarlo conforme a las reglas del ejercicio. Sigue cada comprobación hasta el resultado devuelto, especialmente los casos de datos obligatorios o de cohesión insuficiente. Una respuesta válida significa que cumple ese contrato de validación, no que la frontera haya sido aprobada por expertos del dominio.

La clase ValidadorLimiteServicio aplica los principios de diseño de forma automática. Recibe un LimiteServicio y devuelve un ResultadoValidacion con la lista de errores detectados. Cada comprobación corresponde a un principio concreto.

Errores comunes:

- Olvidar el using System.Collections.Generic;: List no se resuelve.
- No devolver los errores: el validador no informa de qué principio se ha violado.
- Usar bool en lugar de ResultadoValidacion: pierde la información de los errores.

Verificación: la clase ValidadorLimiteServicio compila sin errores y expone un método Validar y un record auxiliar.

Interpretación del resultado: Cada mensaje representa una infracción diferente. Al recibir un límite correcto, el resultado debe ser válido y no contener errores.

Paso 8 — Añadir el using necesario en ValidadorLimiteServicio

En `src/AceriaApp.Monolito.Api/Diseno/ValidadorLimiteServicio.cs`, examina las directivas `using` que quedaron activas al completar el paso 1.3.7. Comprueba que ya aparecen los imports necesarios (System.Collections.Generic); no agregues una segunda copia si están presentes.

Relaciona cada import con las colecciones y los métodos empleados en `Validar`. Guarda y compila; si hay errores de tipos, comprueba primero el bloque de activación antes de cambiar las dependencias.

El tipo List pertenece a System.Collections.Generic. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Linq; en lugar de System.Collections.Generic: no resuelve List.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorLimiteServicio.cs compila sin errores y no muestra subrayados rojos.

Paso 9 — Implementar y comprender CatalogoLimites.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Diseno/CatalogoLimites.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/Diseno/CatalogoLimites.cs` y busca los marcadores del paso 1.3.9. Descomenta solo las líneas interiores con **Ctrl+K, Ctrl+U**, conservando inactivo el bloque de `RETO`.

Comprueba que el catálogo implementa `LimitesValidos y LimitesInvalidos` y usa los tipos de los pasos anteriores. Guarda y compila, revisando que los métodos no crean una segunda clase con el nombre `CatalogoLimites`.

```
namespace AceriaApp.Monolito.Api.Diseno;

public class CatalogoLimites
{
```

```
    private readonly List<LimiteServicio> _limites;

    public CatalogoLimites(List<LimiteServicio> limites)
    {
        _limites = limites;
    }

    public IEnumerable<LimiteServicio> LimitesValidos(ValidadorLimiteServicio validador)
    {
        return _limites.Where(l => validador.Validar(l).EsValido);
    }

    public IEnumerable<LimiteServicio> LimitesInvalidos(ValidadorLimiteServicio validador)
    {
        return _limites.Where(l => !validador.Validar(l).EsValido);
    }
}
```

`CatalogoLimites` utiliza el validador para separar propuestas válidas e inválidas. La clase no conoce la red, el almacenamiento ni la topología real de AceriaApp. Su función permite comprobar que un mismo criterio se aplica de forma consistente a varias entradas.

La clase CatalogoLimites agrupa los límites de servicio del proyecto y permite filtrarlos por validez. Es el punto de entrada para auditar los límites del monolito antes de extraer microservicios.

Errores comunes:

- No inicializar _limites en el constructor: la clase lanza NullReferenceException.
- Olvidar using System.Linq;: Where no se resuelve.
- Devolver List en lugar de IEnumerable: expone la implementación interna.

Verificación: la clase CatalogoLimites compila sin errores y expone un constructor y dos métodos.

Interpretación del resultado: El catálogo no corrige límites inválidos: los clasifica a partir del resultado del validador para orientar una revisión de arquitectura.

Paso 10 — Comprobar el using necesario en CatalogoLimites

Abre el archivo de **código fuente** `src/AceriaApp.Monolito.Api/Diseno/CatalogoLimites.cs`; no abras por error el `.csproj` de tests para resolver un `using`. Comprueba las directivas existentes (System.Linq) que llegaron con la activación del bloque 1.3.9.

No dupliques imports; localiza las operaciones de filtrado del catálogo que utilizan LINQ o colecciones. Guarda y compila el proyecto API. El resultado de este paso es una importación correcta en `CatalogoLimites.cs`, no un cambio NuGet.

El método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoLimites.cs compila sin errores y no muestra subrayados rojos.

Paso 11 — Comprobar el proyecto de tests del punto 1.3

En `tests/AceriaApp.Diseno.Tests`, localiza `AceriaApp.Diseno.Tests.csproj`: es un proyecto de tests que ya viene incluido, no debes ejecutar otra plantilla xUnit encima. Ejecuta `dotnet sln AceriaApp.sln list` y localiza `AceriaApp.Diseno.Tests` junto a los proyectos anteriores.

Abre el `.csproj` y comprueba `net8.0`, xUnit, FluentAssertions y la referencia de proyecto a la API monolítica. Ejecuta `dotnet restore AceriaApp.sln`; las clases de tests comentadas se activarán en los pasos 14 y 15.

El punto 1.3 añade un proyecto de tests específico para las clases de diseño. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar los proyectos de tests del punto 1.2: viola la regla de no modificar archivos del punto anterior.
- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Diseno.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Diseno.Tests.

Paso 12 — Comprobar la referencia del proyecto de tests a la API monolítica

Abre `tests/AceriaApp.Diseno.Tests/AceriaApp.Diseno.Tests.csproj` y localiza el `ProjectReference` hacia `AceriaApp.Monolito.Api.csproj`. Comprueba que la referencia se dirige desde tests hacia API y que su ruta relativa existe físicamente.

Ejecuta `dotnet sln AceriaApp.sln list` y confirma que los proyectos de la etapa están incluidos en la misma solución. No agregues la referencia dos veces; restaura con `dotnet restore AceriaApp.sln` si Visual Studio no resuelve el proyecto.

La referencia permite que el proyecto de tests acceda a las clases de Diseno del proyecto monolítico. Sin esta referencia, los tests no pueden instanciar LimiteServicio, ValidadorLimiteServicio ni CatalogoLimites.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Diseno.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 13 — Comprobar el paquete FluentAssertions al proyecto de tests

Abre `tests/AceriaApp.Diseno.Tests/AceriaApp.Diseno.Tests.csproj` y localiza la dependencia `FluentAssertions` existente; no la instales dos veces en este proyecto preparado. Ejecuta `dotnet list tests/AceriaApp.Diseno.Tests/AceriaApp.Diseno.Tests.csproj package` desde la raíz de la etapa y comprueba que el paquete aparece con su versión.

Revisa que los tests de `ValidadorLimiteServicio` y `CatalogoLimites` usan `FluentAssertions` para expresar sus resultados esperados. Ejecuta `dotnet restore AceriaApp.sln` antes de activar los tests; si NuGet falla revisa la restauración y no cambies las aserciones.

FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 14 — Implementar y comprender ValidadorLimiteServicioTests.cs

El archivo preparado es `tests/AceriaApp.Diseno.Tests/ValidadorLimiteServicioTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.Diseno.Tests/ValidadorLimiteServicioTests.cs` en Visual Studio y localiza los delimitadores `INICIO PASO 1.3.14` y `FIN PASO 1.3.14`. Selecciona las líneas de C# entre ambos marcadores, pulsa **Ctrl+K, Ctrl+U** y deja comentados los fragmentos del reto opcional.

Lee cada escenario positivo y negativo y vincúlalo a las reglas de `ValidadorLimiteServicio`: detecta dependencias excesivas, datos compartidos y límites que exponen entidades. Guarda y compila el proyecto `AceriaApp.Diseno.Tests`; corrige clases incompletas o imports antes de modificar las expectativas de las pruebas.

```
using AceriaApp.Monolito.Api.Diseno;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Diseno.Tests;

public class ValidadorLimiteServicioTests
{
    private readonly ValidadorLimiteServicio _validador = new();

    [Fact]
    public void Validar_LimiteCompleto_DevuelveValido()
    {
        var limite = new LimiteServicio
        {
            Nombre = "Ordenes",
            CapacidadDeNegocio = "Gestion de ordenes de produccion",
            ContextoDelimitado = "Produccion",
            NumeroDeConceptos = 2,
            NumeroDeDependenciasSalientes = 1,
            ExponeEntidadesDeDominio = false,
            ComparteBaseDeDatos = false
        };

        var resultado = _validador.Validar(limite);

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinCapacidadDeNegocio_DevuelveError()
    {
        var limite = new LimiteServicio
        {
            Nombre = "Ordenes",
            CapacidadDeNegocio = string.Empty,
            ContextoDelimitado = "Produccion"
        };

        var resultado = _validador.Validar(limite);
```

```
        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("capacidad de negocio"));
    }

    [Fact]
    public void Validar_ConMuchosConceptos_DevuelveErrorDeCohesion()
    {
        var limite = new LimiteServicio
        {
            CapacidadDeNegocio = "Gestion multiple",
            ContextoDelimitado = "Produccion",
            NumeroDeConceptos = 5
        };

        var resultado = _validador.Validar(limite);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("cohesión baja"));
    }
}
```

`ValidadorLimiteServicioTests` prueba un ejemplo completo y distintos incumplimientos de las reglas. Cada aserción debe poder asociarse con una condición del método `Validar`. Los tests hacen verificable el contrato programado, aunque no sustituyen la evaluación de una frontera de negocio real.

los tests verifican que el validador acepta un límite completo y rechaza los límites que violan los principios de responsabilidad única, cohesión y contexto delimitado.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Diseno;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de límite válido: el test solo verifica los errores.

Verificación: el archivo ValidadorLimiteServicioTests.cs compila y contiene tres métodos de test.

Paso 15 — Implementar y comprender CatalogoLimitesTests.cs

El archivo preparado es `tests/AceriaApp.Diseno.Tests/CatalogoLimitesTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.Diseno.Tests/CatalogoLimitesTests.cs` y busca `INICIO PASO 1.3.15` y el fin del mismo bloque. Descomenta solo las declaraciones internas con **Ctrl+K, Ctrl+U**, conservando comentados los bloques de reto y evitando duplicar la clase de tests.

Analiza qué datos del escenario producen los resultados de `LimitesValidos y LimitesInvalidos` y por qué el catálogo usa el validador o el indicador creado en pasos anteriores. Guarda y compila el proyecto de tests, comprobando que no requiere modificar fuentes del punto anterior.

```
using AceriaApp.Monolito.Api.Diseno;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Diseno.Tests;

public class CatalogoLimitesTests
{
    [Fact]
    public void LimitesValidos_FiltraCorrectamente()
    {
        var limites = new List<LimiteServicio>
        {
            new()
```

```
            {
                Nombre = "Ordenes",
                CapacidadDeNegocio = "Gestion de ordenes",
                ContextoDelimitado = "Produccion",
                NumeroDeConceptos = 2,
                NumeroDeDependenciasSalientes = 1
            },
            new()
            {
                Nombre = "Cajon",
                CapacidadDeNegocio = "Multiples",
                ContextoDelimitado = "Mixto",
                NumeroDeConceptos = 5,
                NumeroDeDependenciasSalientes = 5
            }
        };
        var catalogo = new CatalogoLimites(limites);
        var validador = new ValidadorLimiteServicio();

        var validos = catalogo.LimitesValidos(validador).ToList();
        var invalidos = catalogo.LimitesInvalidos(validador).ToList();

        validos.Should().HaveCount(1);
        validos[0].Nombre.Should().Be("Ordenes");
        invalidos.Should().HaveCount(1);
        invalidos[0].Nombre.Should().Be("Cajon");
    }
}
```

`CatalogoLimitesTests` comprueba que el catálogo conserva en cada colección los límites que corresponden según el validador. No configura soluciones ni crea servicios: filtra propuestas. La distinción resulta importante al interpretar lo que demuestra la prueba.

El test verifica que el catálogo filtra correctamente los límites válidos y los inválidos según el validador.

Errores comunes:

- Olvidar using System.Collections.Generic;: List no se resuelve.
- Olvidar using System.Linq;: ToList() no se resuelve.
- No comprobar el nombre del límite válido: el test podría pasar por casualidad.

Verificación: el archivo CatalogoLimitesTests.cs compila y contiene un método de test.

Paso 16 — Compilar la solución completa

En la terminal de `1.3-principios-diseno`, donde está `AceriaApp.sln`, ejecuta `dotnet build AceriaApp.sln -c Release`. Comprueba que compilan los 5 proyectos acumulados y que el resumen muestra **0 errores**.

Ante un error, identifica primero archivo y línea; localiza el marcador `INICIO PASO` responsable antes de intentar crear una clase nueva. Conserva la salida de compilación y verifica que los proyectos heredados no han cambiado respecto a `1.2-ventajas-inconvenientes`.

compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.3

Abre el **Explorador de pruebas** de Visual Studio o una terminal en la carpeta de la solución 1.3. Ejecuta `dotnet test tests/AceriaApp.Diseno.Tests/AceriaApp.Diseno.Tests.csproj -c Release` para comprobar únicamente los tests nuevos de la etapa.

Verifica que pasan **4 pruebas** de `AceriaApp.Diseno.Tests` con 0 fallos; relaciona cada caso con `ValidadorLimiteServicio` o `CatalogoLimites`. Si falla algún test, consulta el primer mensaje de aserción y examina los bloques activados en los pasos 14 y 15 sin cambiar valores esperados a ciegas.

ejecutar los tests del punto 1.3 verifica que las clases de diseño funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 4, Skipped: 0, Total: 4.

Paso 18 — Ejecutar todos los tests de la solución

En la carpeta de `AceriaApp.sln` ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después del build Release del paso 16. Comprueba que pasan **13 pruebas acumuladas**: 9 heredadas más 4 de la etapa 1.3.

Si no se encuentran los ensamblados, vuelve a compilar en Release y repite el comando sin mezclar resultados Debug y Release. Guarda el resumen: los tests heredados deben seguir pasando, pues una nueva etapa no puede romper las capacidades previas de AceriaApp.

ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1, 1.2 y 1.3 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 13, Skipped: 0, Total: 13.

Paso 19 — Revisar el documento preparado: README.md

Abre el archivo `README.md` de la carpeta `1.3-principios-diseno`, no el del punto `1.2-ventajas-inconvenientes`. Localiza en el documento el objetivo de la etapa y los archivos `LimiteServicio`, `ValidadorLimiteServicio`, `CatalogoLimites` y `AceriaApp.Diseno.Tests`.

Comprueba que las rutas de ejecución y tests mencionadas corresponden a los proyectos físicos de esta etapa. Anota qué información nueva aporta este README respecto al anterior; no lo sobrescribas con la versión heredada.

```
# Punto 1.3 — Principios de diseño

## Objetivo
Comprender los principios de diseño que guían la descomposición de un sistema en servicios autónomos y añadir
al proyecto las clases de validación de límites de servicio.
```

```
## Estado del proyecto
- Solución `AceriaApp.sln` con cuatro proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests`,
`AceriaApp.Metricas.Tests` y `AceriaApp.Diseno.Tests`.
- Documento `docs/principios-diseno.md` con los ocho principios y su aplicación al proyecto.
- Clases `LimiteServicio`, `ValidadorLimiteServicio` y `CatalogoLimites` en `src/AceriaApp.Monolito.Api/
Diseno/`.
- Cuatro tests nuevos en `tests/AceriaApp.Diseno.Tests`.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/principios-diseno.md`.
- Carpeta `Diseno` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.Diseno.Tests` con cuatro tests nuevos.
- No se ha modificado ningún archivo del punto 1.2.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [ADR-002](docs/ADR-002-metricas-acoplamiento.md) (heredado del punto 1.2).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md) (heredado del punto 1.2).
- [Principios de diseño](docs/principios-diseno.md).
```

el README del punto 1.3 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.2.

Errores comunes:

- Modificar el README del punto 1.2: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 1.2: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.3 con las cinco secciones.

Paso 20 — Revisar el documento preparado: ADR-003-principios-diseno.md

Abre `docs/ADR-003-principios-diseno.md` dentro de `1.3-principios-diseno` en el editor Markdown; el archivo se entrega preparado. Lee Estado, Contexto, Decisión, Alternativas y Consecuencias, vinculando la decisión con límites de servicio, autonomía y cohesión.

Identifica un resultado de los tests de 1.3 que apoye la decisión y una limitación que el ejemplo no demuestra. Comprueba que el ADR del punto anterior permanece presente e idéntico: cada decisión arquitectónica se documenta por separado.

```
# ADR-003: Aplicación de principios de diseño a los límites de servicio

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado un monolito modular y ha definido métricas de acoplamiento. Es necesario
aplicar principios de diseño explícitos para decidir los límites entre módulos y preparar la extracción
futura de microservicios.

## Decisión
Se adoptan ocho principios: responsabilidad única, ocultación de información, acoplamiento débil y cohesión
fuerte, límite del contexto, autonomía, contrato explícito, tolerancia al fallo y evolución independiente. Se
implementa un validador automático que audita los límites de servicio del monolito.

## Alternativas consideradas
- Aplicar los principios sin validación automática: descartado por falta de trazabilidad.
- Aplicar solo algunos principios: descartado por incoherencia.
```

```
## Consecuencias
- Positivas: decisiones trazables, auditoría automática, preparación para la extracción.
- Negativas: el validador puede requerir ajuste de umbrales.
- Mitigación: revisar los umbrales al final de cada módulo.
```

el ADR-003 documenta la decisión de aplicar principios de diseño y de validarlos automáticamente. Es un artefacto nuevo del punto 1.3 que no modifica los ADR existentes.

Errores comunes:

- Modificar el ADR-001 o el ADR-002: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-003: rompe la convención.

Verificación: el archivo docs\ADR-003-principios-diseno.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.2

En tu copia del repositorio localiza las carpetas `docs` de `1.2-ventajas-inconvenientes` y `1.3-principios-diseno`. Selecciona un fichero **heredado** presente en ambas carpetas y compara su contenido con `fc /L` en Windows o un hash SHA-256.

Los documentos heredados deben ser idénticos; el `README.md` de 1.3 puede ser diferente porque explica la etapa actual. Registra el resultado y no modifiques los originales de `1.2-ventajas-inconvenientes` para forzar igualdad.

La comparación con fc verifica que los documentos del punto 1.2 no se han modificado.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los dos documentos del punto 1.2 no muestran diferencias.

Paso 22 — Verificar la estructura del punto 1.3

Desde `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO` ejecuta `tree /F 1.3-principios-diseno` en una terminal Windows. Comprueba en `src/AceriaApp.Monolito.Api/Diseno` los tres tipos `LimiteServicio.cs`, `ValidadorLimiteServicio.cs` y `CatalogoLimites.cs`.

En `tests/AceriaApp.Diseno.Tests` localiza las pruebas del validador y catálogo; en `docs` localiza `principios-diseno.md`, `ADR-003-principios-diseno.md` y los documentos heredados. Distingue los archivos fuente de las carpetas temporales `bin` y `obj`, que no deben incorporarse como entregables.

La verificación estructural confirma que el punto 1.3 añade archivos nuevos sin modificar los existentes.

Errores comunes:

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 1.2: viola la regla de no modificar.
- Omitir la carpeta Diseno: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.2.

Paso 23 — Revisar el documento preparado: .gitignore

Abre el `.gitignore` de la carpeta `1.3-principios-diseno` en un editor que muestre ficheros ocultos. Comprueba que conserva las exclusiones `bin/`, `obj/`, `.vs/`, resultados y archivos de usuario sin ignorar el código fuente.

Ejecuta `git status --short` en tu copia personal tras compilar y confirma que no aparecen binarios temporales. No sustituyas el fichero ya incluido por una plantilla ajena; verifica que conserva la política del punto anterior.

el .gitignore se hereda del punto 1.2 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

- Modificar el .gitignore copiado del punto 1.2: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

Paso 24 — Hacer el commit del punto 1.3 y verificar el estado final

Desde tu copia personal (no la rama del profesor) ejecuta `git status` para identificar los cambios de `1.3-principios-diseno`. Revisa `git diff --stat` y prepara con `git add` solo archivos de esta etapa, sin incluir carpetas compiladas ni secretos.

Realiza el commit del punto 1.3 con un mensaje descriptivo de límites de servicio, autonomía y cohesión; no sobrescribas los commits de etapas previas. Comprueba `git log -1 --oneline`, `git status` y conserva el resultado de los 13 tests acumulados como evidencia final.

El commit del punto 1.3 registra el estado acumulado del proyecto. Los tres commits consecutivos permiten comparar los puntos 1.1, 1.2 y 1.3 con git diff.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra tres commits: Punto 1.1, Punto 1.2 y Punto 1.3. git status muestra nothing to commit, working tree clean.

Nota de trazabilidad: en una descarga del repositorio puede no haber cambios nuevos que confirmar. Antes de ejecutar `git commit`, usar `git status` y confirmar únicamente los archivos efectivamente modificados durante el recorrido. No ejecutar `git init` dentro de cada punto: la raíz del repositorio ya contiene el historial.

Reto de ampliación: solución guiada

Problema a resolver: añadir un método Resumen a CatalogoLimites que devuelva el número de límites válidos, el número de límites inválidos y el porcentaje de validez, y un test que verifique el cálculo cuando hay tres límites válidos y uno inválido.

Etapa 1 — En CatalogoLimites.cs, añadir el siguiente record al final del archivo:

```
public record ResumenLimites(int Validos, int Invalidos, double PorcentajeValidez);
```

Etapa 2 — Explicar línea por línea:

- **Línea 1:** `public record ResumenLimites(` — declara un contrato inmutable para el resumen.
- **Línea 2:** `int Validos,` — número de límites válidos.
- **Línea 3:** `int Invalidos,` — número de límites inválidos.
- **Línea 4:** `double PorcentajeValidez);` — porcentaje de validez sobre el total.

Etapa 3 — En CatalogoLimites.cs, añadir el siguiente método dentro de la clase:

```
public ResumenLimites Resumen(ValidadorLimiteServicio validador)
{
    var validos = _limites.Count(l => validador.Validar(l).EsValido);
    var invalidos = _limites.Count - validos;
    var total = _limites.Count;
    var porcentaje = total == 0 ? 0 : (double)validos / total * 100;
    return new ResumenLimites(validos, invalidos, porcentaje);
}
```

Etapa 4 — Explicar línea por línea:

- **Línea 1:** `public ResumenLimites Resumen(ValidadorLimiteServicio validador)` — declara el método.
- **Línea 2:** `{` — abre el cuerpo del método.
- **Línea 3:** `var validos = _limites.Count(l => validador.Validar(l).EsValido);` — cuenta los válidos.
- **Línea 4:** `var invalidos = _limites.Count - validos;` — calcula los inválidos por diferencia.
- **Línea 5:** `var total = _limites.Count;` — obtiene el total.
- **Línea 6:** `var porcentaje = total == 0 ? 0 : (double)validos / total * 100;` — calcula el porcentaje con protección contra división por cero.
- **Línea 7:** `return new ResumenLimites(validos, invalidos, porcentaje);` — devuelve el resumen.
- **Línea 8:** `}` — cierra el cuerpo del método.

Etapa 5 — Guardar el archivo con Ctrl+S.

Etapa 6 — En CatalogoLimitesTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresValidosYUnInvalido_CalculaPorcentaje()
{
    var limites = new List<LimiteServicio>
    {
        new() { CapacidadDeNegocio = "A", ContextoDelimitado = "X", NumeroDeConceptos = 1,
NumeroDeDependenciasSalientes = 1 },
        new() { CapacidadDeNegocio = "B", ContextoDelimitado = "X", NumeroDeConceptos = 1,
NumeroDeDependenciasSalientes = 1 },
        new() { CapacidadDeNegocio = "C", ContextoDelimitado = "X", NumeroDeConceptos = 1,
NumeroDeDependenciasSalientes = 1 },
        new() { CapacidadDeNegocio = string.Empty, ContextoDelimitado = "X" }
    };
    var catalogo = new CatalogoLimites(limites);
    var validador = new ValidadorLimiteServicio();

    var resumen = catalogo.Resumen(validador);

    resumen.Validos.Should().Be(3);
    resumen.Invalidos.Should().Be(1);
    resumen.PorcentajeValidez.Should().Be(75.0);
}
```

Etapa 7 — Explicar línea por línea:

- **Línea 1:** `[Fact]` — marca el método como test de xUnit.
- **Línea 2:** `public void Resumen_ConTresValidosYUnInvalido_CalculaPorcentaje()` — declara el test.
- **Línea 3:** `{` — abre el cuerpo del test.
- **Línea 4:** `var limites = new List<LimiteServicio>` — crea la lista.
- **Línea 5:** `{` — abre el inicializador.
- **Línea 6:** `new() { CapacidadDeNegocio = "A", ... }` — primer límite válido.
- **Línea 7:** `new() { CapacidadDeNegocio = "B", ... }` — segundo límite válido.
- **Línea 8:** `new() { CapacidadDeNegocio = "C", ... }` — tercer límite válido.
- **Línea 9:** `new() { CapacidadDeNegocio = string.Empty, ... }` — límite inválido por falta de capacidad.
- **Línea 10:** `};` — cierra el inicializador.
- **Línea 11:** `var catalogo = new CatalogoLimites(limites);` — instancia el catálogo.
- **Línea 12:** `var validador = new ValidadorLimiteServicio();` — instancia el validador.
- **Línea 13:** `var resumen = catalogo.Resumen(validador);` — invoca el método.
- **Línea 14:** `resumen.Validos.Should().Be(3);` — verifica los válidos.
- **Línea 15:** `resumen.Invalidos.Should().Be(1);` — verifica los inválidos.
- **Línea 16:** `resumen.PorcentajeValidez.Should().Be(75.0);` — verifica el porcentaje.
- **Línea 17:** `}` — cierra el cuerpo del test.

Etapa 8 — Ejecutar dotnet test tests\AceriaApp.Diseno.Tests\AceriaApp.Diseno.Tests.csproj y verificar que pasan los cinco tests.

Etapa 9 — Verificar que el método Resumen no modifica ningún archivo del punto 1.2.

Etapa 10 — Hacer commit con git commit -m "Reto 1.3: método Resumen en CatalogoLimites con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 5.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-1-introduccion-microservicios/1.3-principios-diseno/ con la solución AceriaApp.sln que contiene cinco proyectos.
- El documento docs/principios-diseno.md con los ocho principios y su aplicación al proyecto.
- El documento docs/ADR-003-principios-diseno.md con la decisión sobre la validación automática.
- Las clases LimiteServicio, ValidadorLimiteServicio y CatalogoLimites en src/AceriaApp.Monolito.Api/Diseno/.
- El proyecto tests/AceriaApp.Diseno.Tests con cuatro tests (cinco tras el reto).
- Los tests de los puntos 1.1, 1.2 y 1.3 pasando conjuntamente con dotnet test AceriaApp.sln.
- Los ADR-001 y ADR-002 del punto 1.2 sin modificar.
- Tres commits en Git: el del punto 1.1, el del punto 1.2 y el del punto 1.3.

Conclusión y enlace al siguiente punto

El punto 1.3 ha completado el bloque de fundamentos con los principios de diseño que guían la descomposición de un sistema en servicios autónomos. El proyecto cuenta ahora con métricas de acoplamiento y con un validador automático de límites de servicio que aplica los principios de responsabilidad, ocultación, cohesión, acoplamiento, contexto, autonomía, contrato y tolerancia. Esta base permite avanzar hacia el punto 1.4, donde se estudiará la organización concreta de una solución basada en microservicios: estructura de carpetas, convenciones de nombres y separación entre servicios.

Fin del punto 1.3.

Punto 1.4 — Organización de una solución basada en microservicios

Dar nombres comprensibles a los proyectos sin esconder su arquitectura

El nombre de un proyecto es una señal útil para quien mantiene una solución: puede indicar si estamos viendo una API, una biblioteca de dominio o un adaptador. Sin embargo, una denominación correcta no impide referencias inadecuadas ni garantiza límites de despliegue. En este punto separaremos esos dos planos y estudiaremos qué aportan las convenciones sin transformarlas en reglas de negocio.

Sobre el estado acumulado de AceriaApp, implementaremos `ConvencionNombres` para representar la denominación, `ValidadorConvencionNombres` para explicar por qué una propuesta cumple o incumple la convención del curso y `CatalogoProyectos` para clasificar los resultados. La práctica conservará sus pruebas y sus documentos de decisión; no necesita crear nuevos servicios porque aparezcan nombres distintos.

La comprobación final permitirá identificar nombres válidos e inválidos y reconocer los límites del validador. Para evaluar de verdad la separación entre proyectos habrá que estudiar también sus referencias, contratos y dependencias.

Desarrollo práctico

Ejercicio: Añadir al punto 1.3 un documento de organización de la solución en docs/, una clase de validación de convenciones de nombres en el proyecto monolítico y un test unitario que verifique la validación. El punto 1.4 parte del contenido del punto 1.3 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: utiliza la carpeta real de tu copia del repositorio en Windows; `C:\cursos\` solo representa una ubicación posible.AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Localizar el proyecto independiente del punto 1.4

Localiza `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO/1.4-organizacion-solucion` en tu copia del repositorio; no sobrescribas la carpeta del punto anterior `1.3-principios-diseno`. Comprueba `AceriaApp.sln`, `src`, `tests`, `docs` y `README.md`; las clases nuevas están preparadas, pero parte del código permanece comentado.

Abre la terminal en esa misma carpeta y ejecuta `dotnet --list-sdks` para comprobar la disponibilidad del SDK .NET 8. Registra la ruta de trabajo y diferencia el proyecto distribuido de uno vacío que no debes crear otra vez.

Cada punto constituye una solución autónoma. Se evita copiar la solución anterior sobre archivos nuevos ya preparados y se garantiza que los bloques de código comentados correspondan exactamente a este punto.

Errores comunes: abrir la solución del punto anterior; copiar archivos que sobrescriban los bloques preparados; confundir la carpeta del repositorio con la carpeta del proyecto.

Verificación: `AceriaApp.sln` existe y la terminal está situada dentro de la carpeta de este punto.

Paso 2 — Abrir la solución y comprobar los proyectos heredados

Abre en Visual Studio el archivo `AceriaApp.sln` de `1.4-organizacion-solucion` desde **Archivo > Abrir > Proyecto o solución**. Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 6 proyectos acumulados, incluido `AceriaApp.Organizacion.Tests`.

Expande la carpeta `src/AceriaApp.Monolito.Api/Organizacion` y observa que los tipos de esta etapa tienen marcadores `INICIO PASO` sin activar. Ejecuta `dotnet build AceriaApp.sln` y guarda la salida de la compilación inicial antes de editar ninguna clase.

se identifica el estado heredado ejecutable y se evita confundir una clase comentada con un proyecto todavía no creado.

Errores comunes: abrir la carpeta de otro punto; contar únicamente los proyectos con pruebas activas; descomentar todos los bloques a la vez.

Verificación: la solución abre y compila antes de introducir los cambios del punto.

Paso 3 — Reconocer documentación heredada y nuevas fuentes de Organizacion

Abre `docs` en la carpeta `1.4-organizacion-solucion` y comprueba que conserva los documentos del punto anterior `1.3-principios-diseno`. Identifica los archivos nuevos `docs/organizacion-solucion.md` y `docs/ADR-004-organizacion-solucion.md` que vienen incluidos como material preparado.

Contrasta uno de los ADR heredados con el fichero homónimo de la etapa anterior, sin reescribirlo. Anota qué archivos son heredados y cuáles nuevos; no exijas una carpeta `docs` vacía antes de comenzar, porque los documentos ya están entregados.

El punto 1.4 parte del contenido del punto 1.3, que ya incluye la carpeta docs con cinco documentos. No se modifica ningún documento existente.

Errores comunes:

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 1.3.
- Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los cinco documentos del punto 1.3 sin modificaciones.

Paso 4 — Revisar el documento preparado: organizacion-solucion.md

Abre `docs/organizacion-solucion.md` en Visual Studio con la vista Markdown, sin crear otro archivo de igual nombre. Localiza los apartados sobre organización de proyectos y convenios de nombres y vincula dos de sus decisiones con AceriaApp.

Elabora una tabla breve de ventajas, riesgos y criterios técnicos con un ejemplo aplicable al dominio de órdenes de producción. Contrasta la estructura con el texto incluido en esta práctica y guarda tus conclusiones en un archivo personal separado.

```
# Organización de la solución — Punto 1.4

## Tres decisiones de organización

1. Estructura de repositorios.
2. Estructura de carpetas.
3. Convenciones de nombres.

## Estructura de repositorios

Monorepo con una carpeta por módulo y una carpeta por punto.

## Estructura de carpetas

Cada punto contiene `src/`, `tests/` y `docs/`.

## Convenciones de nombres de proyectos

Patrón: `AceriaApp.Componente.Tipo`.

Tipos: `Api`, `Domain`, `Infrastructure`, `Application`, `Tests`.

## Convenciones de espacios de nombres

El espacio de nombres coincide con la estructura de carpetas del proyecto.

## Convenciones de commits

Patrón: `Punto N.M: descripción`.

## Convenciones de ramas

Prefijos: `feature/`, `fix/`, `docs/`, `refactor/`.

## Estructura con varios servicios

Cada servicio tiene su propio conjunto de proyectos y su propio conjunto de tests.

## Aplicación al proyecto del curso

| Decisión | Aplicación |
|----------|------------|
| Monorepo | `AceriaApp/` con módulos y puntos |
| Carpetas | `src/`, `tests/`, `docs/` |
| Nombres | `AceriaApp.Componente.Tipo` |
| Namespaces | Coinciden con la carpeta |
| Commits | `Punto N.M: descripción` |
```

El documento de organización de la solución es el artefacto de docs específico del punto 1.4. No modifica los cinco documentos existentes del punto 1.3.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\organizacion-solucion.md existe con las nueve secciones.

Paso 5 — Localizar la carpeta de código ya preparada

En el Explorador de soluciones abre `src/AceriaApp.Monolito.Api/Organizacion` y localiza `ConvencionNombres.cs`, `ValidadorConvencionNombres.cs` y `CatalogoProyectos.cs`. En cada archivo identifica el marcador `INICIO PASO 1.4.6`, `1.4.7` o `1.4.9`; comprueba que las declaraciones nuevas están comentadas.

Confirma que las clases heredadas de Dominio e Infraestructura siguen activas y no se deben modificar. Planifica la secuencia de activación: primero `ConvencionNombres`, después `ValidadorConvencionNombres` y finalmente `CatalogoProyectos`.

La carpeta ya existe en el proyecto preparado; la progresión didáctica se logra activando el contenido, no creando archivos duplicados.

Errores comunes: crear un archivo con el mismo nombre; descomentar código de otros pasos; modificar archivos heredados.

Verificación: la carpeta `Organizacion` y los archivos del punto están presentes, inicialmente comentados.

Paso 6 — Implementar y comprender ConvencionNombres.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Organizacion/ConvencionNombres.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/Organizacion/ConvencionNombres.cs` y localiza los delimitadores `INICIO PASO 1.4.6` y `FIN PASO 1.4.6`. Selecciona solo las líneas C# interiores, pulsa **Ctrl+K, Ctrl+U** una vez y conserva los marcadores como comentarios.

Comprueba que aparecen las propiedades Prefijo, Componente, Tipo y NombreCompleto y explica qué describe cada una. Guarda y compila `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` sin crear una segunda declaración de `ConvencionNombres`.

```
namespace AceriaApp.Monolito.Api.Organizacion;

public class ConvencionNombres
{
    public string Prefijo { get; init; } = "AceriaApp";
    public string Componente { get; init; } = string.Empty;
    public string Tipo { get; init; } = string.Empty;

    public string NombreCompleto()
    {
        return $"{Prefijo}.{Componente}.{Tipo}";
    }
}
```

`ConvencionNombres` reúne las partes necesarias para formar un nombre de proyecto según la convención del curso. La composición debe producir un identificador comprensible y estable. Ese resultado mejora la lectura de una solución, pero no restringe de manera automática las referencias entre proyectos.

La clase ConvencionNombres modela la convención de nombres de proyectos y expone un método que devuelve el nombre completo. Es la base para validar que todos los proyectos del curso siguen la convención.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.
- No inicializar Prefijo con "AceriaApp": la convención no se aplica por defecto.

- Usar set en lugar de init: permite modificar las propiedades después de crear la instancia.

Verificación: la clase ConvencionNombres compila sin errores y expone tres propiedades y un método.

Paso 7 — Implementar y comprender ValidadorConvencionNombres.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Organizacion/ValidadorConvencionNombres.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/Organizacion/ValidadorConvencionNombres.cs` y localiza el bloque delimitado para el paso 1.4.7. Descomenta únicamente su código con **Ctrl+K, Ctrl+U** y deja inactivos los miembros marcados como `RETO`.

Sigue la lógica de `Validar` para prefijo correcto, tres partes y tipo permitido; identifica qué condición genera cada error. Guarda y compila; comprueba que la clase devuelve su tipo de resultado de validación y no depende de un reto opcional.

```
namespace AceriaApp.Monolito.Api.Organizacion;

public class ValidadorConvencionNombres
{
    private readonly string _prefijo;
    private readonly IReadOnlyList<string> _tiposPermitidos;

    public ValidadorConvencionNombres(string prefijo, IReadOnlyList<string> tiposPermitidos)
    {
        _prefijo = prefijo;
        _tiposPermitidos = tiposPermitidos;
    }

    public ResultadoConvencion Validar(string nombreProyecto)
    {
        var errores = new List<string>();

        if (!nombreProyecto.StartsWith(_prefijo + ".", StringComparison.Ordinal))
        {
            errores.Add($"El proyecto no empieza por '{_prefijo}.'.");
        }

        var partes = nombreProyecto.Split('.');
        if (partes.Length != 3)
        {
            errores.Add("El nombre no tiene el formato 'Prefijo.Componente.Tipo'.");
        }
        else if (!_tiposPermitidos.Contains(partes[2]))
        {
            errores.Add($"El tipo '{partes[2]}' no está permitido.");
        }

        return new ResultadoConvencion(errores.Count == 0, errores);
    }
}

public record ResultadoConvencion(bool EsValido, IReadOnlyList<string> Errores);
```

`ValidadorConvencionNombres` descompone una denominación y comprueba las condiciones de prefijo, estructura y tipo permitido. Los errores indican qué convención se incumplió, no que el proyecto compilado contenga un defecto funcional. Podemos cambiar un nombre sin haber modificado una sola dependencia arquitectónica.

La clase ValidadorConvencionNombres comprueba que un nombre de proyecto cumple el patrón Prefijo.Componente.Tipo y que el tipo pertenece a la lista de tipos permitidos. Es el mecanismo de auditoría de la convención de nombres.

Errores comunes:

- Olvidar el using System.Collections.Generic;: IReadOnlyList y List no se resuelven.
- Usar Contains sin using System.Linq;: el método no se resuelve en listas.
- No comprobar la longitud de las partes: el validador acepta nombres mal formados.

Verificación: la clase ValidadorConvencionNombres compila sin errores y expone un constructor, un método y un record.

Interpretación del resultado: El nombre tiene tres segmentos y el último pertenece a un conjunto permitido. El validador ilustra reglas de nomenclatura, no comprueba la estructura física del repositorio.

Paso 8 — Añadir los using necesarios en ValidadorConvencionNombres

En `src/AceriaApp.Monolito.Api/Organizacion/ValidadorConvencionNombres.cs`, examina las directivas `using` que quedaron activas al completar el paso 1.4.7. Comprueba que ya aparecen los imports necesarios (System.Collections.Generic, System.Linq); no agregues una segunda copia si están presentes.

Relaciona cada import con las colecciones y los métodos empleados en `Validar`. Guarda y compila; si hay errores de tipos, comprueba primero el bloque de activación antes de cambiar las dependencias.

los tipos IReadOnlyList y List pertenecen a System.Collections.Generic. El método Contains pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir los usings explícitos.

Errores comunes:

- Añadir solo System.Collections.Generic y olvidar System.Linq: Contains no se resuelve.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorConvencionNombres.cs compila sin errores y no muestra subrayados rojos.

Paso 9 — Implementar y comprender CatalogoProyectos.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/Organizacion/CatalogoProyectos.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/Organizacion/CatalogoProyectos.cs` y busca los marcadores del paso 1.4.9. Descomenta solo las líneas interiores con **Ctrl+K, Ctrl+U**, conservando inactivo el bloque de `RETO`.

Comprueba que el catálogo implementa `ProyectosValidos y ProyectosInvalidos` y usa los tipos de los pasos anteriores. Guarda y compila, revisando que los métodos no crean una segunda clase con el nombre `CatalogoProyectos`.

```
namespace AceriaApp.Monolito.Api.Organizacion;

public class CatalogoProyectos
{
    private readonly List<string> _nombres;

    public CatalogoProyectos(List<string> nombres)
    {
        _nombres = nombres;
    }

    public IEnumerable<string> ProyectosValidos(ValidadorConvencionNombres validador)
```

```
    {
        return _nombres.Where(n => validador.Validar(n).EsValido);
    }

    public IEnumerable<string> ProyectosInvalidos(ValidadorConvencionNombres validador)
    {
        return _nombres.Where(n => !validador.Validar(n).EsValido);
    }
}
```

`CatalogoProyectos` filtra una colección de propuestas de nombres a partir de los resultados del validador. No crea archivos `.csproj` ni inventaría procesos iniciados. El alumno debe relacionar cada clasificación con la regla que la produjo.

La clase CatalogoProyectos agrupa los nombres de los proyectos del curso y permite filtrarlos por validez según la convención.

Errores comunes:

- No inicializar _nombres en el constructor: la clase lanza NullReferenceException.
- Olvidar using System.Linq;: Where no se resuelve.
- Devolver List en lugar de IEnumerable: expone la implementación interna.

Verificación: la clase CatalogoProyectos compila sin errores y expone un constructor y dos métodos.

Paso 10 — Comprobar el using necesario en CatalogoProyectos

Abre el archivo de **código fuente** `src/AceriaApp.Monolito.Api/Organizacion/CatalogoProyectos.cs`; no abras por error el `.csproj` de tests para resolver un `using`. Comprueba las directivas existentes (System.Linq) que llegaron con la activación del bloque 1.4.9.

No dupliques imports; localiza las operaciones de filtrado del catálogo que utilizan LINQ o colecciones. Guarda y compila el proyecto API. El resultado de este paso es una importación correcta en `CatalogoProyectos.cs`, no un cambio NuGet.

El método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoProyectos.cs compila sin errores y no muestra subrayados rojos.

Paso 11 — Comprobar el proyecto de tests del punto 1.4

En `tests/AceriaApp.Organizacion.Tests`, localiza `AceriaApp.Organizacion.Tests.csproj`: es un proyecto de tests que ya viene incluido, no debes ejecutar otra plantilla xUnit encima. Ejecuta `dotnet sln AceriaApp.sln list` y localiza `AceriaApp.Organizacion.Tests` junto a los proyectos anteriores.

Abre el `.csproj` y comprueba `net8.0`, xUnit, FluentAssertions y la referencia de proyecto a la API monolítica. Ejecuta `dotnet restore AceriaApp.sln`; las clases de tests comentadas se activarán en los pasos 14 y 15.

El punto 1.4 añade un proyecto de tests específico para las clases de organización. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar los proyectos de tests del punto 1.3: viola la regla de no modificar archivos del punto anterior.
- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.Organizacion.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.Organizacion.Tests.

Paso 12 — Comprobar la referencia del proyecto de tests a la API monolítica

Abre `tests/AceriaApp.Organizacion.Tests/AceriaApp.Organizacion.Tests.csproj` y localiza el `ProjectReference` hacia `AceriaApp.Monolito.Api.csproj`. Comprueba que la referencia se dirige desde tests hacia API y que su ruta relativa existe físicamente.

Ejecuta `dotnet sln AceriaApp.sln list` y confirma que los proyectos de la etapa están incluidos en la misma solución. No agregues la referencia dos veces; restaura con `dotnet restore AceriaApp.sln` si Visual Studio no resuelve el proyecto.

La referencia permite que el proyecto de tests acceda a las clases de Organizacion del proyecto monolítico.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.Organizacion.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 13 — Comprobar el paquete FluentAssertions al proyecto de tests

Abre `tests/AceriaApp.Organizacion.Tests/AceriaApp.Organizacion.Tests.csproj` y localiza la dependencia `FluentAssertions` existente; no la instales dos veces en este proyecto preparado. Ejecuta `dotnet list tests/AceriaApp.Organizacion.Tests/AceriaApp.Organizacion.Tests.csproj package` desde la raíz de la etapa y comprueba que el paquete aparece con su versión.

Revisa que los tests de `ValidadorConvencionNombres` y `CatalogoProyectos` usan `FluentAssertions` para expresar sus resultados esperados. Ejecuta `dotnet restore AceriaApp.sln` antes de activar los tests; si NuGet falla revisa la restauración y no cambies las aserciones.

FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 14 — Implementar y comprender ValidadorConvencionNombresTests.cs

El archivo preparado es `tests/AceriaApp.Organizacion.Tests/ValidadorConvencionNombresTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.Organizacion.Tests/ValidadorConvencionNombresTests.cs` en Visual Studio y localiza los delimitadores `INICIO PASO 1.4.14` y `FIN PASO 1.4.14`. Selecciona las líneas de C# entre ambos marcadores, pulsa **Ctrl+K, Ctrl+U** y deja comentados los fragmentos del reto opcional.

Lee cada escenario positivo y negativo y vincúlalo a las reglas de `ValidadorConvencionNombres`: comprueba prefijo, tres componentes y sufijos permitidos. Guarda y compila el proyecto `AceriaApp.Organizacion.Tests`; corrige clases incompletas o imports antes de modificar las expectativas de las pruebas.

```
using AceriaApp.Monolito.Api.Organizacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Organizacion.Tests;

public class ValidadorConvencionNombresTests
{
    private readonly ValidadorConvencionNombres _validador = new(
        "AceriaApp",
        new[] { "Api", "Domain", "Infrastructure", "Application", "Tests" });

    [Fact]
    public void Validar_NombreCorrecto_DevuelveValido()
    {
        var resultado = _validador.Validar("AceriaApp.Ordenes.Api");

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_SinPrefijo_DevuelveError()
    {
        var resultado = _validador.Validar("Ordenes.Api");

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no empieza por"));
    }

    [Fact]
    public void Validar_TipoNoPermitido_DevuelveError()
    {
        var resultado = _validador.Validar("AceriaApp.Ordenes.Controlador");

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no está permitido"));
    }

    [Fact]
    public void Validar_NumeroDePartesIncorrecto_DevuelveError()
    {
        var resultado = _validador.Validar("AceriaApp.Ordenes");

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("Prefijo.Componente.Tipo"));
    }
}
```

`ValidadorConvencionNombresTests` ejercita denominaciones correctas y errores concretos, como prefijo ausente, tipo no permitido o un número inesperado de componentes. Estas pruebas previenen regresiones de la convención y muestran la diferencia entre un rechazo sintáctico y un fallo del software.

los tests verifican que el validador acepta nombres correctos y rechaza nombres sin prefijo, con tipo no permitido y con número de partes incorrecto.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.Organizacion;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de nombre correcto: el test solo verifica los errores.

Verificación: el archivo ValidadorConvencionNombresTests.cs compila y contiene cuatro métodos de test.

Paso 15 — Implementar y comprender CatalogoProyectosTests.cs

El archivo preparado es `tests/AceriaApp.Organizacion.Tests/CatalogoProyectosTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.Organizacion.Tests/CatalogoProyectosTests.cs` y busca `INICIO PASO 1.4.15` y el fin del mismo bloque. Descomenta solo las declaraciones internas con **Ctrl+K, Ctrl+U**, conservando comentados los bloques de reto y evitando duplicar la clase de tests.

Analiza qué datos del escenario producen los resultados de `ProyectosValidos y ProyectosInvalidos` y por qué el catálogo usa el validador o el indicador creado en pasos anteriores. Guarda y compila el proyecto de tests, comprobando que no requiere modificar fuentes del punto anterior.

```
using AceriaApp.Monolito.Api.Organizacion;
using FluentAssertions;
using Xunit;

namespace AceriaApp.Organizacion.Tests;

public class CatalogoProyectosTests
{
    [Fact]
    public void ProyectosValidos_FiltraCorrectamente()
    {
        var nombres = new List<string>
        {
            "AceriaApp.Ordenes.Api",
            "AceriaApp.Ordenes.Domain",
            "AceriaApp.Ordenes",
            "AceriaApp.Ordenes.Controlador"
        };
        var catalogo = new CatalogoProyectos(nombres);
        var validador = new ValidadorConvencionNombres(
            "AceriaApp",
            new[] { "Api", "Domain", "Infrastructure", "Application", "Tests" });

        var validos = catalogo.ProyectosValidos(validador).ToList();
        var invalidos = catalogo.ProyectosInvalidos(validador).ToList();

        validos.Should().HaveCount(2);
        validos.Should().Contain("AceriaApp.Ordenes.Api");
        validos.Should().Contain("AceriaApp.Ordenes.Domain");
        invalidos.Should().HaveCount(2);
        invalidos.Should().Contain("AceriaApp.Ordenes");
        invalidos.Should().Contain("AceriaApp.Ordenes.Controlador");
    }
}
```

`CatalogoProyectosTests` demuestra que el filtro conserva únicamente las denominaciones admitidas por el validador. Se trabaja con datos explícitos y sin desplegar servicios. El resultado debe interpretarse como la clasificación de un catálogo, no como la corrección de la estructura física de la solución.

El test verifica que el catálogo filtra correctamente los nombres válidos y los inválidos según el validador de convenciones.

Errores comunes:

- Olvidar using System.Collections.Generic;: List no se resuelve.
- Olvidar using System.Linq;: ToList() no se resuelve.
- No comprobar los nombres concretos: el test podría pasar por casualidad.

Verificación: el archivo CatalogoProyectosTests.cs compila y contiene un método de test.

Paso 16 — Compilar la solución completa

En la terminal de `1.4-organizacion-solucion`, donde está `AceriaApp.sln`, ejecuta `dotnet build AceriaApp.sln -c Release`. Comprueba que compilan los 6 proyectos acumulados y que el resumen muestra **0 errores**.

Ante un error, identifica primero archivo y línea; localiza el marcador `INICIO PASO` responsable antes de intentar crear una clase nueva. Conserva la salida de compilación y verifica que los proyectos heredados no han cambiado respecto a `1.3-principios-diseno`.

compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.4

Abre el **Explorador de pruebas** de Visual Studio o una terminal en la carpeta de la solución 1.4. Ejecuta `dotnet test tests/AceriaApp.Organizacion.Tests/AceriaApp.Organizacion.Tests.csproj -c Release` para comprobar únicamente los tests nuevos de la etapa.

Verifica que pasan **5 pruebas** de `AceriaApp.Organizacion.Tests` con 0 fallos; relaciona cada caso con `ValidadorConvencionNombres` o `CatalogoProyectos`. Si falla algún test, consulta el primer mensaje de aserción y examina los bloques activados en los pasos 14 y 15 sin cambiar valores esperados a ciegas.

ejecutar los tests del punto 1.4 verifica que las clases de organización funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 5, Skipped: 0, Total: 5.

Paso 18 — Ejecutar todos los tests de la solución

En la carpeta de `AceriaApp.sln` ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después del build Release del paso 16. Comprueba que pasan **18 pruebas acumuladas**: 13 heredadas más 5 de la etapa 1.4.

Si no se encuentran los ensamblados, vuelve a compilar en Release y repite el comando sin mezclar resultados Debug y Release. Guarda el resumen: los tests heredados deben seguir pasando, pues una nueva etapa no puede romper las capacidades previas de AceriaApp.

ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1, 1.2, 1.3 y 1.4 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 18, Skipped: 0, Total: 18.

Paso 19 — Revisar el documento preparado: README.md

Abre el archivo `README.md` de la carpeta `1.4-organizacion-solucion`, no el del punto `1.3-principios-diseno`. Localiza en el documento el objetivo de la etapa y los archivos `ConvencionNombres`, `ValidadorConvencionNombres`, `CatalogoProyectos` y `AceriaApp.Organizacion.Tests`.

Comprueba que las rutas de ejecución y tests mencionadas corresponden a los proyectos físicos de esta etapa. Anota qué información nueva aporta este README respecto al anterior; no lo sobrescribas con la versión heredada.

```
# Punto 1.4 — Organización de una solución basada en microservicios

## Objetivo
Comprender cómo se organiza una solución basada en microservicios y añadir al proyecto las clases de
validación de convenciones de nombres.

## Estado del proyecto
- Solución `AceriaApp.sln` con cinco proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests`,
`AceriaApp.Metricas.Tests`, `AceriaApp.Diseno.Tests` y `AceriaApp.Organizacion.Tests`.
- Documento `docs/organizacion-solucion.md` con las tres decisiones de organización y las convenciones.
- Clases `ConvencionNombres`, `ValidadorConvencionNombres` y `CatalogoProyectos` en `src/
AceriaApp.Monolito.Api/Organizacion/`.
- Cinco tests nuevos en `tests/AceriaApp.Organizacion.Tests`.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/organizacion-solucion.md`.
- Carpeta `Organizacion` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.Organizacion.Tests` con cinco tests nuevos.
- No se ha modificado ningún archivo del punto 1.3.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [ADR-002](docs/ADR-002-metricas-acoplamiento.md) (heredado del punto 1.2).
- [ADR-003](docs/ADR-003-principios-diseno.md) (heredado del punto 1.3).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md) (heredado del punto 1.2).
```

```
- [Principios de diseño](docs/principios-diseno.md) (heredado del punto 1.3).
- [Organización de la solución](docs/organizacion-solucion.md).
```

el README del punto 1.4 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.3.

Errores comunes:

- Modificar el README del punto 1.3: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 1.3: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.4 con las cinco secciones.

Paso 20 — Revisar el documento preparado: ADR-004-organizacion-solucion.md

Abre `docs/ADR-004-organizacion-solucion.md` dentro de `1.4-organizacion-solucion` en el editor Markdown; el archivo se entrega preparado. Lee Estado, Contexto, Decisión, Alternativas y Consecuencias, vinculando la decisión con organización de soluciones y nomenclatura.

Identifica un resultado de los tests de 1.4 que apoye la decisión y una limitación que el ejemplo no demuestra. Comprueba que el ADR del punto anterior permanece presente e idéntico: cada decisión arquitectónica se documenta por separado.

```
# ADR-004: Organización de la solución y convenciones de nombres

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha adoptado un monolito modular, métricas de acoplamiento y principios de diseño. Es necesario definir la estructura de repositorios, la estructura de carpetas y las convenciones de nombres para preparar la evolución hacia microservicios.

## Decisión
Se adopta un monorepo con una carpeta por módulo y una carpeta por punto. Dentro de cada punto se separan `src/`, `tests/` y `docs/`. Los proyectos siguen el patrón `AceriaApp.Componente.Tipo`. Los espacios de nombres coinciden con la estructura de carpetas. Los commits siguen el patrón `Punto N.M: descripción`.

## Alternativas consideradas
- Polyrepo por servicio: descartado por el tamaño del equipo y la fase de aprendizaje.
- Estructura plana sin separación de carpetas: descartado por la pérdida de previsibilidad.

## Consecuencias
- Positivas: previsibilidad, trazabilidad, preparación para la extracción de servicios.
- Negativas: el monorepo requiere disciplina para mantener la separación.
- Mitigación: validación automática de convenciones con la clase `ValidadorConvencionNombres`.
```

el ADR-004 documenta la decisión de organización y convenciones. Es un artefacto nuevo del punto 1.4 que no modifica los ADR existentes.

Errores comunes:

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-004: rompe la convención.

Verificación: el archivo docs\ADR-004-organizacion-solucion.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.3

En tu copia del repositorio localiza las carpetas `docs` de `1.3-principios-diseno` y `1.4-organizacion-solucion`. Selecciona un fichero **heredado** presente en ambas carpetas y compara su contenido con `fc /L` en Windows o un hash SHA-256.

Los documentos heredados deben ser idénticos; el `README.md` de 1.4 puede ser diferente porque explica la etapa actual. Registra el resultado y no modifiques los originales de `1.3-principios-diseno` para forzar igualdad.

La comparación con fc verifica que los documentos del punto 1.3 no se han modificado.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los dos documentos del punto 1.3 no muestran diferencias.

Paso 22 — Verificar la estructura del punto 1.4

Desde `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO` ejecuta `tree /F 1.4-organizacion-solucion` en una terminal Windows. Comprueba en `src/AceriaApp.Monolito.Api/Organizacion` los tres tipos `ConvencionNombres.cs`, `ValidadorConvencionNombres.cs` y `CatalogoProyectos.cs`.

En `tests/AceriaApp.Organizacion.Tests` localiza las pruebas del validador y catálogo; en `docs` localiza `organizacion-solucion.md`, `ADR-004-organizacion-solucion.md` y los documentos heredados. Distingue los archivos fuente de las carpetas temporales `bin` y `obj`, que no deben incorporarse como entregables.

La verificación estructural confirma que el punto 1.4 añade archivos nuevos sin modificar los existentes.

Errores comunes:

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 1.3: viola la regla de no modificar.
- Omitir la carpeta Organizacion: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.3.

Paso 23 — Revisar el documento preparado: .gitignore

Abre el `.gitignore` de la carpeta `1.4-organizacion-solucion` en un editor que muestre ficheros ocultos. Comprueba que conserva las exclusiones `bin/`, `obj/`, `.vs/`, resultados y archivos de usuario sin ignorar el código fuente.

Ejecuta `git status --short` en tu copia personal tras compilar y confirma que no aparecen binarios temporales. No sustituyas el fichero ya incluido por una plantilla ajena; verifica que conserva la política del punto anterior.

el .gitignore se hereda del punto 1.3 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

- Modificar el .gitignore copiado del punto 1.3: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.

- Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

Paso 24 — Hacer el commit del punto 1.4 y verificar el estado final

Desde tu copia personal (no la rama del profesor) ejecuta `git status` para identificar los cambios de `1.4-organizacion-solucion`. Revisa `git diff --stat` y prepara con `git add` solo archivos de esta etapa, sin incluir carpetas compiladas ni secretos.

Realiza el commit del punto 1.4 con un mensaje descriptivo de organización de soluciones y nomenclatura; no sobrescribas los commits de etapas previas. Comprueba `git log -1 --oneline`, `git status` y conserva el resultado de los 18 tests acumulados como evidencia final.

El commit del punto 1.4 registra el estado acumulado del proyecto. Los cuatro commits consecutivos permiten comparar los puntos 1.1, 1.2, 1.3 y 1.4 con git diff.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra cuatro commits: Punto 1.1, Punto 1.2, Punto 1.3 y Punto 1.4. git status muestra nothing to commit, working tree clean.

Nota de trazabilidad: en una descarga del repositorio puede no haber cambios nuevos que confirmar. Antes de ejecutar `git commit`, usar `git status` y confirmar únicamente los archivos efectivamente modificados durante el recorrido. No ejecutar `git init` dentro de cada punto: la raíz del repositorio ya contiene el historial.

Reto de ampliación: solución guiada

Problema a resolver: añadir un método Resumen a CatalogoProyectos que devuelva el número de proyectos válidos, el número de inválidos y el porcentaje de validez, y un test que verifique el cálculo cuando hay tres válidos y dos inválidos.

Etapa 1 — En CatalogoProyectos.cs, añadir el siguiente record al final del archivo:

```
public record ResumenProyectos(int Validos, int Invalidos, double PorcentajeValidez);
```

Etapa 2 — Explicar línea por línea:

- **Línea 1:** `public record ResumenProyectos(` — declara un contrato inmutable.
- **Línea 2:** `int Validos,` — número de proyectos válidos.
- **Línea 3:** `int Invalidos,` — número de proyectos inválidos.
- **Línea 4:** `double PorcentajeValidez);` — porcentaje de validez.

Etapa 3 — En CatalogoProyectos.cs, añadir el siguiente método dentro de la clase:

```
public ResumenProyectos Resumen(ValidadorConvencionNombres validador)
{
    var validos = _nombres.Count(n => validador.Validar(n).EsValido);
    var invalidos = _nombres.Count - validos;
    var total = _nombres.Count;
    var porcentaje = total == 0 ? 0 : (double)validos / total * 100;
```

```
    return new ResumenProyectos(validos, invalidos, porcentaje);
}
```

Etapa 4 — Explicar línea por línea:

- **Línea 1:** `public ResumenProyectos Resumen(ValidadorConvencionNombres validador)` — declara el método.
- **Línea 2:** `{` — abre el cuerpo.
- **Línea 3:** `var validos = _nombres.Count(n => validador.Validar(n).EsValido);` — cuenta los válidos.
- **Línea 4:** `var invalidos = _nombres.Count - validos;` — calcula los inválidos por diferencia.
- **Línea 5:** `var total = _nombres.Count;` — obtiene el total.
- **Línea 6:** `var porcentaje = total == 0 ? 0 : (double)validos / total * 100;` — calcula el porcentaje con protección contra división por cero.
- **Línea 7:** `return new ResumenProyectos(validos, invalidos, porcentaje);` — devuelve el resumen.
- **Línea 8:** `}` — cierra el cuerpo.

Etapa 5 — Guardar el archivo con Ctrl+S.

Etapa 6 — En CatalogoProyectosTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresValidosYDosInvalidos_CalculaPorcentaje()
{
    var nombres = new List<string>
    {
        "AceriaApp.Ordenes.Api",
        "AceriaApp.Ordenes.Domain",
        "AceriaApp.Inventario.Api",
        "AceriaApp.Ordenes",
        "AceriaApp.Inventario.Controlador"
    };
    var catalogo = new CatalogoProyectos(nombres);
    var validador = new ValidadorConvencionNombres(
        "AceriaApp",
        new[] { "Api", "Domain", "Infrastructure", "Application", "Tests" });

    var resumen = catalogo.Resumen(validador);

    resumen.Validos.Should().Be(3);
    resumen.Invalidos.Should().Be(2);
    resumen.PorcentajeValidez.Should().Be(60.0);
}
```

Etapa 7 — Explicar línea por línea:

- **Línea 1:** `[Fact]` — marca el método como test de xUnit.
- **Línea 2:** `public void Resumen_ConTresValidosYDosInvalidos_CalculaPorcentaje()` — declara el test.
- **Línea 3:** `{` — abre el cuerpo.
- **Línea 4:** `var nombres = new List<string>` — crea la lista.
- **Línea 5:** `{` — abre el inicializador.
- **Línea 6:** `"AceriaApp.Ordenes.Api",` — primer nombre válido.
- **Línea 7:** `"AceriaApp.Ordenes.Domain",` — segundo nombre válido.
- **Línea 8:** `"AceriaApp.Inventario.Api",` — tercer nombre válido.
- **Línea 9:** `"AceriaApp.Ordenes",` — primer nombre inválido por falta de tipo.
- **Línea 10:** `"AceriaApp.Inventario.Controlador"` — segundo nombre inválido por tipo no permitido.
- **Línea 11:** `};` — cierra el inicializador.
- **Línea 12:** `var catalogo = new CatalogoProyectos(nombres);` — instancia el catálogo.
- **Línea 13:** `var validador = new ValidadorConvencionNombres(...);` — instancia el validador.

- **Línea 14:** `var resumen = catalogo.Resumen(validador);` — invoca el método.
- **Línea 15:** `resumen.Validos.Should().Be(3);` — verifica los válidos.
- **Línea 16:** `resumen.Invalidos.Should().Be(2);` — verifica los inválidos.
- **Línea 17:** `resumen.PorcentajeValidez.Should().Be(60.0);` — verifica el porcentaje.
- **Línea 18:** `}` — cierra el cuerpo.

Etapa 8 — Ejecutar dotnet test tests\AceriaApp.Organizacion.Tests\AceriaApp.Organizacion.Tests.csproj y verificar que pasan los seis tests.

Etapa 9 — Verificar que el método Resumen no modifica ningún archivo del punto 1.3.

Etapa 10 — Hacer commit con git commit -m "Reto 1.4: método Resumen en CatalogoProyectos con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 6.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-1-introduccion-microservicios/1.4-organizacion-solucion/ con la solución AceriaApp.sln que contiene seis proyectos.
- El documento docs/organizacion-solucion.md con las tres decisiones y las convenciones.
- El documento docs/ADR-004-organizacion-solucion.md con la decisión sobre organización.
- Las clases ConvencionNombres, ValidadorConvencionNombres y CatalogoProyectos en src/AceriaApp.Monolito.Api/Organizacion/.
- El proyecto tests/AceriaApp.Organizacion.Tests con cinco tests (seis tras el reto).
- Los tests de los puntos 1.1, 1.2, 1.3 y 1.4 pasando conjuntamente con dotnet test AceriaApp.sln.
- Los ADR-001, ADR-002 y ADR-003 del punto 1.3 sin modificar.
- Cuatro commits en Git: del punto 1.1 al 1.4.

Conclusión y enlace al siguiente punto

El punto 1.4 ha definido la organización de la solución con un monorepo, una separación clara entre src/, tests/ y docs/, y convenciones de nombres de proyectos, espacios de nombres y commits. El proyecto cuenta ahora con un validador automático que audita las convenciones y prepara la extracción de microservicios. Esta base permite avanzar hacia el punto 1.5, donde se estudiarán las buenas prácticas de desarrollo de microservicios: idempotencia, versionado, contratos, documentación y gestión de errores.

Fin del punto 1.4.

Punto 1.5 — Buenas prácticas

Convertir las buenas prácticas en decisiones verificables

Cerramos el módulo reuniendo criterios de mantenibilidad que han ido apareciendo en los puntos anteriores: claridad de responsabilidades, pruebas, documentación de decisiones y límites comprensibles. Para discutirlos construiremos un pequeño modelo de prácticas y un validador explícito. Las reglas son ejemplos de análisis y no una puntuación universal de calidad.

La etapa 1.5 conserva la solución anterior y añade `Practica`, `ValidadorBuenasPracticas` y `CatalogoPracticas`. Al implementar estas clases estudiaremos qué información reciben, por qué aceptan o rechazan casos concretos y qué aportan sus tests. Un catálogo que marca una práctica como aplicada no demuestra por sí solo que la arquitectura la utilice: es necesario distinguir la descripción de la evidencia.

Al finalizar, podremos ejecutar las pruebas de todos los puntos de M01, revisar las decisiones documentadas y explicar cuáles son los comportamientos reales de la API. Esta base prepara el desarrollo REST del módulo 2 sin inventar capacidades distribuidas que todavía no existen.

Desarrollo práctico

Ejercicio: Añadir al punto 1.4 un documento de buenas prácticas en docs/, una clase de validación de buenas prácticas en el proyecto monolítico, un test unitario que verifique la validación y el resumen del estado del proyecto al final del módulo 1. El punto 1.5 parte del contenido del punto 1.4 sin modificar ningún archivo existente: solo añade archivos nuevos.

Ubicación de trabajo: utiliza la carpeta real de tu copia del repositorio en Windows; `C:\cursos\` solo representa una ubicación posible.AceriaApp\modulo-1-introduccion-microservicios\

Paso 1 — Localizar el proyecto independiente del punto 1.5

Localiza `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO/1.5-buenas-practicas` en tu copia del repositorio; no sobrescribas la carpeta del punto anterior `1.4-organizacion-solucion`. Comprueba `AceriaApp.sln`, `src`, `tests`, `docs` y `README.md`; las clases nuevas están preparadas, pero parte del código permanece comentado.

Abre la terminal en esa misma carpeta y ejecuta `dotnet --list-sdks` para comprobar la disponibilidad del SDK .NET 8. Registra la ruta de trabajo y diferencia el proyecto distribuido de uno vacío que no debes crear otra vez.

Cada punto constituye una solución autónoma. Se evita copiar la solución anterior sobre archivos nuevos ya preparados y se garantiza que los bloques de código comentados correspondan exactamente a este punto.

Errores comunes: abrir la solución del punto anterior; copiar archivos que sobrescriban los bloques preparados; confundir la carpeta del repositorio con la carpeta del proyecto.

Verificación: `AceriaApp.sln` existe y la terminal está situada dentro de la carpeta de este punto.

Paso 2 — Abrir la solución y comprobar los proyectos heredados

Abre en Visual Studio el archivo `AceriaApp.sln` de `1.5-buenas-practicas` desde **Archivo > Abrir > Proyecto o solución**. Ejecuta `dotnet sln AceriaApp.sln list` y verifica los 7 proyectos acumulados, incluido `AceriaApp.BuenasPracticas.Tests`.

Expande la carpeta `src/AceriaApp.Monolito.Api/BuenasPracticas` y observa que los tipos de esta etapa tienen marcadores `INICIO PASO` sin activar. Ejecuta `dotnet build AceriaApp.sln` y guarda la salida de la compilación inicial antes de editar ninguna clase.

se identifica el estado heredado ejecutable y se evita confundir una clase comentada con un proyecto todavía no creado.

Errores comunes: abrir la carpeta de otro punto; contar únicamente los proyectos con pruebas activas; descomentar todos los bloques a la vez.

Verificación: la solución abre y compila antes de introducir los cambios del punto.

Paso 3 — Reconocer documentación heredada y nuevas fuentes de BuenasPracticas

Abre `docs` en la carpeta `1.5-buenas-practicas` y comprueba que conserva los documentos del punto anterior `1.4-organizacion-solucion`. Identifica los archivos nuevos `docs/buenas-practicas.md` y `docs/ADR-005-buenas-practicas.md` que vienen incluidos como material preparado.

Contrasta uno de los ADR heredados con el fichero homónimo de la etapa anterior, sin reescribirlo. Anota qué archivos son heredados y cuáles nuevos; no exijas una carpeta `docs` vacía antes de comenzar, porque los documentos ya están entregados.

El punto 1.5 parte del contenido del punto 1.4, que ya incluye la carpeta docs con siete documentos. No se modifica ningún documento existente.

Errores comunes:

- Modificar alguno de los ADR o documentos existentes: viola la regla de no modificar archivos del punto anterior.
- Eliminar algún documento existente: pierde la trazabilidad del punto 1.4.
- Crear la carpeta docs si ya existe: mkdir falla.

Verificación: la carpeta docs contiene los siete documentos del punto 1.4 sin modificaciones.

Paso 4 — Revisar el documento preparado: buenas-practicas.md

Abre `docs/buenas-practicas.md` en Visual Studio con la vista Markdown, sin crear otro archivo de igual nombre. Localiza los apartados sobre buenas prácticas declaradas y su aplicación y vincula dos de sus decisiones con AceriaApp.

Elabora una tabla breve de ventajas, riesgos y criterios técnicos con un ejemplo aplicable al dominio de órdenes de producción. Contrasta la estructura con el texto incluido en esta práctica y guarda tus conclusiones en un archivo personal separado.

```
# Buenas prácticas — Punto 1.5

## Categorías

1. Contrato.
2. Operación.
3. Comunicación.
4. Evolución.

## Buenas prácticas de contrato

- Exponer DTOs específicos en lugar de entidades de dominio.
- Versionar el contrato público en la URL con el prefijo `/api/v1/`.

## Buenas prácticas de operación
```

```
- Idempotencia mediante cabecera `Idempotency-Key`.
- Gestión explícita de errores con códigos HTTP y problem details.

## Buenas prácticas de comunicación

- Timeouts en todas las llamadas externas.
- Reintentos con retroceso exponencial.
- Circuit breaker para evitar saturación.

## Buenas prácticas de evolución

- Mantener la compatibilidad hacia atrás de los contratos.
- Verificar la compatibilidad con tests automáticos.
- Mantener documentación viva con OpenAPI.

## Aplicación al proyecto del curso

| Práctica | Aplicación |
|----------|------------|
| DTOs separados | `OrdenDto` en lugar de `Orden` |
| Versionado en URL | `/api/v1/ordenes` |
| Idempotencia | Cabecera `Idempotency-Key` |
| Errores explícitos | Problem details |
| Timeouts y reintentos | Polly |
| Circuit breaker | Polly |
| Contratos compatibles | Tests de compatibilidad |
| Documentación viva | OpenAPI |

## Cierre del módulo 1

El proyecto AceriaApp dispone, al terminar el módulo 1, de los siguientes componentes realmente
implementados:

- Monolito modular de referencia con API Minimal.
- Persistencia con EF Core 8.0.0 y SQL Server Express.
- Documentación OpenAPI con Swagger.
- Métricas de acoplamiento.
- Validador de límites de servicio.
- Validador de convenciones de nombres.
- Validador de declaraciones de buenas prácticas (no verifica automáticamente DTOs, idempotencia HTTP ni
políticas Polly).
- Tests unitarios acumulados en cuatro proyectos de tests.
```

El documento de buenas prácticas es el artefacto de docs específico del punto 1.5. Incluye el cierre del módulo 1 con el resumen del estado del proyecto. No modifica los siete documentos existentes del punto 1.4.

Errores comunes:

- Nombrar el archivo con otro nombre: rompe la trazabilidad del punto.
- Modificar los ADR existentes: viola la regla de no modificar archivos del punto anterior.
- Guardar el archivo en la raíz del punto en lugar de docs: rompe la estructura del repositorio.

Verificación: el archivo docs\buenas-practicas.md existe con las siete secciones.

Paso 5 — Localizar la carpeta de código ya preparada

En el Explorador de soluciones abre `src/AceriaApp.Monolito.Api/BuenasPracticas` y localiza `Practica.cs`, `ValidadorBuenasPracticas.cs` y `CatalogoPracticas.cs`. En cada archivo identifica el marcador `INICIO PASO 1.5.6`, `1.5.7` o `1.5.9`; comprueba que las declaraciones nuevas están comentadas.

Confirma que las clases heredadas de Dominio e Infraestructura siguen activas y no se deben modificar. Planifica la secuencia de activación: primero `Practica`, después `ValidadorBuenasPracticas` y finalmente `CatalogoPracticas`.

La carpeta ya existe en el proyecto preparado; la progresión didáctica se logra activando el contenido, no creando archivos duplicados.

Errores comunes: crear un archivo con el mismo nombre; descomentar código de otros pasos; modificar archivos heredados.

Verificación: la carpeta `BuenasPracticas` y los archivos del punto están presentes, inicialmente comentados.

Paso 6 — Implementar y comprender Practica.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/BuenasPracticas/Practica.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/BuenasPracticas/Practica.cs` y localiza los delimitadores `INICIO PASO 1.5.6` y `FIN PASO 1.5.6`. Selecciona solo las líneas C# interiores, pulsa **Ctrl+K, Ctrl+U** una vez y conserva los marcadores como comentarios.

Comprueba que aparecen las propiedades Codigo, Categoria, Descripcion y EstaAplicada y explica qué describe cada una. Guarda y compila `src/AceriaApp.Monolito.Api/AceriaApp.Monolito.Api.csproj` sin crear una segunda declaración de `Practica`.

```
namespace AceriaApp.Monolito.Api.BuenasPracticas;

public class Practica
{
    public string Codigo { get; init; } = string.Empty;
    public string Categoria { get; init; } = string.Empty;
    public string Descripcion { get; init; } = string.Empty;
    public bool EstaAplicada { get; init; }
}
```

`Practica` representa los datos que necesitamos para registrar una práctica y su estado. Su valor consiste en mantener homogénea la información; no observa automáticamente el código del repositorio. Una afirmación de cumplimiento necesitaría evidencias adicionales fuera de este modelo.

La clase Practica modela una buena práctica con su código, categoría, descripción y estado de aplicación. Es la unidad de información que maneja el validador.

Errores comunes:

- Olvidar el namespace: la clase no se resuelve desde otros archivos.
- No inicializar las propiedades de tipo string con string.Empty: el compilador emite advertencias de nulabilidad.
- Usar set en lugar de init: permite modificar la instancia después de crearla.

Verificación: la clase Practica compila sin errores y expone cuatro propiedades.

Paso 7 — Implementar y comprender ValidadorBuenasPracticas.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/BuenasPracticas/ValidadorBuenasPracticas.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/BuenasPracticas/ValidadorBuenasPracticas.cs` y localiza el bloque delimitado para el paso 1.5.7. Descomenta únicamente su código con **Ctrl+K, Ctrl+U** y deja inactivos los miembros marcados como `RET0`.

Sigue la lógica de `Validar` para categorías válidas, colección vacía y prácticas pendientes; identifica qué condición genera cada error. Guarda y compila; comprueba que la clase devuelve su tipo de resultado de validación y no depende de un reto opcional.

```
namespace AceriaApp.Monolito.Api.BuenasPracticas;

public class ValidadorBuenasPracticas
{
    private readonly IReadOnlyList<string> _categoriasPermitidas;

    public ValidadorBuenasPracticas(IReadOnlyList<string> categoriasPermitidas)
    {
        _categoriasPermitidas = categoriasPermitidas;
    }

    public ResultadoPracticas Validar(IEnumerable<Practica> practicas)
    {
        var errores = new List<string>();
        var lista = practicas.ToList();

        if (lista.Count == 0)
        {
            errores.Add("No se ha declarado ninguna práctica.");
        }

        foreach (var practica in lista)
        {
            if (!_categoriasPermitidas.Contains(practica.Categoria))
            {
                errores.Add($"Categoría no permitida: '{practica.Categoria}'.");
            }

            if (!practica.EstaAplicada)
            {
                errores.Add($"Práctica no aplicada: '{practica.Codigo}'.");
            }
        }

        return new ResultadoPracticas(errores.Count == 0, errores);
    }
}

public record ResultadoPracticas(bool EsValido, IReadOnlyList<string> Errores);
```

`ValidadorBuenasPracticas` revisa las categorías y condiciones acordadas, añadiendo errores para los casos rechazados. Es preferible estudiar cada condición junto a su test que convertir el resultado en una puntuación general de calidad. El método responde a un contrato concreto, no certifica el proyecto completo.

La clase ValidadorBuenasPracticas comprueba que todas las prácticas declaradas pertenecen a categorías permitidas y están aplicadas. Es el mecanismo de auditoría de buenas prácticas.

Errores comunes:

- Olvidar el using System.Collections.Generic;: IReadOnlyList y List no se resuelven.
- Olvidar el using System.Linq;: ToList() y Contains no se resuelven.
- No comprobar el caso de lista vacía: el validador devuelve válido sin prácticas.

Verificación: la clase ValidadorBuenasPracticas compila sin errores y expone un constructor, un método y un record.

Interpretación del resultado: El validador inspecciona declaraciones. Una práctica marcada como aplicada puede no estar implementada; para certificarla hace falta una prueba funcional del endpoint.

Paso 8 — Añadir los using necesarios en ValidadorBuenasPracticas

En `src/AceriaApp.Monolito.Api/BuenasPracticas/ValidadorBuenasPracticas.cs`, examina las directivas `using` que quedaron activas al completar el paso 1.5.7. Comprueba que ya aparecen los imports necesarios (System.Collections.Generic, System.Linq); no agregues una segunda copia si están presentes.

Relaciona cada import con las colecciones y los métodos empleados en `Validar`. Guarda y compila; si hay errores de tipos, comprueba primero el bloque de activación antes de cambiar las dependencias.

los tipos IReadOnlyList y List pertenecen a System.Collections.Generic. Los métodos ToList y Contains pertenecen a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir los usings explícitos.

Errores comunes:

- Añadir solo System.Collections.Generic y olvidar System.Linq: ToList y Contains no se resuelven.
- Añadir los usings después del namespace: el compilador no los reconoce.
- Olvidar los usings y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: ValidadorBuenasPracticas.cs compila sin errores y no muestra subrayados rojos.

Paso 9 — Implementar y comprender CatalogoPracticas.cs

El archivo preparado es `src/AceriaApp.Monolito.Api/BuenasPracticas/CatalogoPracticas.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `src/AceriaApp.Monolito.Api/BuenasPracticas/CatalogoPracticas.cs` y busca los marcadores del paso 1.5.9. Descomenta solo las líneas interiores con **Ctrl+K, Ctrl+U**, conservando inactivo el bloque de `RETO`.

Comprueba que el catálogo implementa `PracticasAplicadas y PracticasPendientes` y usa los tipos de los pasos anteriores. Guarda y compila, revisando que los métodos no crean una segunda clase con el nombre `CatalogoPracticas`.

```
namespace AceriaApp.Monolito.Api.BuenasPracticas;

public class CatalogoPracticas
{
    private readonly List<Practica> _practicas;

    public CatalogoPracticas(List<Practica> practicas)
    {
        _practicas = practicas;
    }

    public IEnumerable<Practica> PracticasAplicadas()
    {
        return _practicas.Where(p => p.EstaAplicada);
    }

    public IEnumerable<Practica> PracticasPendientes()
    {
        return _practicas.Where(p => !p.EstaAplicada);
    }
}
```

`CatalogoPracticas` agrupa entradas que cumplen los criterios previstos y permite distinguir aplicadas de pendientes. La clasificación no cambia archivos de código, despliegues ni configuraciones. Su utilidad está en la consulta coherente de los registros creados.

La clase CatalogoPracticas agrupa las prácticas del proyecto y permite filtrarlas por estado de aplicación.

Errores comunes:

- No inicializar _practicas en el constructor: la clase lanza NullReferenceException.
- Olvidar using System.Linq;: Where no se resuelve.
- Devolver List en lugar de IEnumerable: expone la implementación interna.

Verificación: la clase CatalogoPracticas compila sin errores y expone un constructor y dos métodos.

Interpretación del resultado: La separación entre aplicadas y pendientes depende exclusivamente del estado de los objetos `Practica` y puede verificarse con las pruebas del catálogo.

Paso 10 — Comprobar el using necesario en CatalogoPracticas

Abre el archivo de **código fuente** `src/AceriaApp.Monolito.Api/BuenasPracticas/CatalogoPracticas.cs`; no abras por error el `.csproj` de tests para resolver un `using`. Comprueba las directivas existentes (System.Collections.Generic, System.Linq) que llegaron con la activación del bloque 1.5.9.

No dupliques imports; localiza las operaciones de filtrado del catálogo que utilizan LINQ o colecciones. Guarda y compila el proyecto API. El resultado de este paso es una importación correcta en `CatalogoPracticas.cs`, no un cambio NuGet.

El método Where pertenece a System.Linq. En proyectos con ImplicitUsings desactivado, hay que añadir el using explícito.

Errores comunes:

- Añadir using System.Collections.Generic; en lugar de System.Linq: no resuelve Where.
- Añadir el using después del namespace: el compilador no lo reconoce.
- Olvidar el using y confiar en ImplicitUsings: el proyecto puede tenerlo desactivado.

Verificación: CatalogoPracticas.cs compila sin errores y no muestra subrayados rojos.

Paso 11 — Comprobar el proyecto de tests del punto 1.5

En `tests/AceriaApp.BuenasPracticas.Tests`, localiza `AceriaApp.BuenasPracticas.Tests.csproj`: es un proyecto de tests que ya viene incluido, no debes ejecutar otra plantilla xUnit encima. Ejecuta `dotnet sln AceriaApp.sln list` y localiza `AceriaApp.BuenasPracticas.Tests` junto a los proyectos anteriores.

Abre el `.csproj` y comprueba `net8.0`, xUnit, FluentAssertions y la referencia de proyecto a la API monolítica. Ejecuta `dotnet restore AceriaApp.sln`; las clases de tests comentadas se activarán en los pasos 14 y 15.

El punto 1.5 añade un proyecto de tests específico para las clases de buenas prácticas. Se crea como proyecto nuevo sin modificar los proyectos de tests existentes.

Errores comunes:

- Crear el proyecto dentro de src en lugar de tests: rompe la estructura.
- Modificar los proyectos de tests del punto 1.4: viola la regla de no modificar archivos del punto anterior.
- Usar dotnet new mstest: el curso usa xUnit.

Verificación: el proyecto AceriaApp.BuenasPracticas.Tests aparece en la solución y su carpeta existe en tests\AceriaApp.BuenasPracticas.Tests.

Paso 12 — Comprobar la referencia del proyecto de tests a la API monolítica

Abre `tests/AceriaApp.BuenasPracticas.Tests/AceriaApp.BuenasPracticas.Tests.csproj` y localiza el `ProjectReference` hacia `AceriaApp.Monolito.Api.csproj`. Comprueba que la referencia se dirige desde tests hacia API y que su ruta relativa existe físicamente.

Ejecuta `dotnet sln AceriaApp.sln list` y confirma que los proyectos de la etapa están incluidos en la misma solución. No agregues la referencia dos veces; restaura con `dotnet restore AceriaApp.sln` si Visual Studio no resuelve el proyecto.

La referencia permite que el proyecto de tests acceda a las clases de BuenasPracticas del proyecto monolítico.

Errores comunes:

- Añadir la referencia en dirección inversa: crea una dependencia circular.
- Olvidar el nombre completo del archivo .csproj: el comando falla.
- Ejecutar el comando desde la carpeta equivocada: la ruta relativa no se resuelve.

Verificación: el archivo AceriaApp.BuenasPracticas.Tests.csproj contiene un ProjectReference al proyecto monolítico.

Paso 13 — Comprobar el paquete FluentAssertions al proyecto de tests

Abre `tests/AceriaApp.BuenasPracticas.Tests/AceriaApp.BuenasPracticas.Tests.csproj` y localiza la dependencia `FluentAssertions` existente; no la instales dos veces en este proyecto preparado. Ejecuta `dotnet list tests/AceriaApp.BuenasPracticas.Tests/AceriaApp.BuenasPracticas.Tests.csproj package` desde la raíz de la etapa y comprueba que el paquete aparece con su versión.

Revisa que los tests de `ValidadorBuenasPracticas` y `CatalogoPracticas` usan `FluentAssertions` para expresar sus resultados esperados. Ejecuta `dotnet restore AceriaApp.sln` antes de activar los tests; si NuGet falla revisa la restauración y no cambies las aserciones.

FluentAssertions proporciona la sintaxis de aserción usada en el curso. Se añade al nuevo proyecto de tests sin modificar los proyectos de tests existentes.

Errores comunes:

- Instalar una versión preliminar: el curso usa la última estable.
- Olvidar el paquete: Should() no se resuelve en los tests.
- Añadir el paquete al proyecto equivocado: los tests no compilan.

Verificación: la salida de dotnet list package muestra FluentAssertions con una versión estable.

Paso 14 — Implementar y comprender ValidadorBuenasPracticasTests.cs

El archivo preparado es `tests/AceriaApp.BuenasPracticas.Tests/ValidadorBuenasPracticasTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.BuenasPracticas.Tests/ValidadorBuenasPracticasTests.cs` en Visual Studio y localiza los delimitadores `INICIO PASO 1.5.14` y `FIN PASO 1.5.14`. Selecciona las líneas de C# entre ambos marcadores, pulsa **Ctrl+K, Ctrl+U** y deja comentados los fragmentos del reto opcional.

Lee cada escenario positivo y negativo y vincúlalo a las reglas de `ValidadorBuenasPracticas`: rechaza categorías ajenas al catálogo, listas vacías y prácticas no aplicadas. Guarda y compila el proyecto `AceriaApp.BuenasPracticas.Tests`; corrige clases incompletas o imports antes de modificar las expectativas de las pruebas.

```
using AceriaApp.Monolito.Api.BuenasPracticas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.BuenasPracticas.Tests;

public class ValidadorBuenasPracticasTests
{
    private readonly ValidadorBuenasPracticas _validador = new(
        new[] { "Contrato", "Operación", "Comunicación", "Evolución" });

    [Fact]
    public void Validar_PracticasCompletas_DevuelveValido()
    {
        var practicas = new List<Practica>
        {
            new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = true },
            new() { Codigo = "O-01", Categoria = "Operación", EstaAplicada = true },
            new() { Codigo = "M-01", Categoria = "Comunicación", EstaAplicada = true },
            new() { Codigo = "E-01", Categoria = "Evolución", EstaAplicada = true }
        };

        var resultado = _validador.Validar(practicas);

        resultado.EsValido.Should().BeTrue();
        resultado.Errores.Should().BeEmpty();
    }

    [Fact]
    public void Validar_CategoriaNoPermitida_DevuelveError()
    {
        var practicas = new List<Practica>
        {
            new() { Codigo = "X-01", Categoria = "Inventada", EstaAplicada = true }
        };

        var resultado = _validador.Validar(practicas);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no permitida"));
    }

    [Fact]
    public void Validar_PracticaNoAplicada_DevuelveError()
    {
        var practicas = new List<Practica>
        {
            new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = false }
        };

        var resultado = _validador.Validar(practicas);

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("no aplicada"));
    }

    [Fact]
    public void Validar_SinPracticas_DevuelveError()
    {
        var resultado = _validador.Validar(new List<Practica>());

        resultado.EsValido.Should().BeFalse();
        resultado.Errores.Should().Contain(e => e.Contains("ninguna práctica"));
    }
}
```

`ValidadorBuenasPracticasTests` comprueba ejemplos completos y casos como categorías rechazadas o registros que no cumplen la regla. Cada resultado esperado revela qué condición se está probando. Las aserciones pertenecen al comportamiento del validador, no a una auditoría automática de todos los módulos.

los tests verifican que el validador acepta prácticas completas y rechaza prácticas con categoría no permitida, prácticas no aplicadas y listas vacías.

Errores comunes:

- Olvidar el using AceriaApp.Monolito.Api.BuenasPracticas;: las clases no se resuelven.
- Usar Assert.True en lugar de Should().BeTrue(): el curso usa FluentAssertions.
- No cubrir el caso de lista vacía: el test no verifica el mensaje de error.

Verificación: el archivo ValidadorBuenasPracticasTests.cs compila y contiene cuatro métodos de test.

Paso 15 — Implementar y comprender CatalogoPracticasTests.cs

El archivo preparado es `tests/AceriaApp.BuenasPracticas.Tests/CatalogoPracticasTests.cs`. El ejemplo contiene el código que debe quedar implementado. Puedes escribirlo en el esqueleto o descomentar únicamente el bloque delimitado por el marcador correspondiente, evitando duplicar declaraciones.

Abre `tests/AceriaApp.BuenasPracticas.Tests/CatalogoPracticasTests.cs` y busca `INICIO PASO 1.5.15` y el fin del mismo bloque. Descomenta solo las declaraciones internas con **Ctrl+K, Ctrl+U**, conservando comentados los bloques de reto y evitando duplicar la clase de tests.

Analiza qué datos del escenario producen los resultados de `PracticasAplicadas y PracticasPendientes` y por qué el catálogo usa el validador o el indicador creado en pasos anteriores. Guarda y compila el proyecto de tests, comprobando que no requiere modificar fuentes del punto anterior.

```
using AceriaApp.Monolito.Api.BuenasPracticas;
using FluentAssertions;
using Xunit;

namespace AceriaApp.BuenasPracticas.Tests;

public class CatalogoPracticasTests
{
    [Fact]
    public void PracticasAplicadas_FiltraCorrectamente()
    {
        var practicas = new List<Practica>
        {
            new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = true },
            new() { Codigo = "O-01", Categoria = "Operación", EstaAplicada = false },
            new() { Codigo = "M-01", Categoria = "Comunicación", EstaAplicada = true }
        };
        var catalogo = new CatalogoPracticas(practicas);

        var aplicadas = catalogo.PracticasAplicadas().ToList();
        var pendientes = catalogo.PracticasPendientes().ToList();

        aplicadas.Should().HaveCount(2);
        pendientes.Should().HaveCount(1);
        pendientes[0].Codigo.Should().Be("O-01");
    }
}
```

`CatalogoPracticasTests` asegura que la clasificación de la colección devuelve los elementos correctos. No prueba que se hayan aplicado prácticas en un sistema real: para eso sería necesario contrastar código, tests y comportamiento. Esta limitación forma parte de la interpretación del resultado.

El test verifica que el catálogo filtra correctamente las prácticas aplicadas y las pendientes.

Errores comunes:

- Olvidar using System.Collections.Generic;: List no se resuelve.
- Olvidar using System.Linq;: ToList() no se resuelve.
- No comprobar el código de la práctica pendiente: el test podría pasar por casualidad.

Verificación: el archivo CatalogoPracticasTests.cs compila y contiene un método de test.

Paso 16 — Compilar la solución completa

En la terminal de `1.5-buenas-practicas`, donde está `AceriaApp.sln`, ejecuta `dotnet build AceriaApp.sln -c Release`. Comprueba que compilan los 7 proyectos acumulados y que el resumen muestra **0 errores**.

Ante un error, identifica primero archivo y línea; localiza el marcador `INICIO PASO` responsable antes de intentar crear una clase nueva. Conserva la salida de compilación y verifica que los proyectos heredados no han cambiado respecto a `1.4-organizacion-solucion`.

compilar la solución completa verifica que todos los proyectos, incluidos los nuevos, se resuelven correctamente.

Errores comunes:

- Tener la API en ejecución: el archivo .dll está bloqueado y la compilación falla.
- No restaurar paquetes: dotnet build restaura por defecto, pero si hay un error de red, hay que ejecutar dotnet restore primero.
- Usar una versión distinta del SDK: dotnet --version debe devolver 8.x.x.

Verificación: la salida debe mostrar Build succeeded. con 0 Error(s).

Paso 17 — Ejecutar todos los tests del punto 1.5

Abre el **Explorador de pruebas** de Visual Studio o una terminal en la carpeta de la solución 1.5. Ejecuta `dotnet test tests/AceriaApp.BuenasPracticas.Tests/AceriaApp.BuenasPracticas.Tests.csproj -c Release` para comprobar únicamente los tests nuevos de la etapa.

Verifica que pasan **5 pruebas** de `AceriaApp.BuenasPracticas.Tests` con 0 fallos; relaciona cada caso con `ValidadorBuenasPracticas` o `CatalogoPracticas`. Si falla algún test, consulta el primer mensaje de aserción y examina los bloques activados en los pasos 14 y 15 sin cambiar valores esperados a ciegas.

ejecutar los tests del punto 1.5 verifica que las clases de buenas prácticas funcionan correctamente.

Errores comunes:

- Ejecutar dotnet test sin especificar el proyecto: ejecuta también los tests de los puntos anteriores, lo que alarga el tiempo.
- No compilar antes: dotnet test compila por defecto, pero si hay errores, los tests no se ejecutan.
- Tener la API en ejecución: el puerto puede estar ocupado.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 5, Skipped: 0, Total: 5.

Paso 18 — Ejecutar todos los tests de la solución

En la carpeta de `AceriaApp.sln` ejecuta `dotnet test AceriaApp.sln -c Release --no-build` después del build Release del paso 16. Comprueba que pasan **23 pruebas acumuladas**: 18 heredadas más 5 de la etapa 1.5.

Si no se encuentran los ensamblados, vuelve a compilar en Release y repite el comando sin mezclar resultados Debug y Release. Guarda el resumen: los tests heredados deben seguir pasando, pues una nueva etapa no puede romper las capacidades previas de AceriaApp.

ejecutar todos los tests de la solución verifica que los tests de los puntos 1.1, 1.2, 1.3, 1.4 y 1.5 pasan conjuntamente.

Errores comunes:

- No haber compilado antes: --no-build falla si no hay artefactos.
- Tener tests de otros módulos: la solución solo contiene los proyectos de este punto.
- Modificar los tests de los puntos anteriores: viola la regla de no modificar archivos del punto anterior.

Verificación: la salida debe mostrar Passed! - Failed: 0, Passed: 23, Skipped: 0, Total: 23.

Paso 19 — Revisar el documento preparado: README.md

Abre el archivo `README.md` de la carpeta `1.5-buenas-practicas`, no el del punto `1.4-organizacion-solucion`. Localiza en el documento el objetivo de la etapa y los archivos `Practica`, `ValidadorBuenasPracticas`, `CatalogoPracticas` y `AceriaApp.BuenasPracticas.Tests`.

Comprueba que las rutas de ejecución y tests mencionadas corresponden a los proyectos físicos de esta etapa. Anota qué información nueva aporta este README respecto al anterior; no lo sobrescribas con la versión heredada.

```
# Punto 1.5 — Buenas prácticas

## Objetivo
Comprender las buenas prácticas aplicables al desarrollo de microservicios y añadir al proyecto las clases de
validación de buenas prácticas. Cerrar el módulo 1 con el resumen del estado del proyecto.

## Estado del proyecto
- Solución `AceriaApp.sln` con seis proyectos: `AceriaApp.Monolito.Api`, `AceriaApp.Monolito.Tests`,
`AceriaApp.Metricas.Tests`, `AceriaApp.Diseno.Tests`, `AceriaApp.Organizacion.Tests` y
`AceriaApp.BuenasPracticas.Tests`.
- Documento `docs/buenas-practicas.md` con las cuatro categorías y el cierre del módulo 1.
- Clases `Practica`, `ValidadorBuenasPracticas` y `CatalogoPracticas` en `src/AceriaApp.Monolito.Api/
BuenasPracticas/`.
- Cinco tests nuevos en `tests/AceriaApp.BuenasPracticas.Tests`.

## Cómo ejecutarlo
1. Abrir `AceriaApp.sln` en Visual Studio 2026.
2. Verificar que SQL Server Express está arrancado.
3. Ejecutar `dotnet ef database update` en `src\AceriaApp.Monolito.Api`.
4. Ejecutar `dotnet run` en `src\AceriaApp.Monolito.Api`.
5. Navegar a `https://localhost:7xxx/swagger`.
6. Ejecutar `dotnet test AceriaApp.sln` para verificar los tests.

## Qué se ha añadido respecto al punto anterior
- Documento `docs/buenas-practicas.md`.
- Carpeta `BuenasPracticas` en el proyecto monolítico con tres clases nuevas.
- Proyecto `tests/AceriaApp.BuenasPracticas.Tests` con cinco tests nuevos.
- Cierre del módulo 1 con el resumen del estado del proyecto.
- No se ha modificado ningún archivo del punto 1.4.

## Documentación
- [ADR-001](docs/ADR-001-monolito-vs-microservicios.md) (heredado del punto 1.1).
- [ADR-002](docs/ADR-002-metricas-acoplamiento.md) (heredado del punto 1.2).
- [ADR-003](docs/ADR-003-principios-diseno.md) (heredado del punto 1.3).
- [ADR-004](docs/ADR-004-organizacion-solucion.md) (heredado del punto 1.4).
- [Análisis de ventajas e inconvenientes](docs/analisis-ventajas-inconvenientes.md) (heredado del punto 1.2).
- [Principios de diseño](docs/principios-diseno.md) (heredado del punto 1.3).
- [Organización de la solución](docs/organizacion-solucion.md) (heredado del punto 1.4).
- [Buenas prácticas](docs/buenas-practicas.md).
```

el README del punto 1.5 explica el objetivo, el estado, la ejecución y los añadidos. No modifica el README del punto 1.4.

Errores comunes:

- Modificar el README del punto 1.4: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Qué se ha añadido respecto al punto anterior: pierde la trazabilidad.
- Escribir el README en la carpeta del punto 1.4: rompe la separación.

Verificación: el archivo README.md existe en la carpeta del punto 1.5 con las cinco secciones.

Paso 20 — Revisar el documento preparado: ADR-005-buenas-practicas.md

Abre `docs/ADR-005-buenas-practicas.md` dentro de `1.5-buenas-practicas` en el editor Markdown; el archivo se entrega preparado. Lee Estado, Contexto, Decisión, Alternativas y Consecuencias, vinculando la decisión con buenas prácticas declaradas de la solución.

Identifica un resultado de los tests de 1.5 que apoye la decisión y una limitación que el ejemplo no demuestra. Comprueba que el ADR del punto anterior permanece presente e idéntico: cada decisión arquitectónica se documenta por separado.

```
# ADR-005: Adopción de buenas prácticas y cierre del módulo 1

## Estado
Aceptado

## Contexto
El proyecto AceriaApp ha completado los puntos 1.1 a 1.4 del módulo 1. Es necesario cerrar el módulo con la
adopción explícita de buenas prácticas y con una auditoría automática de su aplicación.

## Decisión
Se adoptan las buenas prácticas de contrato, operación, comunicación y evolución. Se implementa un validador
de declaraciones que comprueba categorías y el estado declarado, sin inspeccionar la implementación. El
módulo 1 se cierra con el resumen del estado del proyecto.

## Alternativas consideradas
- Adoptar las buenas prácticas sin auditoría: descartado por falta de trazabilidad.
- Adoptar solo algunas categorías: descartado por incoherencia.

## Consecuencias
- Positivas: auditoría automática, trazabilidad, preparación para el módulo 2.
- Negativas: el validador puede requerir ajuste de categorías.
- Mitigación: revisar las categorías al final de cada módulo.
```

el ADR-005 documenta la decisión de adoptar buenas prácticas y de cerrar el módulo 1. Es un artefacto nuevo del punto 1.5 que no modifica los ADR existentes.

Errores comunes:

- Modificar los ADR anteriores: viola la regla de no modificar archivos del punto anterior.
- Omitir la sección Alternativas consideradas: el ADR pierde valor.
- Nombrar el archivo sin el prefijo ADR-005: rompe la convención.

Verificación: el archivo docs\ADR-005-buenas-practicas.md existe con las cinco secciones.

Paso 21 — Verificar que no se ha modificado ningún archivo del punto 1.4

En tu copia del repositorio localiza las carpetas `docs` de `1.4-organizacion-solucion` y `1.5-buenas-practicas`. Selecciona un fichero **heredado** presente en ambas carpetas y compara su contenido con `fc /L` en Windows o un hash SHA-256.

Los documentos heredados deben ser idénticos; el `README.md` de 1.5 puede ser diferente porque explica la etapa actual. Registra el resultado y no modifiques los originales de `1.4-organizacion-solucion` para forzar igualdad.

La comparación con fc verifica que los documentos del punto 1.4 no se han modificado.

Errores comunes:

- Usar comp en lugar de fc: comp compara binarios y no muestra diferencias de texto.
- Olvidar el flag /L: fc compara en modo ASCII y muestra líneas largas sin ajuste.
- Comparar archivos de carpetas equivocadas: la comparación no es válida.

Verificación: los dos documentos del punto 1.4 no muestran diferencias.

Paso 22 — Verificar la estructura del punto 1.5

Desde `M01_INTRODUCCION_MICROSERVICIOS/PROYECTO` ejecuta `tree /F 1.5-buenas-practicas` en una terminal Windows. Comprueba en `src/AceriaApp.Monolito.Api/BuenasPracticas` los tres tipos `Practica.cs`, `ValidadorBuenasPracticas.cs` y `CatalogoPracticas.cs`.

En `tests/AceriaApp.BuenasPracticas.Tests` localiza las pruebas del validador y catálogo; en `docs` localiza `buenas-practicas.md`, `ADR-005-buenas-practicas.md` y los documentos heredados. Distingue los archivos fuente de las carpetas temporales `bin` y `obj`, que no deben incorporarse como entregables.

La verificación estructural confirma que el punto 1.5 añade archivos nuevos sin modificar los existentes.

Errores comunes:

- Crear carpetas auxiliares que no pertenecen a la estructura: rompe la convención.
- Modificar las carpetas del punto 1.4: viola la regla de no modificar.
- Omitir la carpeta BuenasPracticas: los tests no encuentran las clases.

Verificación: la estructura contiene los archivos nuevos y conserva los del punto 1.4.

Paso 23 — Revisar el documento preparado: .gitignore

Abre el `.gitignore` de la carpeta `1.5-buenas-practicas` en un editor que muestre ficheros ocultos. Comprueba que conserva las exclusiones `bin/`, `obj/`, `.vs/`, resultados y archivos de usuario sin ignorar el código fuente.

Ejecuta `git status --short` en tu copia personal tras compilar y confirma que no aparecen binarios temporales. No sustituyas el fichero ya incluido por una plantilla ajena; verifica que conserva la política del punto anterior.

el .gitignore se hereda del punto 1.4 sin modificaciones. La verificación confirma que no se ha tocado.

Errores comunes:

- Modificar el .gitignore copiado del punto 1.4: viola la regla de no modificar archivos del punto anterior.
- Eliminar alguna línea: el repositorio se llena de artefactos.
- Crear el .gitignore con otro nombre: el archivo no se reconoce.

Verificación: el .gitignore existe y contiene las seis líneas sin modificaciones.

Paso 24 — Hacer el commit del punto 1.5 y verificar el estado final del módulo 1

Desde tu copia personal (no la rama del profesor) ejecuta `git status` para identificar los cambios de `1.5-buenas-practicas`. Revisa `git diff --stat` y prepara con `git add` solo archivos de esta etapa, sin incluir carpetas compiladas ni secretos.

Realiza el commit del punto 1.5 con un mensaje descriptivo de buenas prácticas declaradas de la solución; no sobrescribas los commits de etapas previas. Comprueba `git log -1 --oneline`, `git status` y conserva el resultado de los 23 tests acumulados como evidencia final.

El commit del punto 1.5 cierra el módulo 1. Los cinco commits consecutivos permiten comparar el estado del proyecto entre puntos consecutivos y entre módulos completos.

Errores comunes:

- No tener configurado user.name y user.email: git commit falla.
- Añadir la carpeta bin/ y obj/ al commit: el .gitignore lo evita.
- Hacer commit antes de git add .: no hay cambios en el índice.

Verificación: git log --oneline muestra cinco commits: Punto 1.1, Punto 1.2, Punto 1.3, Punto 1.4 y Punto 1.5. git status muestra nothing to commit, working tree clean.

Nota de trazabilidad: en una descarga del repositorio puede no haber cambios nuevos que confirmar. Antes de ejecutar `git commit`, usar `git status` y confirmar únicamente los archivos efectivamente modificados durante el recorrido. No ejecutar `git init` dentro de cada punto: la raíz del repositorio ya contiene el historial.

Reto de ampliación: solución guiada

Problema a resolver: añadir un método Resumen a CatalogoPracticas que devuelva el número de prácticas aplicadas, el número de pendientes y el porcentaje de aplicación, y un test que verifique el cálculo cuando hay tres aplicadas y dos pendientes.

Etapa 1 — En CatalogoPracticas.cs, añadir el siguiente record al final del archivo:

```
public record ResumenPracticas(int Aplicadas, int Pendientes, double PorcentajeAplicacion);
```

Etapa 2 — Explicar línea por línea:

- **Línea 1:** `public record ResumenPracticas(` — declara un contrato inmutable.
- **Línea 2:** `int Aplicadas,` — número de prácticas aplicadas.
- **Línea 3:** `int Pendientes,` — número de prácticas pendientes.
- **Línea 4:** `double PorcentajeAplicacion);` — porcentaje de aplicación.

Etapa 3 — En CatalogoPracticas.cs, añadir el siguiente método dentro de la clase:

```
public ResumenPracticas Resumen()
{
    var aplicadas = _practicas.Count(p => p.EstaAplicada);
    var pendientes = _practicas.Count - aplicadas;
    var total = _practicas.Count;
    var porcentaje = total == 0 ? 0 : (double)aplicadas / total * 100;
    return new ResumenPracticas(aplicadas, pendientes, porcentaje);
}
```

Etapa 4 — Explicar línea por línea:

- **Línea 1:** `public ResumenPracticas Resumen()` — declara el método.
- **Línea 2:** `{` — abre el cuerpo.
- **Línea 3:** `var aplicadas = _practicas.Count(p => p.EstaAplicada);` — cuenta las aplicadas.
- **Línea 4:** `var pendientes = _practicas.Count - aplicadas;` — calcula las pendientes por diferencia.
- **Línea 5:** `var total = _practicas.Count;` — obtiene el total.
- **Línea 6:** `var porcentaje = total == 0 ? 0 : (double)aplicadas / total * 100;` — calcula el porcentaje con protección contra división por cero.
- **Línea 7:** `return new ResumenPracticas(aplicadas, pendientes, porcentaje);` — devuelve el resumen.
- **Línea 8:** `}` — cierra el cuerpo.

Etapa 5 — Guardar el archivo con Ctrl+S.

Etapa 6 — En CatalogoPracticasTests.cs, añadir el siguiente test:

```
[Fact]
public void Resumen_ConTresAplicadasYDosPendientes_CalculaPorcentaje()
{
    var practicas = new List<Practica>
    {
        new() { Codigo = "C-01", Categoria = "Contrato", EstaAplicada = true },
        new() { Codigo = "C-02", Categoria = "Contrato", EstaAplicada = true },
        new() { Codigo = "O-01", Categoria = "Operación", EstaAplicada = true },
        new() { Codigo = "M-01", Categoria = "Comunicación", EstaAplicada = false },
        new() { Codigo = "E-01", Categoria = "Evolución", EstaAplicada = false }
    };
    var catalogo = new CatalogoPracticas(practicas);

    var resumen = catalogo.Resumen();

    resumen.Aplicadas.Should().Be(3);
    resumen.Pendientes.Should().Be(2);
    resumen.PorcentajeAplicacion.Should().Be(60.0);
}
```

Etapa 7 — Explicar línea por línea:

- **Línea 1:** `[Fact]` — marca el método como test de xUnit.
- **Línea 2:** `public void Resumen_ConTresAplicadasYDosPendientes_CalculaPorcentaje()` — declara el test.
- **Línea 3:** `{` — abre el cuerpo.
- **Línea 4:** `var practicas = new List<Practica>` — crea la lista.
- **Línea 5:** `{` — abre el inicializador.
- **Línea 6:** `new() { Codigo = "C-01", ..., EstaAplicada = true },` — primera práctica aplicada.
- **Línea 7:** `new() { Codigo = "C-02", ..., EstaAplicada = true },` — segunda práctica aplicada.
- **Línea 8:** `new() { Codigo = "O-01", ..., EstaAplicada = true },` — tercera práctica aplicada.
- **Línea 9:** `new() { Codigo = "M-01", ..., EstaAplicada = false },` — primera práctica pendiente.
- **Línea 10:** `new() { Codigo = "E-01", ..., EstaAplicada = false }` — segunda práctica pendiente.
- **Línea 11:** `};` — cierra el inicializador.
- **Línea 12:** `var catalogo = new CatalogoPracticas(practicas);` — instancia el catálogo.
- **Línea 13:** `var resumen = catalogo.Resumen();` — invoca el método.
- **Línea 14:** `resumen.Aplicadas.Should().Be(3);` — verifica las aplicadas.
- **Línea 15:** `resumen.Pendientes.Should().Be(2);` — verifica las pendientes.
- **Línea 16:** `resumen.PorcentajeAplicacion.Should().Be(60.0);` — verifica el porcentaje.
- **Línea 17:** `}` — cierra el cuerpo.

Etapa 8 — Ejecutar dotnet test tests\AceriaApp.BuenasPracticas.Tests\AceriaApp.BuenasPracticas.Tests.csproj y verificar que pasan los seis tests.

Etapa 9 — Verificar que el método Resumen no modifica ningún archivo del punto 1.4.

Etapa 10 — Hacer commit con git commit -m "Reto 1.5: método Resumen en CatalogoPracticas con cálculo de porcentaje".

Verificación: la salida de dotnet test muestra Passed: 6.

Resultado esperado

Al finalizar este punto, el alumno debe tener:

- La carpeta modulo-1-introduccion-microservicios/1.5-buenas-practicas/ con la solución AceriaApp.sln que contiene siete proyectos.
- El documento docs/buenas-practicas.md con las cuatro categorías y el cierre del módulo 1.
- El documento docs/ADR-005-buenas-practicas.md con la decisión sobre buenas prácticas.
- Las clases Practica, ValidadorBuenasPracticas y CatalogoPracticas en src/AceriaApp.Monolito.Api/BuenasPracticas/.
- El proyecto tests/AceriaApp.BuenasPracticas.Tests con cinco tests (seis tras el reto).
- Los tests de los puntos 1.1 a 1.5 pasando conjuntamente con dotnet test AceriaApp.sln.
- Los ADR-001 a ADR-004 del punto 1.4 sin modificar.
- Cinco commits en Git: del punto 1.1 al 1.5.
- El módulo 1 cerrado con el resumen del estado del proyecto en docs/buenas-practicas.md.

Conclusión y enlace al siguiente punto

El punto 1.5 ha cerrado el módulo 1 con la adopción explícita de buenas prácticas de contrato, operación, comunicación y evolución. El proyecto AceriaApp dispone ahora de un monolito modular con API Minimal, persistencia con EF Core 8.0.0 y SQL Server Express, documentación OpenAPI, métricas de acoplamiento, validador de límites de servicio, validador de convenciones de nombres, validador de buenas prácticas y 23 tests unitarios acumulados. El módulo 1 queda cerrado con el resumen del estado del proyecto en docs/buenas-practicas.md. Esta base permite avanzar hacia el módulo 2, donde se estudiará el desarrollo de APIs REST con ASP.NET Core: diseño de APIs, recursos y verbos HTTP, validación de peticiones y documentación OpenAPI.

Fin del punto 1.5 y del módulo 1.